



KALYX

COMPLETE SOURCE CODE REPOSITORY

Official technical submission document containing all source code modules for the FastAPI backend, Next.js frontend, and multi-agent LangGraph workflow engine.

SUBMISSION FOR: Hackathon Evaluation Committee

LANGUAGES: TypeScript, Python, Tailwind CSS, HTML

TEAM: KALYX Core Engineers

DATE: June 2026

File: backend/app/database.py

```
1 | import os
2 | from sqlalchemy import create_engine
3 | from sqlalchemy.orm import declarative_base, sessionmaker
4 | from dotenv import load_dotenv
5 |
6 | # Load env variables
7 | load_dotenv()
8 |
9 | DATABASE_URL = os.getenv("DATABASE_URL")
10 |
11 | # SQLite fallback path
12 | if not DATABASE_URL or DATABASE_URL.strip() == "":
13 |     BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
14 |     sqlite_path = os.path.join(BASE_DIR, "kalyx.db")
15 |     DATABASE_URL = f"sqlite:/// {sqlite_path}"
16 |     print(f"[Database] DATABASE_URL not set. Falling back to SQLite: {DATABASE_URL}")
17 | else:
18 |     print(f"[Database] Connecting to database: {DATABASE_URL.split('@')[-1]}")
19 |
20 | # Configuration arguments for engine
21 | connect_args = {}
22 | if DATABASE_URL.startswith("sqlite"):
23 |     # SQLite-specific multi-threading safety argument
24 |     connect_args = {"check_same_thread": False}
25 |
26 | # Create SQLAlchemy engine
27 | engine = create_engine(DATABASE_URL, connect_args=connect_args, echo=False)
28 |
29 | # Create SessionLocal
30 | SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
31 |
32 | # Declarative base class
33 | Base = declarative_base()
34 |
35 | # Context manager for DB sessions
36 | def get_db():
37 |     db = SessionLocal()
38 |     try:
39 |         yield db
40 |     finally:
41 |         db.close()
42 |
```

File: backend/app/models.py

```
1 | import datetime
2 | from sqlalchemy import Column, Integer, String, Text, ForeignKey, Float, DateTime, JSON, Boolean, Table
3 | from sqlalchemy.orm import relationship
4 | from app.database import Base
5 |
6 | class User(Base):
7 |     __tablename__ = "users"
8 |
9 |     id = Column(Integer, primary_key=True, index=True)
10 |    email = Column(String, unique=True, index=True, nullable=False)
11 |    username = Column(String, unique=True, index=True, nullable=True)
12 |    hashed_password = Column(String, nullable=False)
13 |    full_name = Column(String, nullable=True)
14 |    created_at = Column(DateTime, default=datetime.datetime.utcnow)
15 |
16 |    # Relationships
17 |    courses = relationship("Course", back_populates="user", cascade="all, delete-orphan")
18 |
19 | class Course(Base):
20 |     __tablename__ = "courses"
21 |
22 |     id = Column(Integer, primary_key=True, index=True)
23 |     user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
24 |     title = Column(String, nullable=False)
25 |     description = Column(Text, nullable=True)
26 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
27 |
28 |     # Relationships
29 |     user = relationship("User", back_populates="courses")
30 |     syllabi = relationship("Syllabus", back_populates="course", cascade="all, delete-orphan")
31 |     curriculum_analyses = relationship("CurriculumAnalysis", back_populates="course", cascade="all,
delete-orphan")
32 |     learning_outcomes = relationship("LearningOutcome", back_populates="course", cascade="all,
delete-orphan")
33 |     slides = relationship("GeneratedSlide", back_populates="course", cascade="all, delete-orphan")
34 |     notes = relationship("InstructorNote", back_populates="course", cascade="all, delete-orphan")
35 |     assessments = relationship("Assessment", back_populates="course", cascade="all, delete-orphan")
36 |     readiness_scores = relationship("ReadinessScore", back_populates="course", cascade="all,
delete-orphan")
37 |     uploaded_documents = relationship("UploadedDocument", back_populates="course", cascade="all,
delete-orphan")
38 |     personalization = relationship("PersonalizationProfile", back_populates="course", uselist=False,
cascade="all, delete-orphan")
39 |     exports = relationship("PptExport", back_populates="course", cascade="all, delete-orphan")
40 |
41 | class Syllabus(Base):
42 |     __tablename__ = "syllabi"
43 |
44 |     id = Column(Integer, primary_key=True, index=True)
45 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
46 |     raw_text = Column(Text, nullable=False)
47 |     file_name = Column(String, nullable=True)
48 |     file_path = Column(String, nullable=True)
49 |     uploaded_at = Column(DateTime, default=datetime.datetime.utcnow)
50 |
51 |     course = relationship("Course", back_populates="syllabi")
52 |
53 | class CurriculumAnalysis(Base):
54 |     __tablename__ = "curriculum_analysis"
55 |
56 |     id = Column(Integer, primary_key=True, index=True)
57 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
```

```

58 | curriculum_map = Column(JSON, nullable=True) # Structure: Modules, topics, subtopics
59 | gap_analysis = Column(JSON, nullable=True) # Detected internal content/topic gaps
60 | industry_gap_report = Column(JSON, nullable=True) # Modern trends comparison
61 | pipeline_telemetry = Column(JSON, nullable=True) # Agent execution duration and timestamp telemetry
62 | created_at = Column(DateTime, default=datetime.datetime.utcnow)
63 |
64 | course = relationship("Course", back_populates="curriculum_analyses")
65 |
66 | class LearningOutcome(Base):
67 |     __tablename__ = "learning_outcomes"
68 |
69 |     id = Column(Integer, primary_key=True, index=True)
70 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
71 |     outcome_text = Column(Text, nullable=False)
72 |     bloom_level = Column(String, nullable=False) # Remembering, Understanding, Applying, etc.
73 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
74 |
75 |     course = relationship("Course", back_populates="learning_outcomes")
76 |     assessments = relationship("Assessment", back_populates="learning_outcome")
77 |
78 | class GeneratedSlide(Base):
79 |     __tablename__ = "generated_slides"
80 |
81 |     id = Column(Integer, primary_key=True, index=True)
82 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
83 |     slide_index = Column(Integer, nullable=False)
84 |     title = Column(String, nullable=False)
85 |     content = Column(JSON, nullable=False) # List of bullet points / section details
86 |     suggested_visuals = Column(Text, nullable=True)
87 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
88 |
89 |     course = relationship("Course", back_populates="slides")
90 |
91 | class InstructorNote(Base):
92 |     __tablename__ = "instructor_notes"
93 |
94 |     id = Column(Integer, primary_key=True, index=True)
95 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
96 |     slide_index = Column(Integer, nullable=False)
97 |     talking_points = Column(JSON, nullable=False) # List of speaker notes
98 |     teaching_tips = Column(Text, nullable=True)
99 |     examples = Column(JSON, nullable=True) # Clarifying examples
100 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
101 |
102 |     course = relationship("Course", back_populates="notes")
103 |
104 | class Assessment(Base):
105 |     __tablename__ = "assessments"
106 |
107 |     id = Column(Integer, primary_key=True, index=True)
108 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
109 |     learning_outcome_id = Column(Integer, ForeignKey("learning_outcomes.id"), nullable=True)
110 |     question_text = Column(Text, nullable=False)
111 |     question_type = Column(String, nullable=False) # MCQ, Short Answer, Long Answer, Viva
112 |     options = Column(JSON, nullable=True) # JSON List of options if MCQ
113 |     correct_answer = Column(Text, nullable=True)
114 |     bloom_level = Column(String, nullable=False) # Bloom taxonomy tier
115 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
116 |
117 |     course = relationship("Course", back_populates="assessments")
118 |     learning_outcome = relationship("LearningOutcome", back_populates="assessments")
119 |
120 | class ReadinessScore(Base):
121 |     __tablename__ = "readiness_scores"
122 |
123 |     id = Column(Integer, primary_key=True, index=True)

```

```

124 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
125 |     score = Column(Float, nullable=False)
126 |     completeness = Column(Float, nullable=False)
127 |     outcome_coverage = Column(Float, nullable=False)
128 |     assessment_quality = Column(Float, nullable=False)
129 |     bloom_coverage = Column(Float, nullable=False)
130 |     industry_relevance = Column(Float, nullable=False)
131 |     breakdown = Column(JSON, nullable=True) # Detailed comments & visual metrics
132 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
133 |
134 |     course = relationship("Course", back_populates="readiness_scores")
135 |
136 | class UploadedDocument(Base):
137 |     __tablename__ = "uploaded_documents"
138 |
139 |     id = Column(Integer, primary_key=True, index=True)
140 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
141 |     file_name = Column(String, nullable=False)
142 |     file_path = Column(String, nullable=False)
143 |     document_type = Column(String, nullable=False) # RAG_SOURCE, SYLLABUS
144 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
145 |
146 |     course = relationship("Course", back_populates="uploaded_documents")
147 |     embeddings = relationship("Embedding", back_populates="document", cascade="all, delete-orphan")
148 |
149 | class Embedding(Base):
150 |     __tablename__ = "embeddings"
151 |
152 |     id = Column(Integer, primary_key=True, index=True)
153 |     document_id = Column(Integer, ForeignKey("uploaded_documents.id"), nullable=False)
154 |     chunk_text = Column(Text, nullable=False)
155 |     embedding = Column(JSON, nullable=False) # Stored as a list of floats (supports pgvector mapping &
SQLite fallback)
156 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
157 |
158 |     document = relationship("UploadedDocument", back_populates="embeddings")
159 |
160 | class PersonalizationProfile(Base):
161 |     __tablename__ = "instructor_personalization"
162 |
163 |     id = Column(Integer, primary_key=True, index=True)
164 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
165 |     profile = Column(JSON, nullable=False) # Tone, slide style preference, custom instructions
166 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
167 |
168 |     course = relationship("Course", back_populates="personalization")
169 |
170 | class PptExport(Base):
171 |     __tablename__ = "ppt_exports"
172 |
173 |     id = Column(Integer, primary_key=True, index=True)
174 |     course_id = Column(Integer, ForeignKey("courses.id"), nullable=False)
175 |     file_name = Column(String, nullable=False)
176 |     file_path = Column(String, nullable=False)
177 |     created_at = Column(DateTime, default=datetime.datetime.utcnow)
178 |
179 |     course = relationship("Course", back_populates="exports")
180 |

```

File: backend/app/security.py

```
1 | import os
2 | import time
3 | import jwt
4 | from collections import defaultdict
5 | from fastapi import Request, HTTPException, status
6 |
7 | JWT_SECRET = os.getenv("JWT_SECRET", "hackathon_super_secret_key_kalyx_2026")
8 | JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")
9 |
10 | class RateLimiter:
11 |     def __init__(self):
12 |         # Maps limit_key -> list of timestamps
13 |         self.history = defaultdict(list)
14 |
15 |     def check_rate_limit(self, key: str, limit: int, window: int = 60):
16 |         now = time.time()
17 |         # Clean up timestamps older than the window
18 |         self.history[key] = [t for t in self.history[key] if now - t < window]
19 |         if len(self.history[key]) >= limit:
20 |             raise HTTPException(
21 |                 status_code=status.HTTP_429_TOO_MANY_REQUESTS,
22 |                 detail={
23 |                     "error": "Too many requests. Please try again later.",
24 |                     "detail": "Too many requests. Please try again later."
25 |                 }
26 |             )
27 |         self.history[key].append(now)
28 |
29 | limiter = RateLimiter()
30 |
31 | def get_user_identifier(request: Request) -> str:
32 |     # 1. Try Authorization header
33 |     auth_header = request.headers.get("Authorization")
34 |     token = None
35 |     if auth_header and auth_header.startswith("Bearer "):
36 |         token = auth_header.split(" ")[1]
37 |
38 |     # 2. Try query parameter
39 |     if not token:
40 |         token = request.query_params.get("token")
41 |
42 |     if not token:
43 |         # Fallback to IP address if no token is provided
44 |         return f"ip_{request.client.host if request.client else 'unknown'}"
45 |
46 |     try:
47 |         payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
48 |         email = payload.get("sub")
49 |         if email:
50 |             return f"user_{email}"
51 |     except Exception:
52 |         pass
53 |
54 |     return f"ip_{request.client.host if request.client else 'unknown'}"
55 |
56 | # Dependencies for rate limiting
57 | def limit_login(request: Request):
58 |     ip = request.client.host if request.client else "unknown"
59 |     limiter.check_rate_limit(f"login_{ip}", 10, 60)
60 |
61 | def limit_signup(request: Request):
62 |     ip = request.client.host if request.client else "unknown"
```

```
63 |     limiter.check_rate_limit(f"signup_{ip}", 5, 60)
64 |
65 | def limit_course_creation(request: Request):
66 |     key = get_user_identifier(request)
67 |     limiter.check_rate_limit(f"course_create_{key}", 20, 60)
68 |
69 | def limit_syllabus_upload(request: Request):
70 |     key = get_user_identifier(request)
71 |     limiter.check_rate_limit(f"syllabus_upload_{key}", 5, 60)
72 |
73 | def limit_analysis_generation(request: Request):
74 |     key = get_user_identifier(request)
75 |     limiter.check_rate_limit(f"analysis_gen_{key}", 5, 60)
76 |
```

File: backend/app/agents/workflow.py

```
1 | import os
2 | import json
3 | import logging
4 | from typing import Dict, Any, List, Optional
5 | from pydantic import BaseModel, Field
6 | from langgraph.graph import StateGraph, END
7 | from app.agents.state import SharedState
8 | from app.services.rag_service import retrieve_relevant_chunks
9 | from app.database import SessionLocal
10 |
11 | logger = logging.getLogger("kalyx.agents")
12 |
13 | # Helper to strip unsupported additionalProperties for Developer API mode compatibility
14 | def strip_additional_properties(schema: Any) -> Any:
15 |     if isinstance(schema, dict):
16 |         schema.pop("additionalProperties", None)
17 |         schema.pop("extraProperties", None)
18 |         for k, v in list(schema.items()):
19 |             schema[k] = strip_additional_properties(v)
20 |     elif isinstance(schema, list):
21 |         return [strip_additional_properties(item) for item in schema]
22 |     return schema
23 |
24 | # Helper to recursively inline references from $defs to simplify schema for LLMs
25 | def resolve_defs(schema: Any, defs: Optional[Dict[str, Any]] = None) -> Any:
26 |     if defs is None and isinstance(schema, dict):
27 |         defs = schema.get("$defs", {})
28 |
29 |     if isinstance(schema, dict):
30 |         if "$ref" in schema:
31 |             ref_path = schema["$ref"]
32 |             def_name = ref_path.split("/")[-1]
33 |             if def_name in defs:
34 |                 resolved = resolve_defs(defs[def_name], defs)
35 |                 # Merge keys other than $ref if any
36 |                 for k, v in schema.items():
37 |                     if k != "$ref" and k not in resolved:
38 |                         resolved[k] = v
39 |                 return resolved
40 |             new_schema = {}
41 |             for k, v in schema.items():
42 |                 if k == "$defs":
43 |                     continue
44 |                 new_schema[k] = resolve_defs(v, defs)
45 |             return new_schema
46 |         elif isinstance(schema, list):
47 |             return [resolve_defs(item, defs) for item in schema]
48 |     return schema
49 |
50 | # Helper to run LLM prompts safely using Groq API via OpenAI client
51 | def call_llm(system_prompt: str, user_prompt: str, response_format: str = "json", response_schema: Any =
None) -> str:
52 |     api_key = os.getenv("GROQ_API_KEY")
53 |     if not api_key or api_key.strip() == "" or api_key.startswith("your_"):
54 |         raise ValueError("GROQ_API_KEY environment variable is not set. Please configure it in your .env
file.")
55 |
56 |     from openai import OpenAI
57 |     client = OpenAI(api_key=api_key, base_url="https://api.groq.com/openai/v1")
58 |
59 |     # Pre-process Pydantic models to strip unsupported additionalProperties and resolve defs
60 |     if response_schema is not None:
```



```

61 |         if isinstance(response_schema, type) and issubclass(response_schema, BaseModel):
62 |             schema_dict = response_schema.model_json_schema()
63 |             schema_dict = strip_additional_properties(schema_dict)
64 |             schema_dict = resolve_defs(schema_dict)
65 |             response_schema = schema_dict
66 |
67 |         import time
68 |         backoff_times = [2, 5, 10, 20, 30, 45]
69 |         attempt = 0
70 |         while True:
71 |             try:
72 |                 sys_content = system_prompt
73 |                 if response_schema is not None:
74 |                     sys_content += f"\n\nYou MUST return a JSON response matching the required
structure/schema. Output actual data values for the fields defined in the schema, NOT the schema definition
itself.\n\nCRITICAL: Do NOT output the schema itself. Do NOT output keys like '$defs', 'properties', 'type',
'required', etc., unless they are part of the actual data. Output concrete values.\n\nStrict JSON Schema
target:\n{json.dumps(response_schema)}"
75 |                     elif response_format == "json":
76 |                         sys_content += "\n\nYou MUST return a JSON response. Do not output any markdown
formatting (like ``json ... ``) or conversational filler; return raw JSON only."
77 |
78 |                         messages = [
79 |                             {"role": "system", "content": sys_content},
80 |                             {"role": "user", "content": user_prompt}
81 |                         ]
82 |
83 |                         response = client.chat.completions.create(
84 |                             model=os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile"),
85 |                             messages=messages,
86 |                             response_format={"type": "json_object"} if (response_format == "json" or response_schema
is not None) else None,
87 |                             temperature=0.2
88 |                         )
89 |                         return response.choices[0].message.content
90 |             except Exception as e:
91 |                 err_str = str(e)
92 |                 is_transient = "429" in err_str or "503" in err_str or "rate limit" in err_str.lower()
93 |
94 |                 if is_transient and attempt < len(backoff_times):
95 |                     import re
96 |                     sleep_time = None
97 |                     try:
98 |                         err_str_lower = err_str.lower()
99 |                         if "try again in" in err_str_lower:
100 |                             match = re.search(r"try again in ([\d\.]+)(ms|s)", err_str_lower)
101 |                             if match:
102 |                                 val = float(match.group(1))
103 |                                 unit = match.group(2)
104 |                                 if unit == "ms":
105 |                                     sleep_time = (val / 1000.0) + 0.5
106 |                                 else:
107 |                                     sleep_time = val + 0.5
108 |                     except Exception:
109 |                         pass
110 |
111 |                     if sleep_time is None:
112 |                         sleep_time = backoff_times[attempt]
113 |
114 |                     logger.warning(f"[Groq API] Rate limit or transient error ({err_str}). Retrying in
{sleep_time:.2f}s...")
115 |                     time.sleep(sleep_time)
116 |                     attempt += 1
117 |                 else:
118 |                     logger.error(f"Error calling Groq API: {e}")
119 |                     raise e

```

```

120 |
121 |
122 |
123 | # PYDANTIC STRUCTURED OUT-SCHEMAS (Enforcing educational standards at runtime)
124 | class ModuleItem(BaseModel):
125 |     id: int
126 |     title: str
127 |     topics: List[str]
128 |
129 | class CurriculumMap(BaseModel):
130 |     course_title: str
131 |     modules: List[ModuleItem]
132 |     gaps: List[str]
133 |
134 | class LearningOutcomeItem(BaseModel):
135 |     id: int
136 |     text: str
137 |     bloom_level: str
138 |
139 | class LearningOutcomesList(BaseModel):
140 |     learning_outcomes: List[LearningOutcomeItem]
141 |
142 | class LessonItem(BaseModel):
143 |     week: int
144 |     module_id: int
145 |     title: str
146 |     objectives: str
147 |
148 | class CurriculumPlan(BaseModel):
149 |     duration_weeks: int
150 |     lesson_sequence: List[LessonItem]
151 |
152 | class SlideItem(BaseModel):
153 |     slide_index: int
154 |     title: str
155 |     content: List[str]
156 |     suggested_visuals: str
157 |
158 | class SlideDeck(BaseModel):
159 |     slides: List[SlideItem]
160 |
161 | class NoteItem(BaseModel):
162 |     slide_index: int = Field(..., description="The 1-based index of the slide this note corresponds to")
163 |     talking_points: List[str] = Field(..., description="At least 8 to 10 highly detailed, comprehensive
lecture talking points/explanations for this slide (MUST be at least 8 to 10 points)")
164 |     teaching_tips: str = Field(..., description="Detailed interactive pedagogy tips and whiteboard layout
guidelines")
165 |     examples: List[str] = Field(..., description="At least 5 to 6 concrete, distinct, real-world
clarifying examples/scenarios for this slide (MUST be at least 5 to 6 examples)")
166 |
167 | class InstructorNotesList(BaseModel):
168 |     notes: List[NoteItem] = Field(..., description="List of instructor notes, one for each slide in the
deck")
169 |
170 | class AssessmentItem(BaseModel):
171 |     question_text: str
172 |     question_type: str
173 |     options: Optional[List[str]] = None
174 |     correct_answer: Optional[str] = None
175 |     bloom_level: str
176 |     learning_outcome_id: int
177 |
178 | class AssessmentBank(BaseModel):
179 |     assessments: List[AssessmentItem]
180 |
181 | class BloomReport(BaseModel):

```

```

182 |     Remembering: int
183 |     Understanding: int
184 |     Applying: int
185 |     Analyzing: int
186 |     Evaluating: int
187 |     Creating: int
188 |     average_coverage: float
189 |     recommendation: str
190 |
191 | class ReadinessScore(BaseModel):
192 |     score: float
193 |     completeness: float
194 |     outcome_coverage: float
195 |     assessment_quality: float
196 |     bloom_coverage: float
197 |     industry_relevance: float
198 |     breakdown: Dict[str, str]
199 |
200 | class IndustryGapReport(BaseModel):
201 |     status: str
202 |     missing_topics: List[str]
203 |     recommendations: List[str]
204 |
205 | # Agent 1: Curriculum Analysis Agent
206 | def curriculum_analysis_agent(state: SharedState) -> SharedState:
207 |     state["logs"].append("Curriculum Analysis Agent started.")
208 |     state["current_agent"] = "Curriculum Analysis Agent"
209 |
210 |     if state.get("error_info"):
211 |         state["logs"].append("Curriculum Analysis Agent skipped due to previous error.")
212 |         return state
213 |
214 |     course_id = state.get("course_id")
215 |     rag_context = ""
216 |     if course_id:
217 |         try:
218 |             db = SessionLocal()
219 |             chunks = retrieve_relevant_chunks(db, course_id=course_id, query="syllabus course details
topics modules learning outcomes objectives", limit=8)
220 |             db.close()
221 |             if chunks:
222 |                 rag_context = "\n### RELEVANT SYLLABUS REFERENCE CHUNKS:\n" + "\n".join([f"-
{c['chunk_text']}" for c in chunks])
223 |             except Exception as e:
224 |                 logger.warning(f"RAG context search bypassed: {e}")
225 |
226 |         try:
227 |             # Self-healing warning detection
228 |             alert_context = ""
229 |             if any("Audit Alert" in log for log in state["logs"]):
230 |                 alert_context = "\n### ACCREDITATION AUDIT WARNING:\nYour previous curriculum plan had
insufficient higher-order Bloom levels. You MUST enrich the topics and structures to support advanced learning
outcomes mapped strictly to 'Evaluating' or 'Creating' cognitive levels to ensure curriculum balance."
231 |
232 |             sys_prompt = (
233 |                 "You are a Principal Curriculum Architect. "
234 |                 "Extract the core course title, structured modules, and comprehensive topics from the
syllabus. "
235 |                 "Identify internal content/topic gaps within the syllabus structure. "
236 |                 "Return JSON matching the schema."
237 |             )
238 |             user_prompt = f"Analyze the following
syllabus:\n{state['syllabus_text']}\n{rag_context}\n{alert_context}"
239 |
240 |             res = call_llm(sys_prompt, user_prompt, response_schema=CurriculumMap)
241 |             data = json.loads(res)

```

```

242 |
243 |         state["curriculum_map"] = data
244 |         state["logs"].append("Curriculum Analysis successfully processed via LLM.")
245 |     except Exception as e:
246 |         logger.error(f"Curriculum Analysis Error: {e}")
247 |         err_str = str(e)
248 |         err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
249 |         state["error_info"] = {
250 |             "status": "partial_generation",
251 |             "failed_agent": "Curriculum Analysis Agent",
252 |             "error_type": err_type,
253 |             "successful_agents": [],
254 |             "retry_recommended": True
255 |         }
256 |         state["logs"].append(f"Curriculum Analysis Agent failed: {err_str}")
257 |
258 |     return state
259 |
260 |
261 | # Agent 2: Learning Outcome Agent
262 | def learning_outcome_agent(state: SharedState) -> SharedState:
263 |     state["logs"].append("Learning Outcome Agent started.")
264 |     state["current_agent"] = "Learning Outcome Agent"
265 |
266 |     if state.get("error_info"):
267 |         state["logs"].append("Learning Outcome Agent skipped due to previous error.")
268 |         return state
269 |
270 |     try:
271 |         sys_prompt = (
272 |             "You are an Educational Standards Director. "
273 |             "Based on the provided curriculum map, generate clear, actionable, measurable learning
outcomes. "
274 |             "Each outcome must be mapped strictly to one of the six Bloom's Revised Taxonomy levels
(Remembering, Understanding, Applying, Analyzing, Evaluating, Creating). "
275 |             "Return JSON matching the schema."
276 |         )
277 |         user_prompt = f"Generate learning outcomes for the curriculum map:
{json.dumps(state['curriculum_map'])}"
278 |
279 |         res = call_llm(sys_prompt, user_prompt, response_schema=LearningOutcomesList)
280 |         data = json.loads(res)
281 |
282 |         outcomes_list = []
283 |         if isinstance(data, list):
284 |             outcomes_list = data
285 |         elif isinstance(data, dict):
286 |             outcomes_list = data.get("learning_outcomes", [])
287 |
288 |         state["learning_outcomes"] = [{"id": o.get("id"), "text": o.get("text"), "bloom_level":
o.get("bloom_level")} for o in outcomes_list if isinstance(o, dict)]
289 |         state["logs"].append("Learning Outcome Agent successfully processed via LLM.")
290 |     except Exception as e:
291 |         logger.error(f"Learning Outcome Error: {e}")
292 |         err_str = str(e)
293 |         err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
294 |         state["error_info"] = {
295 |             "status": "partial_generation",
296 |             "failed_agent": "Learning Outcome Agent",
297 |             "error_type": err_type,
298 |             "successful_agents": ["Curriculum Analysis Agent"],
299 |             "retry_recommended": True
300 |         }
301 |         state["logs"].append(f"Learning Outcome Agent failed: {err_str}")

```

```

302 |
303 |     return state
304 |
305 |
306 | # Agent 3: Curriculum Planning Agent
307 | def curriculum_planning_agent(state: SharedState) -> SharedState:
308 |     state["logs"].append("Curriculum Planning Agent started.")
309 |     state["current_agent"] = "Curriculum Planning Agent"
310 |
311 |     if state.get("error_info"):
312 |         state["logs"].append("Curriculum Planning Agent skipped due to previous error.")
313 |         return state
314 |
315 |     try:
316 |         sys_prompt = (
317 |             "You are an Academic Planning Expert. "
318 |             "Design a detailed, week-by-week lesson roadmap (sequence of lessons) of at least 25 to 30
detailed lessons/weeks. "
319 |             "Ensure it sequences all topics from the curriculum map logically and aligns with the
generated learning outcomes. "
320 |             "Return JSON matching the schema."
321 |         )
322 |         user_prompt_base = (
323 |             f"Create a weekly curriculum plan for curriculum map: {json.dumps(state['curriculum_map'])} "
324 |             f"and learning outcomes: {json.dumps(state['learning_outcomes'])}. "
325 |             f"CRITICAL REQUIREMENT: You MUST generate at least 25 to 30 distinct weeks in the
lesson_sequence. Break down topics and subtopics into multiple weeks if necessary."
326 |         )
327 |
328 |         attempts = 3
329 |         data = {}
330 |         for attempt in range(attempts):
331 |             feedback = ""
332 |             if attempt > 0:
333 |                 feedback = f"\n\nWARNING: In your previous attempt, you only generated
{len(data.get('lesson_sequence', []))} weeks. You MUST generate at least 25 to 30 weeks (lesson sequence items).
Please expand the topics and create more fine-grained weekly lessons."
334 |
335 |             res = call_llm(sys_prompt, user_prompt_base + feedback, response_schema=CurriculumPlan)
336 |             data = json.loads(res)
337 |
338 |             lesson_sequence = data.get("lesson_sequence", [])
339 |             if len(lesson_sequence) >= 25:
340 |                 break
341 |
342 |             state["curriculum_plan"] = data
343 |             state["logs"].append(f"Curriculum Planning Agent successfully processed via LLM with
{len(data.get('lesson_sequence', []))} weeks.")
344 |         except Exception as e:
345 |             logger.error(f"Curriculum Planning Error: {e}")
346 |             err_str = str(e)
347 |             err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
348 |             state["error_info"] = {
349 |                 "status": "partial_generation",
350 |                 "failed_agent": "Curriculum Planning Agent",
351 |                 "error_type": err_type,
352 |                 "successful_agents": ["Curriculum Analysis Agent", "Learning Outcome Agent"],
353 |                 "retry_recommended": True
354 |             }
355 |             state["logs"].append(f"Curriculum Planning Agent failed: {err_str}")
356 |
357 |     return state
358 |
359 |
360 | # Agent 4: Slide Generation Agent

```

```

361 | def slide_generation_agent(state: SharedState) -> SharedState:
362 |     state["logs"].append("Slide Generation Agent started.")
363 |     state["current_agent"] = "Slide Generation Agent"
364 |
365 |     if state.get("error_info"):
366 |         state["logs"].append("Slide Generation Agent skipped due to previous error.")
367 |         return state
368 |
369 |     course_id = state.get("course_id")
370 |     rag_context = ""
371 |     if course_id:
372 |         try:
373 |             db = SessionLocal()
374 |             chunks = retrieve_relevant_chunks(db, course_id=course_id, query="course content core
concepts theory details lecture explanations student examples", limit=5)
375 |             db.close()
376 |             if chunks:
377 |                 rag_context = "\n### RELEVANT CONCEPT DETAILS:\n" + "\n".join([f"- {c['chunk_text']}" for
c in chunks])
378 |         except Exception as e:
379 |             logger.warning(f"RAG context search bypassed: {e}")
380 |
381 |         try:
382 |             personalization = state.get("personalization_profile", {})
383 |             style_instruction = ""
384 |             if personalization:
385 |                 style_instruction = f" Apply style/theme: {personalization.get('style', 'Sleek Dark Mode')}"
and tone: {personalization.get('tone', 'Professional & Academic')}."
386 |                 if personalization.get("customInstructions"):
387 |                     style_instruction += f" Additional instructions:
{personalization.get('customInstructions')}"
388 |
389 |                 sys_prompt = (
390 |                     "You are an Elite Instructional Designer. "
391 |                     "Generate beautiful, logically organized, highly detailed slides based on the curriculum
sequence. "
392 |                     "For each slide, construct complete concepts, and extensive detailed explanations. "
393 |                     "Each item in the content list MUST be a long, highly informative paragraph of 3-4 sentences
containing deep academic/technical explanations, NOT short summary bullet points. Make the slide deck extremely
detailed. "
394 |                     f"Return JSON matching the schema.{style_instruction}"
395 |                 )
396 |
397 |                 lesson_sequence = []
398 |                 if isinstance(state.get("curriculum_plan"), dict):
399 |                     lesson_sequence = state["curriculum_plan"].get("lesson_sequence", [])
400 |                 elif isinstance(state.get("curriculum_plan"), list):
401 |                     lesson_sequence = state["curriculum_plan"]
402 |
403 |                 slides_list = []
404 |                 batch_size = 5
405 |                 for i in range(0, len(lesson_sequence), batch_size):
406 |                     batch_lessons = lesson_sequence[i:i+batch_size]
407 |                     user_prompt = f"Generate slide deck items for this batch of lessons/weeks:
{json.dumps(batch_lessons)}\n{rag_context}"
408 |
409 |                     res = call_llm(sys_prompt, user_prompt, response_schema=SlideDeck)
410 |                     data = json.loads(res)
411 |
412 |                     batch_slides = []
413 |                     if isinstance(data, list):
414 |                         batch_slides = data
415 |                     elif isinstance(data, dict):
416 |                         batch_slides = data.get("slides", [])
417 |
418 |                     slides_list.extend(batch_slides)

```

```

419 |
420 |         state["slide_deck"] = [{"slide_index": s.get("slide_index"), "title": s.get("title"), "content":
s.get("content", []), "suggested_visuals": s.get("suggested_visuals", "")} for s in slides_list if isinstance(s,
dict)]
421 |         state["logs"].append(f"Slide Generation Agent successfully processed {len(state['slide_deck'])}
slides via LLM.")
422 |     except Exception as e:
423 |         logger.error(f"Slide Generation Error: {e}")
424 |         err_str = str(e)
425 |         err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
426 |         state["error_info"] = {
427 |             "status": "partial_generation",
428 |             "failed_agent": "Slide Generation Agent",
429 |             "error_type": err_type,
430 |             "successful_agents": ["Curriculum Analysis Agent", "Learning Outcome Agent", "Curriculum
Planning Agent"],
431 |             "retry_recommended": True
432 |         }
433 |         state["logs"].append(f"Slide Generation Agent failed: {err_str}")
434 |
435 |     return state
436 |
437 |
438 | # Agent 5: Instructor Notes Agent
439 | def instructor_notes_agent(state: SharedState) -> SharedState:
440 |     state["logs"].append("Instructor Notes Agent started.")
441 |     state["current_agent"] = "Instructor Notes Agent"
442 |
443 |     if state.get("error_info"):
444 |         state["logs"].append("Instructor Notes Agent skipped due to previous error.")
445 |         return state
446 |
447 |     try:
448 |         personalization = state.get("personalization_profile", {})
449 |         style_instruction = ""
450 |         if personalization:
451 |             style_instruction = f"Apply tone: {personalization.get('tone', 'Professional & Academic')}."
452 |             if personalization.get("customInstructions"):
453 |                 style_instruction += f"Additional instructions:
{personalization.get('customInstructions')}"
454 |
455 |         sys_prompt = (
456 |             "You are an Expert Educator. "
457 |             "Generate instructor lecture scripts/notes for the generated slides. "
458 |             "For each slide note, you MUST generate at least 8-10 highly detailed, comprehensive lecture
talking points/explanations (minimum 8 talking points), detailed interactive pedagogy tips, whiteboard layout
guidelines, and at least 5-6 concrete real-world clarifying examples (minimum 5 examples). Make notes extremely
detailed and comprehensive. "
459 |             f"Return JSON matching the schema.{style_instruction}"
460 |         )
461 |
462 |         slide_deck = state.get("slide_deck", [])
463 |         notes_list = []
464 |         batch_size = 5
465 |
466 |         for i in range(0, len(slide_deck), batch_size):
467 |             batch_slides = slide_deck[i:i+batch_size]
468 |             user_prompt = f"Generate instructor notes for this batch of slides:
{json.dumps(batch_slides)}"
469 |
470 |             res = call_llm(sys_prompt, user_prompt, response_schema=InstructorNotesList)
471 |             data = json.loads(res)
472 |
473 |             batch_notes = []
474 |             if isinstance(data, list):

```

```

475 |         batch_notes = data
476 |         elif isinstance(data, dict):
477 |             batch_notes = data.get("notes", [])
478 |
479 |             notes_list.extend(batch_notes)
480 |
481 |             state["instructor_notes"] = [{"slide_index": n.get("slide_index"), "talking_points":
n.get("talking_points", []), "teaching_tips": n.get("teaching_tips", ""), "examples": n.get("examples", [])} for
n in notes_list if isinstance(n, dict)]
482 |             state["logs"].append(f"Instructor Notes Agent successfully processed
{len(state['instructor_notes'])} slide notes via LLM.")
483 |         except Exception as e:
484 |             logger.error(f"Instructor Notes Error: {e}")
485 |             err_str = str(e)
486 |             err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
487 |             state["error_info"] = {
488 |                 "status": "partial_generation",
489 |                 "failed_agent": "Instructor Notes Agent",
490 |                 "error_type": err_type,
491 |                 "successful_agents": ["Curriculum Analysis Agent", "Learning Outcome Agent", "Curriculum
Planning Agent", "Slide Generation Agent"],
492 |                 "retry_recommended": True
493 |             }
494 |             state["logs"].append(f"Instructor Notes Agent failed: {err_str}")
495 |
496 |             return state
497 |
498 |
499 | # Agent 6: Assessment Agent
500 | def assessment_agent(state: SharedState) -> SharedState:
501 |     state["logs"].append("Assessment Agent started.")
502 |     state["current_agent"] = "Assessment Agent"
503 |
504 |     if state.get("error_info"):
505 |         state["logs"].append("Assessment Agent skipped due to previous error.")
506 |         return state
507 |
508 |     try:
509 |         sys_prompt = (
510 |             "You are a Psychometric Evaluator and Examination Director. "
511 |             "Design highly rigorous multiple-choice assessment questions (MCQs with options and correct
answers). "
512 |             "You MUST generate distinct high-quality MCQs covering all aspects of the curriculum mapped
to learning outcomes. "
513 |             "Return JSON matching the schema."
514 |         )
515 |
516 |         learning_outcomes = state.get("learning_outcomes", [])
517 |         slide_deck = state.get("slide_deck", [])
518 |         # Extract only index and title to reduce prompt token footprint
519 |         slide_summary = [{"slide_index": s.get("slide_index"), "title": s.get("title")} for s in
slide_deck]
520 |
521 |         assessments_list = []
522 |         num_questions_per_batch = 6
523 |         num_batches = 5
524 |
525 |         for b in range(num_batches):
526 |             user_prompt = (
527 |                 f"Generate exactly {num_questions_per_batch} distinct high-quality MCQs (Batch {b+1}/5)
covering outcomes: {json.dumps(learning_outcomes)} "
528 |                 f"and slides: {json.dumps(slide_summary)}. Each question must be uniquely mapped to a
learning_outcome_id."
529 |             )
530 |

```



```

531 |         res = call_llm(sys_prompt, user_prompt, response_schema=AssessmentBank)
532 |         data = json.loads(res)
533 |
534 |         batch_assessments = []
535 |         if isinstance(data, list):
536 |             batch_assessments = data
537 |         elif isinstance(data, dict):
538 |             batch_assessments = data.get("assessments", [])
539 |
540 |         assessments_list.extend(batch_assessments)
541 |
542 |         state["assessment_bank"] = [
543 |             {
544 |                 "question_text": a.get("question_text"),
545 |                 "question_type": a.get("question_type"),
546 |                 "options": a.get("options", []),
547 |                 "correct_answer": a.get("correct_answer"),
548 |                 "bloom_level": a.get("bloom_level", "Remembering"),
549 |                 "learning_outcome_id": a.get("learning_outcome_id")
550 |             }
551 |             for a in assessments_list
552 |             if isinstance(a, dict) and a.get("question_text") and a.get("question_type")
553 |         ]
554 |         state["logs"].append(f"Assessment Agent successfully processed {len(state['assessment_bank'])}
MCQs via LLM.")
555 |     except Exception as e:
556 |         logger.error(f"Assessment Error: {e}")
557 |         err_str = str(e)
558 |         err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
559 |         state["error_info"] = {
560 |             "status": "partial_generation",
561 |             "failed_agent": "Assessment Agent",
562 |             "error_type": err_type,
563 |             "successful_agents": ["Curriculum Analysis Agent", "Learning Outcome Agent", "Curriculum
Planning Agent", "Slide Generation Agent", "Instructor Notes Agent"],
564 |             "retry_recommended": True
565 |         }
566 |         state["logs"].append(f"Assessment Agent failed: {err_str}")
567 |
568 |     return state
569 |
570 |
571 | # Agent 7: Bloom Audit Agent
572 | def bloom_audit_agent(state: SharedState) -> SharedState:
573 |     state["logs"].append("Bloom Audit Agent started.")
574 |     state["current_agent"] = "Bloom Audit Agent"
575 |
576 |     if state.get("error_info"):
577 |         state["logs"].append("Bloom Audit Agent skipped due to previous error.")
578 |         return state
579 |
580 |     try:
581 |         sys_prompt = (
582 |             "You are an Educational Quality Assurance Auditor. "
583 |             "Review the learning outcomes and assessments to calculate the exact cognitive balance across
the six Bloom dimensions, "
584 |             "and provide high-impact recommendations to improve cognitive depth. "
585 |             "You MUST calculate and return the average_coverage as a decimal float value between 0.0 and
100.0 (e.g., 75.5 or 63.3). "
586 |             "Do NOT output mathematical expressions, divisions, slashes, or formulas (like '38 / 60') for
average_coverage. It must be a raw float value. "
587 |             "Return JSON matching the schema."
588 |         )
589 |         user_prompt = (
590 |             f"Perform a Bloom audit for outcomes: {json.dumps(state['learning_outcomes'])} "

```

```

591 |         f"and assessments: {json.dumps(state['assessment_bank'])}"
592 |     )
593 |
594 |     res = call_llm(sys_prompt, user_prompt, response_schema=BloomReport)
595 |     data = json.loads(res)
596 |
597 |     state["bloom_report"] = data
598 |     state["logs"].append("Bloom Audit Agent successfully processed via LLM.")
599 | except Exception as e:
600 |     logger.error(f"Bloom Audit Error: {e}")
601 |     err_str = str(e)
602 |     err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
603 |     state["error_info"] = {
604 |         "status": "partial_generation",
605 |         "failed_agent": "Bloom Audit Agent",
606 |         "error_type": err_type,
607 |         "successful_agents": ["Curriculum Analysis Agent", "Learning Outcome Agent", "Curriculum
Planning Agent", "Slide Generation Agent", "Instructor Notes Agent", "Assessment Agent"],
608 |         "retry_recommended": True
609 |     }
610 |     state["logs"].append(f"Bloom Audit Agent failed: {err_str}")
611 |
612 |     return state
613 |
614 |
615 | # Agent 8: Industry Gap Agent
616 | def industry_gap_agent(state: SharedState) -> SharedState:
617 |     state["logs"].append("Industry Gap Agent started.")
618 |     state["current_agent"] = "Industry Gap Agent"
619 |
620 |     if state.get("error_info"):
621 |         state["logs"].append("Industry Gap Agent skipped due to previous error.")
622 |         return state
623 |
624 |     course_id = state.get("course_id")
625 |     rag_context = ""
626 |     if course_id:
627 |         try:
628 |             db = SessionLocal()
629 |             chunks = retrieve_relevant_chunks(db, course_id=course_id, query="modern industrial
requirements skills tools technology", limit=5)
630 |             db.close()
631 |             if chunks:
632 |                 rag_context = "\n### RELEVANT INDUSTRIAL STANDARD CHUNKS:\n" + "\n".join([f"-
{c['chunk_text']}" for c in chunks])
633 |             except Exception as e:
634 |                 logger.warning(f"RAG context search bypassed: {e}")
635 |
636 |         try:
637 |             sys_prompt = (
638 |                 "You are a Silicon Valley Tech Lead and Industry Readiness Auditor. "
639 |                 "Compare the course modules and topics against modern technology and industry requirements to
identify missing critical topics and recommend updates. "
640 |                 "Return JSON matching the schema."
641 |             )
642 |             user_prompt = (
643 |                 f"Analyze industry gaps for curriculum map: {json.dumps(state['curriculum_map'])} "
644 |                 f"and learning outcomes: {json.dumps(state['learning_outcomes'])}.\n"
645 |                 f"Industrial context: {rag_context}"
646 |             )
647 |
648 |             res = call_llm(sys_prompt, user_prompt, response_schema=IndustryGapReport)
649 |             data = json.loads(res)
650 |
651 |             state["industry_gap_report"] = data

```

```

652 |         state["logs"].append("Industry Gap Agent successfully processed via LLM.")
653 |     except Exception as e:
654 |         logger.error(f"Industry Gap Error: {e}")
655 |         err_str = str(e)
656 |         err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
657 |         state["error_info"] = {
658 |             "status": "partial_generation",
659 |             "failed_agent": "Industry Gap Agent",
660 |             "error_type": err_type,
661 |             "successful_agents": ["Curriculum Analysis Agent", "Learning Outcome Agent", "Curriculum
Planning Agent", "Slide Generation Agent", "Instructor Notes Agent", "Assessment Agent", "Bloom Audit Agent"],
662 |             "retry_recommended": True
663 |         }
664 |         state["logs"].append(f"Industry Gap Agent failed: {err_str}")
665 |
666 |     return state
667 |
668 |
669 | # Agent 9: Readiness Score Agent
670 | def readiness_score_agent(state: SharedState) -> SharedState:
671 |     state["logs"].append("Readiness Score Agent started.")
672 |     state["current_agent"] = "Readiness Score Agent"
673 |
674 |     if state.get("error_info"):
675 |         state["logs"].append("Readiness Score Agent skipped due to previous error.")
676 |         return state
677 |
678 |     try:
679 |         sys_prompt = (
680 |             "You are a Curriculum Quality Director. "
681 |             "Evaluate the overall quality, completeness, and readiness of the generated course deck on a
scale of 0 to 100. "
682 |             "Each score or coverage attribute must be a raw decimal float value between 0.0 and 100.0
(e.g., 85.0 or 92.5). "
683 |             "Do NOT output mathematical expressions, divisions, slashes, or formulas for any float
fields. It must be a raw float value. "
684 |             "Return JSON matching the schema."
685 |         )
686 |         slide_summary = [{"slide_index": s.get("slide_index"), "title": s.get("title")} for s in
state.get("slide_deck", [])]
687 |         user_prompt = (
688 |             f"Evaluate course readiness for curriculum: {json.dumps(state['curriculum_map'])}, "
689 |             f"outcomes: {json.dumps(state['learning_outcomes'])}, slides: {json.dumps(slide_summary)}, "
690 |             f"assessments: {json.dumps(state['assessment_bank'])}, and Bloom audit:
{json.dumps(state['bloom_report'])}."
691 |         )
692 |
693 |         res = call_llm(sys_prompt, user_prompt, response_schema=ReadinessScore)
694 |         data = json.loads(res)
695 |
696 |         state["readiness_score"] = data
697 |         state["logs"].append("Readiness Score Agent successfully processed via LLM.")
698 |     except Exception as e:
699 |         logger.error(f"Readiness Score Error: {e}")
700 |         err_str = str(e)
701 |         err_type = "429" if "429" in err_str or "RESOURCE_EXHAUSTED" in err_str else "503" if "503" in
err_str or "UNAVAILABLE" in err_str else "500"
702 |         state["error_info"] = {
703 |             "status": "partial_generation",
704 |             "failed_agent": "Readiness Score Agent",
705 |             "error_type": err_type,
706 |             "successful_agents": ["Curriculum Analysis Agent", "Learning Outcome Agent", "Curriculum
Planning Agent", "Slide Generation Agent", "Instructor Notes Agent", "Assessment Agent", "Bloom Audit Agent",
"Industry Gap Agent"],
707 |             "retry_recommended": True

```

```

708 |     }
709 |     state["logs"].append(f"Readiness Score Agent failed: {err_str}")
710 |
711 |     state["logs"].append("LangGraph workflow execution completed successfully.")
712 |     state["current_agent"] = "Done"
713 |
714 |     return state
715 |
716 |
717 | # Helper conditional function for self-healing Bloom taxonomy loops
718 | def route_bloom_coverage(state: SharedState) -> str:
719 |     if state.get("error_info"):
720 |         return "industry_gap"
721 |
722 |     report = state.get("bloom_report", {})
723 |     if not report:
724 |         return "industry_gap"
725 |     avg = report.get("average_coverage", 100)
726 |
727 |     # Count previous loop iterations in the logs
728 |     loops = sum(1 for log in state.get("logs", []) if "Triggering self-healing feedback loop" in log)
729 |
730 |     if avg < 75 and loops < 1:
731 |         state["logs"].append("Audit Alert: Higher-order Bloom cognitive coverage is below 75%. Triggering
self-healing feedback loop back to Curriculum Analysis Agent to enrich outcome balance.")
732 |         return "curriculum_analysis"
733 |     else:
734 |         return "industry_gap"
735 |
736 |
737 | def make_telemetry_agent(agent_fn, agent_name):
738 |     def wrapped_agent(state: SharedState) -> SharedState:
739 |         import time
740 |         from datetime import datetime
741 |
742 |         start_time = time.time()
743 |         started_at = datetime.utcnow().isoformat() + "Z"
744 |
745 |         has_error_before = bool(state.get("error_info"))
746 |
747 |         new_state = agent_fn(state)
748 |
749 |         end_time = time.time()
750 |         completed_at = datetime.utcnow().isoformat() + "Z"
751 |         duration_ms = int(round((end_time - start_time) * 1000))
752 |
753 |         has_error_after = bool(new_state.get("error_info"))
754 |
755 |         if has_error_after and not has_error_before:
756 |             status = "failure"
757 |         else:
758 |             status = "success"
759 |
760 |         telemetry_item = {
761 |             "agent": agent_name,
762 |             "status": status,
763 |             "start_time": started_at,
764 |             "end_time": completed_at,
765 |             "duration_ms": duration_ms
766 |         }
767 |
768 |         if "pipeline_telemetry" not in new_state or not isinstance(new_state["pipeline_telemetry"],
list):
769 |             new_state["pipeline_telemetry"] = []
770 |
771 |             existing_idx = -1

```

```

772 |         for idx, item in enumerate(new_state["pipeline_telemetry"]):
773 |             if item["agent"] == agent_name:
774 |                 existing_idx = idx
775 |                 break
776 |
777 |         if existing_idx != -1:
778 |             new_state["pipeline_telemetry"][existing_idx] = telemetry_item
779 |         else:
780 |             new_state["pipeline_telemetry"].append(telemetry_item)
781 |
782 |         return new_state
783 |     return wrapped_agent
784 |
785 |
786 | # Compile the Workflow Graph
787 | def build_workflow() -> StateGraph:
788 |     workflow = StateGraph(SharedState)
789 |
790 |     # Register the 9 nodes wrapped in telemetry recorder
791 |     workflow.add_node("curriculum_analysis", make_telemetry_agent(curriculum_analysis_agent, "Curriculum
Analysis Agent"))
792 |     workflow.add_node("learning_outcome", make_telemetry_agent(learning_outcome_agent, "Learning Outcome
Agent"))
793 |     workflow.add_node("curriculum_planning", make_telemetry_agent(curriculum_planning_agent, "Curriculum
Planning Agent"))
794 |     workflow.add_node("slide_generation", make_telemetry_agent(slide_generation_agent, "Slide Generation
Agent"))
795 |     workflow.add_node("instructor_notes", make_telemetry_agent(instructor_notes_agent, "Instructor Notes
Agent"))
796 |     workflow.add_node("assessment", make_telemetry_agent(assessment_agent, "Assessment Agent"))
797 |     workflow.add_node("bloom_audit", make_telemetry_agent(bloom_audit_agent, "Bloom Audit Agent"))
798 |     workflow.add_node("industry_gap", make_telemetry_agent(industry_gap_agent, "Industry Gap Agent"))
799 |     workflow.add_node("readiness_score", make_telemetry_agent(readiness_score_agent, "Readiness Score
Agent"))
800 |
801 |     # Establish entry point
802 |     workflow.set_entry_point("curriculum_analysis")
803 |
804 |     # Sequential flow edges
805 |     workflow.add_edge("curriculum_analysis", "learning_outcome")
806 |     workflow.add_edge("learning_outcome", "curriculum_planning")
807 |     workflow.add_edge("curriculum_planning", "slide_generation")
808 |     workflow.add_edge("slide_generation", "instructor_notes")
809 |     workflow.add_edge("instructor_notes", "assessment")
810 |     workflow.add_edge("assessment", "bloom_audit")
811 |
812 |     # Conditional routing edge for self-healing taxonomy auditing
813 |     workflow.add_conditional_edges(
814 |         "bloom_audit",
815 |         route_bloom_coverage,
816 |         {
817 |             "curriculum_analysis": "curriculum_analysis",
818 |             "industry_gap": "industry_gap"
819 |         }
820 |     )
821 |
822 |     workflow.add_edge("industry_gap", "readiness_score")
823 |     workflow.add_edge("readiness_score", END)
824 |
825 |     return workflow.compile()
826 |

```

File: backend/app/services/rag_service.py

```
1 | import os
2 | import json
3 | import math
4 | from typing import List, Dict, Any
5 | from pypdf import PdfReader
6 | from sqlalchemy.orm import Session
7 | from app.models import UploadedDocument, Embedding
8 |
9 | # Helper to get embeddings from Google Gemini
10 | def get_embedding(text: str) -> List[float]:
11 |     api_key = os.getenv("GEMINI_API_KEY")
12 |     if not api_key or api_key.strip() == "" or api_key.startswith("your_"):
13 |         # Local mock embedding generator (reproducible based on text hashes for demo stability)
14 |         import hashlib
15 |         h = hashlib.md5(text.encode("utf-8")).hexdigest()
16 |         # Create a deterministic mock vector of 768 dimensions
17 |         mock_vector = []
18 |         for i in range(768):
19 |             val = (int(h[i % 32], 16) / 15.0) * math.sin(i + 1)
20 |             mock_vector.append(val)
21 |         # Normalize vector
22 |         norm = math.sqrt(sum(x*x for x in mock_vector))
23 |         return [x/norm for x in mock_vector] if norm > 0 else [0.0]*768
24 |
25 |     try:
26 |         from google import genai
27 |         client = genai.Client(api_key=api_key)
28 |         response = client.models.embed_content(
29 |             model="models/gemini-embedding-2",
30 |             contents=text
31 |         )
32 |         return response.embeddings[0].values
33 |     except Exception as e:
34 |         print(f"[RAG] Error generating Gemini embedding: {e}")
35 |         # Dynamic fallback
36 |         return [0.0] * 3072
37 |
38 | # Parse PDF text
39 | def extract_text_from_pdf(file_path: str) -> str:
40 |     if not os.path.exists(file_path):
41 |         return ""
42 |     try:
43 |         reader = PdfReader(file_path)
44 |         text = ""
45 |         for page in reader.pages:
46 |             t = page.extract_text()
47 |             if t:
48 |                 text += t + "\n"
49 |         return text
50 |     except Exception as e:
51 |         print(f"[RAG] PDF Extraction failed: {e}")
52 |         return ""
53 |
54 | # Split text into overlapping chunks
55 | def chunk_text(text: str, chunk_size: int = 500, overlap: int = 100) -> List[str]:
56 |     chunks = []
57 |     words = text.split()
58 |     # Estimate characters using words (approx 5 chars per word)
59 |     text_len = len(text)
60 |     if text_len <= chunk_size:
61 |         return [text] if text.strip() else []
62 |
```

```

63 |     start = 0
64 |     while start < text_len:
65 |         end = start + chunk_size
66 |         chunk = text[start:end]
67 |         if chunk.strip():
68 |             chunks.append(chunk.strip())
69 |         start += chunk_size - overlap
70 |
71 |     return chunks
72 |
73 | # Store document and generate RAG index
74 | def process_and_index_document(db: Session, course_id: int, file_name: str, file_path: str,
document_type: str = "RAG_SOURCE") -> UploadedDocument:
75 |     # 1. Create uploaded document record
76 |     doc = UploadedDocument(
77 |         course_id=course_id,
78 |         file_name=file_name,
79 |         file_path=file_path,
80 |         document_type=document_type
81 |     )
82 |     db.add(doc)
83 |     db.commit()
84 |     db.refresh(doc)
85 |
86 |     # 2. Extract text
87 |     text = ""
88 |     if file_name.lower().endswith(".pdf"):
89 |         text = extract_text_from_pdf(file_path)
90 |     elif file_name.lower().endswith((".txt", ".md")):
91 |         try:
92 |             with open(file_path, "r", encoding="utf-8") as f:
93 |                 text = f.read()
94 |         except Exception:
95 |             text = ""
96 |     else:
97 |         text = f"Simulated content for document: {file_name}"
98 |
99 |     # 3. Chunk text
100 |    chunks = chunk_text(text)
101 |
102 |    # 4. Generate embeddings and save
103 |    for idx, chunk in enumerate(chunks):
104 |        vector = get_embedding(chunk)
105 |        embedding_record = Embedding(
106 |            document_id=doc.id,
107 |            chunk_text=chunk,
108 |            embedding=vector # Stored as JSON list of floats
109 |        )
110 |        db.add(embedding_record)
111 |
112 |    db.commit()
113 |    return doc
114 |
115 | # Calculate cosine similarity
116 | def cosine_similarity(v1: List[float], v2: List[float]) -> float:
117 |     if len(v1) != len(v2) or not v1 or not v2:
118 |         return 0.0
119 |     dot = sum(a*b for a, b in zip(v1, v2))
120 |     sum1 = sum(a*a for a in v1)
121 |     sum2 = sum(b*b for b in v2)
122 |     denom = math.sqrt(sum1) * math.sqrt(sum2)
123 |     return dot / denom if denom > 0 else 0.0
124 |
125 | # Retrieve relevant chunks for course
126 | def retrieve_relevant_chunks(db: Session, course_id: int, query: str, limit: int = 4) -> List[Dict[str,
Any]]:

```

```
127 |     # Generate query embedding
128 |     query_vector = get_embedding(query)
129 |
130 |     # Get all documents belonging to this course
131 |     docs = db.query(UploadedDocument).filter(UploadedDocument.course_id == course_id).all()
132 |     if not docs:
133 |         return []
134 |
135 |     doc_ids = [d.id for d in docs]
136 |
137 |     # Fetch all chunks
138 |     all_embeddings = db.query(Embedding).filter(Embedding.document_id.in_(doc_ids)).all()
139 |
140 |     # Run in-memory cosine similarity (dual fallback Postgres mapping & SQLite matching)
141 |     results = []
142 |     for item in all_embeddings:
143 |         # DB embeddings might be loaded as JSON-deserialized list or JSON string depending on sqlite
driver
144 |         vector = item.embedding
145 |         if isinstance(vector, str):
146 |             try:
147 |                 vector = json.loads(vector)
148 |             except Exception:
149 |                 continue
150 |
151 |         similarity = cosine_similarity(query_vector, vector)
152 |         results.append({
153 |             "chunk_text": item.chunk_text,
154 |             "similarity": similarity,
155 |             "document_id": item.document_id
156 |         })
157 |
158 |     # Sort by similarity descending
159 |     results.sort(key=lambda x: x["similarity"], reverse=True)
160 |     return results[:limit]
161 |
```


File: backend/app/routers/auth.py

```
1 | import os
2 | import datetime
3 | import logging
4 | from typing import Optional
5 | from fastapi import APIRouter, Depends, HTTPException, status, Request
6 | from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
7 | from sqlalchemy import func
8 | from sqlalchemy.orm import Session
9 | import jwt
10 | import bcrypt
11 | from pydantic import BaseModel, EmailStr
12 | from app.database import get_db
13 | from app.models import User
14 |
15 | # Configure Logger
16 | logger = logging.getLogger("kalyx.auth")
17 |
18 | # Load config
19 | JWT_SECRET = os.getenv("JWT_SECRET", "hackathon_super_secret_key_kalyx_2026")
20 | JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")
21 | ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", "1440"))
22 |
23 | router = APIRouter(prefix="/auth", tags=["Authentication"])
24 | oauth2_scheme = OAuth2PasswordBearer(tokenUrl="api/auth/login", auto_error=False)
25 | from app.security import limit_signup, limit_login
26 |
27 | # Pydantic schemas
28 | class UserSignup(BaseModel):
29 |     email: EmailStr
30 |     username: str
31 |     password: str
32 |     full_name: Optional[str] = None
33 |
34 | class UserLoginSchema(BaseModel):
35 |     username_or_email: str
36 |     password: str
37 |
38 | class TokenResponse(BaseModel):
39 |     access_token: str
40 |     token_type: str
41 |     user: dict
42 |
43 | # Password hashing utilities
44 | def hash_password(password: str) -> str:
45 |     if isinstance(password, str):
46 |         pwd_bytes = password.encode('utf-8')
47 |     elif isinstance(password, bytes):
48 |         pwd_bytes = password
49 |     else:
50 |         pwd_bytes = str(password).encode('utf-8')
51 |     salt = bcrypt.gensalt()
52 |     hashed = bcrypt.hashpw(pwd_bytes, salt)
53 |     if isinstance(hashed, bytes):
54 |         return hashed.decode('utf-8')
55 |     return hashed
56 |
57 | def verify_password(plain_password: str, hashed_password: str) -> bool:
58 |     try:
59 |         # Convert plain password to bytes safely
60 |         if isinstance(plain_password, str):
61 |             plain_bytes = plain_password.encode('utf-8')
62 |         elif isinstance(plain_password, bytes):
```

```

63 |         plain_bytes = plain_password
64 |     else:
65 |         plain_bytes = str(plain_password).encode('utf-8')
66 |
67 |     # Convert hashed password to bytes safely
68 |     if isinstance(hashed_password, str):
69 |         hashed_bytes = hashed_password.encode('utf-8')
70 |     elif isinstance(hashed_password, bytes):
71 |         hashed_bytes = hashed_password
72 |     else:
73 |         hashed_bytes = str(hashed_password).encode('utf-8')
74 |
75 |     return bcrypt.checkpw(plain_bytes, hashed_bytes)
76 | except Exception as e:
77 |     logger.error(f"Error in verify_password: {e}", exc_info=True)
78 |     return False
79 |
80 | # Token utility
81 | def create_access_token(data: dict) -> str:
82 |     to_encode = data.copy()
83 |     expire = datetime.datetime.utcnow() + datetime.timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
84 |     to_encode.update({"exp": expire})
85 |     return jwt.encode(to_encode, JWT_SECRET, algorithm=JWT_ALGORITHM)
86 |
87 | def get_current_user(
88 |     request: Request,
89 |     token: Optional[str] = Depends(oauth2_scheme),
90 |     db: Session = Depends(get_db)
91 | ) -> User:
92 |     if not token:
93 |         token = request.query_params.get("token")
94 |
95 |     credentials_exception = HTTPException(
96 |         status_code=status.HTTP_401_UNAUTHORIZED,
97 |         detail="Could not validate credentials",
98 |         headers={"WWW-Authenticate": "Bearer"},
99 |     )
100 |     if not token:
101 |         raise credentials_exception
102 |     try:
103 |         payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
104 |         email: str = payload.get("sub")
105 |         if email is None:
106 |             raise credentials_exception
107 |     except jwt.PyJWTError:
108 |         raise credentials_exception
109 |
110 |     user = db.query(User).filter(func.lower(User.email) == func.lower(email.strip())).first()
111 |     if user is None:
112 |         raise credentials_exception
113 |     return user
114 |
115 | @router.post("/signup", response_model=TokenResponse, dependencies=[Depends(limit_signup)])
116 | def signup(payload: UserSignup, db: Session = Depends(get_db)):
117 |     email_clean = payload.email.strip().lower()
118 |     username_clean = payload.username.strip().lower()
119 |
120 |     # Check if user already exists (by email)
121 |     existing_email = db.query(User).filter(func.lower(User.email) == email_clean).first()
122 |     if existing_email:
123 |         raise HTTPException(status_code=400, detail="Email is already registered")
124 |
125 |     # Check if username already taken
126 |     existing_username = db.query(User).filter(func.lower(User.username) == username_clean).first()
127 |     if existing_username:
128 |         raise HTTPException(status_code=400, detail="Username is already taken")

```

```
129 |
130 |     hashed = hash_password(payload.password)
131 |     user = User(
132 |         email=email_clean,
133 |         username=username_clean,
134 |         hashed_password=hashed,
135 |         full_name=payload.full_name.strip() if payload.full_name else None
136 |     )
137 |     db.add(user)
138 |     db.commit()
139 |     db.refresh(user)
140 |
141 |     token = create_access_token({"sub": user.email})
142 |     return {
143 |         "access_token": token,
144 |         "token_type": "bearer",
145 |         "user": {
146 |             "id": user.id,
147 |             "email": user.email,
148 |             "username": user.username,
149 |             "full_name": user.full_name
150 |         }
151 |     }
152 |
153 | @router.post("/login", response_model=TokenResponse, dependencies=[Depends(limit_login)])
154 | def login(payload: UserLoginSchema, db: Session = Depends(get_db)):
155 |     login_str = payload.username_or_email.strip()
156 |     user = db.query(User).filter(
157 |         (func.lower(User.email) == func.lower(login_str)) |
158 |         (func.lower(User.username) == func.lower(login_str))
159 |     ).first()
160 |
161 |     if not user or not verify_password(payload.password, user.hashed_password):
162 |         raise HTTPException(status_code=401, detail="Invalid username/email or password")
163 |
164 |     token = create_access_token({"sub": user.email})
165 |     return {
166 |         "access_token": token,
167 |         "token_type": "bearer",
168 |         "user": {
169 |             "id": user.id,
170 |             "email": user.email,
171 |             "username": user.username,
172 |             "full_name": user.full_name
173 |         }
174 |     }
175 |
```

File: backend/app/routers/courses.py

```

1 | import os
2 | import shutil
3 | from typing import List, Optional, Any, Dict
4 | from fastapi import APIRouter, Depends, HTTPException, UploadFile, File, BackgroundTasks
5 | from sqlalchemy.orm import Session
6 | from pydantic import BaseModel
7 | from app.database import get_db
8 | from app.routers.auth import get_current_user
9 | from app.models import (
10 |     User, Course, Syllabus, CurriculumAnalysis, LearningOutcome,
11 |     GeneratedSlide, InstructorNote, Assessment, ReadinessScore, UploadedDocument
12 | )
13 | from app.agents.workflow import build_workflow
14 | from app.services.rag_service import process_and_index_document
15 | from app.security import limit_course_creation, limit_syllabus_upload, limit_analysis_generation
16 |
17 | router = APIRouter(prefix="/courses", tags=["Courses"])
18 |
19 | class CourseCreate(BaseModel):
20 |     title: str
21 |     description: Optional[str] = None
22 |
23 | class CourseResponse(BaseModel):
24 |     id: int
25 |     title: str
26 |     description: Optional[str] = None
27 |     created_at: Any
28 |
29 |     class Config:
30 |         orm_mode = True
31 |
32 | @router.get("/")
33 | def list_courses(db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
34 |     return db.query(Course).filter(Course.user_id == current_user.id).all()
35 |
36 | @router.post("/", dependencies=[Depends(limit_course_creation)])
37 | def create_course(payload: CourseCreate, db: Session = Depends(get_db), current_user: User =
Depends(get_current_user)):
38 |     if not payload.title or not payload.title.strip():
39 |         raise HTTPException(
40 |             status_code=422,
41 |             detail=[{"loc": ["body", "title"], "msg": "Course title cannot be empty", "type":
"value_error"}]
42 |         )
43 |     if not payload.description or not payload.description.strip():
44 |         raise HTTPException(
45 |             status_code=422,
46 |             detail=[{"loc": ["body", "description"], "msg": "Course description cannot be empty", "type":
"value_error"}]
47 |         )
48 |     course = Course(
49 |         user_id=current_user.id,
50 |         title=payload.title,
51 |         description=payload.description
52 |     )
53 |     db.add(course)
54 |     db.commit()
55 |     db.refresh(course)
56 |     return course
57 |
58 | @router.get("/{course_id}")
59 | def get_course_details(course_id: int, db: Session = Depends(get_db), current_user: User =

```

```

Depends(get_current_user)):
60 |     # Verify ownership
61 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
62 |     if not course:
63 |         raise HTTPException(status_code=404, detail="Course not found")
64 |
65 |     analysis = db.query(CurriculumAnalysis).filter(CurriculumAnalysis.course_id == course_id).first()
66 |     outcomes = db.query(LearningOutcome).filter(LearningOutcome.course_id == course_id).all()
67 |     slides = db.query(GeneratedSlide).filter(GeneratedSlide.course_id ==
course_id).order_by(GeneratedSlide.slide_index).all()
68 |     notes = db.query(InstructorNote).filter(InstructorNote.course_id ==
course_id).order_by(InstructorNote.slide_index).all()
69 |     assessments = db.query(Assessment).filter(Assessment.course_id == course_id).all()
70 |     score = db.query(ReadinessScore).filter(ReadinessScore.course_id == course_id).first()
71 |
72 |     return {
73 |         "course": {
74 |             "id": course.id,
75 |             "title": course.title,
76 |             "description": course.description,
77 |             "created_at": course.created_at
78 |         },
79 |         "curriculum_analysis": {
80 |             "curriculum_map": analysis.curriculum_map if analysis else None,
81 |             "gap_analysis": analysis.gap_analysis if analysis else None,
82 |             "industry_gap_report": analysis.industry_gap_report if analysis else None,
83 |             "pipeline_telemetry": getattr(analysis, "pipeline_telemetry", []) if analysis else []
84 |         } if analysis else None,
85 |         "learning_outcomes": [
86 |             {"id": o.id, "text": o.outcome_text, "bloom_level": o.bloom_level} for o in outcomes
87 |         ],
88 |         "slides": [
89 |             {"id": s.id, "slide_index": s.slide_index, "title": s.title, "content": s.content,
"suggested_visuals": s.suggested_visuals}
90 |             for s in slides
91 |         ],
92 |         "notes": [
93 |             {"id": n.id, "slide_index": n.slide_index, "talking_points": n.talking_points,
"teaching_tips": n.teaching_tips, "examples": n.examples}
94 |             for n in notes
95 |         ],
96 |         "assessments": [
97 |             {
98 |                 "id": a.id,
99 |                 "question_text": a.question_text,
100 |                 "question_type": a.question_type,
101 |                 "options": a.options,
102 |                 "correct_answer": a.correct_answer,
103 |                 "bloom_level": a.bloom_level,
104 |                 "learning_outcome_id": a.learning_outcome_id
105 |             } for a in assessments
106 |         ],
107 |         "readiness_score": {
108 |             "score": score.score,
109 |             "completeness": score.completeness,
110 |             "outcome_coverage": score.outcome_coverage,
111 |             "assessment_quality": score.assessment_quality,
112 |             "bloom_coverage": score.bloom_coverage,
113 |             "industry_relevance": score.industry_relevance,
114 |             "breakdown": score.breakdown
115 |         } if score else None
116 |     }
117 |
118 | @router.delete("/{course_id}")
119 | def delete_course(course_id: int, db: Session = Depends(get_db), current_user: User =
Depends(get_current_user)):

```

```

120 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
121 |     if not course:
122 |         raise HTTPException(status_code=404, detail="Course not found")
123 |     db.delete(course)
124 |     db.commit()
125 |     return {"message": f"Course {course_id} deleted successfully"}
126 |
127 | def save_workflow_outputs(db: Session, course_id: int, final_state: Dict[str, Any]):
128 |     # Clean previous generated course details to ensure clean overwrite
129 |     db.query(CurriculumAnalysis).filter(CurriculumAnalysis.course_id == course_id).delete()
130 |     db.query(LearningOutcome).filter(LearningOutcome.course_id == course_id).delete()
131 |     db.query(GeneratedSlide).filter(GeneratedSlide.course_id == course_id).delete()
132 |     db.query(InstructorNote).filter(InstructorNote.course_id == course_id).delete()
133 |     db.query(Assessment).filter(Assessment.course_id == course_id).delete()
134 |     db.query(ReadinessScore).filter(ReadinessScore.course_id == course_id).delete()
135 |     db.commit()
136 |
137 |     # Conditionally save Curriculum Analysis
138 |     if final_state.get("curriculum_map") and isinstance(final_state["curriculum_map"], dict) and
len(final_state["curriculum_map"]) > 0:
139 |         bloom_rec = None
140 |         if final_state.get("bloom_report") and isinstance(final_state["bloom_report"], dict) and
len(final_state["bloom_report"]) > 0:
141 |             bloom_rec = final_state["bloom_report"].get("recommendation")
142 |
143 |             ind_gap = None
144 |             if final_state.get("industry_gap_report") and isinstance(final_state["industry_gap_report"],
dict) and len(final_state["industry_gap_report"]) > 0:
145 |                 ind_gap = final_state["industry_gap_report"]
146 |
147 |             analysis = CurriculumAnalysis(
148 |                 course_id=course_id,
149 |                 curriculum_map=final_state["curriculum_map"],
150 |                 gap_analysis=bloom_rec if bloom_rec else "Review learning modules completeness.",
151 |                 industry_gap_report=ind_gap,
152 |                 pipeline_telemetry=final_state.get("pipeline_telemetry", [])
153 |             )
154 |             db.add(analysis)
155 |             db.commit()
156 |
157 |     # Conditionally save Learning Outcomes
158 |     outcome_map = {}
159 |     if final_state.get("learning_outcomes") and isinstance(final_state["learning_outcomes"], list) and
len(final_state["learning_outcomes"]) > 0:
160 |         for item in final_state["learning_outcomes"]:
161 |             if isinstance(item, dict) and item.get("text") and item.get("bloom_level"):
162 |                 outcome = LearningOutcome(
163 |                     course_id=course_id,
164 |                     outcome_text=item["text"],
165 |                     bloom_level=item["bloom_level"]
166 |                 )
167 |                 db.add(outcome)
168 |                 db.commit()
169 |                 outcome_map[item.get("id")] = outcome.id
170 |
171 |     # Conditionally save Slides
172 |     if final_state.get("slide_deck") and isinstance(final_state["slide_deck"], list) and
len(final_state["slide_deck"]) > 0:
173 |         for slide_item in final_state["slide_deck"]:
174 |             if isinstance(slide_item, dict) and slide_item.get("slide_index") is not None and
slide_item.get("title") and slide_item.get("content"):
175 |                 slide = GeneratedSlide(
176 |                     course_id=course_id,
177 |                     slide_index=slide_item["slide_index"],
178 |                     title=slide_item["title"],
179 |                     content=slide_item["content"],

```

```

180 |         suggested_visuals=slide_item.get("suggested_visuals", "")
181 |     )
182 |     db.add(slide)
183 |
184 |     # Conditionally save Speaker Notes
185 |     if final_state.get("instructor_notes") and isinstance(final_state["instructor_notes"], list) and
len(final_state["instructor_notes"]) > 0:
186 |         for note_item in final_state["instructor_notes"]:
187 |             if isinstance(note_item, dict) and note_item.get("slide_index") is not None and
note_item.get("talking_points"):
188 |                 note = InstructorNote(
189 |                     course_id=course_id,
190 |                     slide_index=note_item["slide_index"],
191 |                     talking_points=note_item["talking_points"],
192 |                     teaching_tips=note_item.get("teaching_tips", ""),
193 |                     examples=note_item.get("examples", [])
194 |                 )
195 |                 db.add(note)
196 |
197 |     # Conditionally save Assessments
198 |     if final_state.get("assessment_bank") and isinstance(final_state["assessment_bank"], list) and
len(final_state["assessment_bank"]) > 0:
199 |         for q_item in final_state["assessment_bank"]:
200 |             if isinstance(q_item, dict) and q_item.get("question_text") and q_item.get("question_type"):
201 |                 mapped_outcome_id = outcome_map.get(q_item.get("learning_outcome_id")) if outcome_map
else None
202 |                 assessment = Assessment(
203 |                     course_id=course_id,
204 |                     learning_outcome_id=mapped_outcome_id,
205 |                     question_text=q_item["question_text"],
206 |                     question_type=q_item["question_type"],
207 |                     options=q_item.get("options"),
208 |                     correct_answer=q_item.get("correct_answer"),
209 |                     bloom_level=q_item.get("bloom_level", "Remembering")
210 |                 )
211 |                 db.add(assessment)
212 |
213 |     # Conditionally save Readiness Score
214 |     if final_state.get("readiness_score") and isinstance(final_state["readiness_score"], dict) and
len(final_state["readiness_score"]) > 0:
215 |         scr = final_state["readiness_score"]
216 |         score = ReadinessScore(
217 |             course_id=course_id,
218 |             score=scr.get("score", 70.0),
219 |             completeness=scr.get("completeness", 70.0),
220 |             outcome_coverage=scr.get("outcome_coverage", 70.0),
221 |             assessment_quality=scr.get("assessment_quality", 70.0),
222 |             bloom_coverage=scr.get("bloom_coverage", 70.0),
223 |             industry_relevance=scr.get("industry_relevance", 70.0),
224 |             breakdown=scr.get("breakdown", {})
225 |         )
226 |         db.add(score)
227 |
228 |     db.commit()
229 |
230 |
231 | @router.post("/{course_id}/analyze", dependencies=[Depends(limit_syllabus_upload)])
232 | def analyze_syllabus(
233 |     course_id: int,
234 |     file: UploadFile = File(...),
235 |     db: Session = Depends(get_db),
236 |     current_user: User = Depends(get_current_user)
237 | ):
238 |     # Validation checks
239 |     if not file.filename or not file.filename.strip():
240 |         raise HTTPException(

```

```

241 |         status_code=422,
242 |         detail=[{"loc": ["body", "file"], "msg": "No file uploaded", "type": "value_error"}]
243 |     )
244 |
245 |     ext = os.path.splitext(file.filename)[1].lower()
246 |     if ext not in [".pdf", ".txt"]:
247 |         raise HTTPException(
248 |             status_code=422,
249 |             detail=[{"loc": ["body", "file"], "msg": "Unsupported file format. Only PDF and TXT are
supported.", "type": "value_error"}]
250 |         )
251 |
252 |     file.file.seek(0, 2)
253 |     size = file.file.tell()
254 |     file.file.seek(0)
255 |
256 |     if size == 0:
257 |         raise HTTPException(
258 |             status_code=422,
259 |             detail=[{"loc": ["body", "file"], "msg": "Uploaded syllabus file is empty", "type":
"value_error"}]
260 |         )
261 |
262 |     if size > 10 * 1024 * 1024:
263 |         raise HTTPException(
264 |             status_code=422,
265 |             detail=[{"loc": ["body", "file"], "msg": "File is too large. Maximum size allowed is 10MB.",
"type": "value_error"}]
266 |         )
267 |
268 |     # 1. Verify course ownership
269 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
270 |     if not course:
271 |         raise HTTPException(status_code=404, detail="Course not found")
272 |
273 |     # Create scratch uploads folder in app workspace
274 |     upload_dir = os.path.join(os.getcwd(), "uploads")
275 |     os.makedirs(upload_dir, exist_ok=True)
276 |
277 |     file_path = os.path.join(upload_dir, file.filename)
278 |     with open(file_path, "wb") as buffer:
279 |         shutil.copyfileobj(file.file, buffer)
280 |
281 |     # Extract syllabus raw text
282 |     from app.services.rag_service import extract_text_from_pdf
283 |
284 |     raw_text = ""
285 |     if file.filename.lower().endswith(".pdf"):
286 |         raw_text = extract_text_from_pdf(file_path)
287 |     else:
288 |         try:
289 |             with open(file_path, "r", encoding="utf-8") as f:
290 |                 raw_text = f.read()
291 |         except Exception:
292 |             raise HTTPException(status_code=400, detail="Could not read uploaded syllabus file.")
293 |
294 |     if not raw_text or len(raw_text.strip()) == 0:
295 |         raw_text = f"Syllabus file: {file.filename}\nDefault Course: Advanced Machine Learning and Neural
Networks"
296 |
297 |     # Save Syllabus record
298 |     syllabus_record = Syllabus(
299 |         course_id=course_id,
300 |         raw_text=raw_text,
301 |         file_name=file.filename,
302 |         file_path=file_path

```



```

303 |     )
304 |     db.add(syllabus_record)
305 |
306 |     # Process document for RAG search
307 |     try:
308 |         process_and_index_document(db, course_id, file.filename, file_path, document_type="SYLLABUS")
309 |     except Exception as e:
310 |         print(f"[RAG] Indexing syllabus document: {e}")
311 |
312 |     db.commit()
313 |
314 |     # 2. RUN LANGGRAPH WORKFLOW
315 |     compiled_graph = build_workflow()
316 |
317 |     from app.models import PersonalizationProfile
318 |     profile_record = db.query(PersonalizationProfile).filter(PersonalizationProfile.course_id ==
course_id).first()
319 |     personalization_profile = profile_record.profile if profile_record else {}
320 |
321 |     initial_state = {
322 |         "course_id": course_id,
323 |         "syllabus_text": raw_text,
324 |         "curriculum_map": {},
325 |         "learning_outcomes": [],
326 |         "curriculum_plan": {},
327 |         "slide_deck": [],
328 |         "instructor_notes": [],
329 |         "assessment_bank": [],
330 |         "bloom_report": {},
331 |         "readiness_score": {},
332 |         "industry_gap_report": {},
333 |         "personalization_profile": personalization_profile,
334 |         "logs": ["Starting multi-agent syllabus analysis pipeline..."],
335 |         "current_agent": "Curriculum Analysis Agent",
336 |         "pipeline_telemetry": []
337 |     }
338 |
339 |     # Invoke State Graph
340 |     final_state = compiled_graph.invoke(initial_state)
341 |
342 |     error_info = final_state.get("error_info")
343 |     save_workflow_outputs(db, course_id, final_state)
344 |
345 |     if error_info:
346 |         return {
347 |             "status": "partial_generation",
348 |             "failed_agent": error_info.get("failed_agent"),
349 |             "error_type": error_info.get("error_type"),
350 |             "successful_agents": error_info.get("successful_agents", []),
351 |             "retry_recommended": error_info.get("retry_recommended", True),
352 |             "logs": final_state["logs"],
353 |             "pipeline_telemetry": final_state.get("pipeline_telemetry", [])
354 |         }
355 |
356 |     return {
357 |         "status": "Success",
358 |         "message": "Syllabus processed and educational package generated successfully.",
359 |         "course_id": course_id,
360 |         "logs": final_state["logs"],
361 |         "pipeline_telemetry": final_state.get("pipeline_telemetry", [])
362 |     }
363 |
364 | @router.post("/{course_id}/regenerate", dependencies=[Depends(limit_analysis_generation)])
365 | def regenerate_course_deck(
366 |     course_id: int,
367 |     db: Session = Depends(get_db),

```

```

368 |     current_user: User = Depends(get_current_user)
369 | ):
370 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
371 |     if not course:
372 |         raise HTTPException(status_code=404, detail="Course not found")
373 |
374 |     syllabus_record = db.query(Syllabus).filter(Syllabus.course_id ==
course_id).order_by(Syllabus.id.desc()).first()
375 |     if not syllabus_record:
376 |         raise HTTPException(status_code=400, detail="No syllabus found for this course. Please upload a
syllabus first.")
377 |
378 |     raw_text = syllabus_record.raw_text
379 |     compiled_graph = build_workflow()
380 |
381 |     from app.models import PersonalizationProfile
382 |     profile_record = db.query(PersonalizationProfile).filter(PersonalizationProfile.course_id ==
course_id).first()
383 |     personalization_profile = profile_record.profile if profile_record else {}
384 |
385 |     initial_state = {
386 |         "course_id": course_id,
387 |         "syllabus_text": raw_text,
388 |         "curriculum_map": {},
389 |         "learning_outcomes": [],
390 |         "curriculum_plan": {},
391 |         "slide_deck": [],
392 |         "instructor_notes": [],
393 |         "assessment_bank": [],
394 |         "bloom_report": {},
395 |         "readiness_score": {},
396 |         "industry_gap_report": {},
397 |         "personalization_profile": personalization_profile,
398 |         "logs": ["Initiating personalized multi-agent regeneration workflow..."],
399 |         "current_agent": "Curriculum Analysis Agent",
400 |         "pipeline_telemetry": []
401 |     }
402 |
403 |     final_state = compiled_graph.invoke(initial_state)
404 |
405 |     error_info = final_state.get("error_info")
406 |     save_workflow_outputs(db, course_id, final_state)
407 |
408 |     if error_info:
409 |         return {
410 |             "status": "partial_generation",
411 |             "failed_agent": error_info.get("failed_agent"),
412 |             "error_type": error_info.get("error_type"),
413 |             "successful_agents": error_info.get("successful_agents", []),
414 |             "retry_recommended": error_info.get("retry_recommended", True),
415 |             "logs": final_state["logs"],
416 |             "pipeline_telemetry": final_state.get("pipeline_telemetry", [])
417 |         }
418 |
419 |     return {
420 |         "status": "Success",
421 |         "message": "Syllabus processed and educational package generated successfully.",
422 |         "course_id": course_id,
423 |         "logs": final_state["logs"],
424 |         "pipeline_telemetry": final_state.get("pipeline_telemetry", [])
425 |     }
426 |
427 | @router.get("/{course_id}/traceability")
428 | def get_course_traceability(
429 |     course_id: int,
430 |     db: Session = Depends(get_db),

```

```

431 |     current_user: User = Depends(get_current_user)
432 | ):
433 |     # Verify course ownership
434 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
435 |     if not course:
436 |         raise HTTPException(status_code=404, detail="Course not found")
437 |
438 |     outcomes = db.query(LearningOutcome).filter(LearningOutcome.course_id == course_id).all()
439 |     slides = db.query(GeneratedSlide).filter(GeneratedSlide.course_id ==
course_id).order_by(GeneratedSlide.slide_index).all()
440 |     notes = db.query(InstructorNote).filter(InstructorNote.course_id ==
course_id).order_by(InstructorNote.slide_index).all()
441 |     assessments = db.query(Assessment).filter(Assessment.course_id == course_id).all()
442 |
443 |     notes_by_index = {n.slide_index: n for n in notes}
444 |
445 |     stop_words = {
446 |         "a", "an", "the", "and", "or", "but", "in", "on", "at", "to", "for", "with", "of", "about",
447 |         "against", "across", "recall", "explain", "apply", "analyze", "evaluate", "design", "create",
448 |         "basic", "core", "techniques", "models", "understands", "comprehends", "methods"
449 |     }
450 |
451 |     traceability_report = []
452 |
453 |     for outcome in outcomes:
454 |         outcome_words = [
455 |             w.strip(",.()\\"";").lower()
456 |             for w in outcome.outcome_text.split()
457 |             if w.strip(",.()\\"";").lower() not in stop_words and len(w.strip(",.()\\"";")) > 2
458 |         ]
459 |
460 |         # Scored matching slides
461 |         scored_slides = []
462 |         for s in slides:
463 |             slide_text = (s.title + " " + " ".join(s.content)).lower()
464 |             score = sum(1 for w in outcome_words if w in slide_text)
465 |             if score > 0:
466 |                 scored_slides.append((s, score))
467 |
468 |         # Sort by match score descending, then by slide index ascending
469 |         scored_slides.sort(key=lambda x: (-x[1], x[0].slide_index))
470 |         related_slides = [s for s, score in scored_slides[:3]]
471 |
472 |         related_slides_data = [
473 |             {"slide_index": s.slide_index, "title": s.title}
474 |             for s in related_slides
475 |         ]
476 |
477 |         related_notes_data = []
478 |         for s in related_slides:
479 |             n = notes_by_index.get(s.slide_index)
480 |             if n:
481 |                 related_notes_data.append({
482 |                     "slide_index": n.slide_index,
483 |                     "talking_points_count": len(n.talking_points) if n.talking_points else 0
484 |                 })
485 |
486 |         related_assessments = [
487 |             a for a in assessments if a.learning_outcome_id == outcome.id
488 |         ]
489 |
490 |         related_assessments_data = [
491 |             {
492 |                 "id": a.id,
493 |                 "question_text": a.question_text,
494 |                 "bloom_level": a.bloom_level,

```

```
495 |         "question_type": a.question_type,
496 |         "correct_answer": a.correct_answer,
497 |         "options": a.options
498 |     }
499 |     for a in related_assessments
500 | ]
501 |
502 | # Calculate coverage score
503 | slides_part = min(len(related_slides_data), 3) / 3.0 * 50.0
504 | assessments_part = min(len(related_assessments_data), 2) / 2.0 * 50.0
505 | coverage = int(slides_part + assessments_part)
506 |
507 | traceability_report.append({
508 |     "id": outcome.id,
509 |     "outcome_text": outcome.outcome_text,
510 |     "bloom_level": outcome.bloom_level,
511 |     "related_slides": related_slides_data,
512 |     "related_notes": related_notes_data,
513 |     "related_assessments": related_assessments_data,
514 |     "coverage_percentage": coverage
515 | })
516 |
517 | return traceability_report
518 |
```

File: backend/app/routers/studio.py

```
1 | from typing import List, Optional, Dict, Any
2 | from fastapi import APIRouter, Depends, HTTPException
3 | from sqlalchemy.orm import Session
4 | from pydantic import BaseModel
5 | from app.database import get_db
6 | from app.routers.auth import get_current_user
7 | from app.models import (
8 |     User, Course, GeneratedSlide, InstructorNote, Assessment, PersonalizationProfile
9 | )
10 |
11 | router = APIRouter(prefix="/studio", tags=["AI Studio"])
12 |
13 | # Pydantic schemas for edits
14 | class SlideUpdatePayload(BaseModel):
15 |     title: str
16 |     content: List[str]
17 |     suggested_visuals: Optional[str] = None
18 |
19 | class NoteUpdatePayload(BaseModel):
20 |     talking_points: List[str]
21 |     teaching_tips: Optional[str] = None
22 |     examples: Optional[List[str]] = None
23 |
24 | class AssessmentUpdatePayload(BaseModel):
25 |     question_text: str
26 |     question_type: str
27 |     options: Optional[List[str]] = None
28 |     correct_answer: Optional[str] = None
29 |     bloom_level: str
30 |
31 | class PersonalizationPayload(BaseModel):
32 |     profile: Dict[str, Any]
33 |
34 | @router.put("/slides/{slide_id}")
35 | def update_slide(
36 |     slide_id: int,
37 |     payload: SlideUpdatePayload,
38 |     db: Session = Depends(get_db),
39 |     current_user: User = Depends(get_current_user)
40 | ):
41 |     slide = db.query(GeneratedSlide).filter(GeneratedSlide.id == slide_id).first()
42 |     if not slide:
43 |         raise HTTPException(status_code=404, detail="Slide not found")
44 |
45 |     # Verify course ownership
46 |     course = db.query(Course).filter(Course.id == slide.course_id, Course.user_id ==
current_user.id).first()
47 |     if not course:
48 |         raise HTTPException(status_code=403, detail="Not authorized to edit this course content")
49 |
50 |     slide.title = payload.title
51 |     slide.content = payload.content
52 |     slide.suggested_visuals = payload.suggested_visuals
53 |
54 |     db.commit()
55 |     return {"status": "Success", "message": f"Slide {slide_id} updated successfully", "slide": {
56 |         "id": slide.id, "title": slide.title, "content": slide.content, "suggested_visuals":
slide.suggested_visuals
57 |     }}
58 |
59 | @router.put("/notes/{note_id}")
60 | def update_note(
```

```

61 |     note_id: int,
62 |     payload: NoteUpdatePayload,
63 |     db: Session = Depends(get_db),
64 |     current_user: User = Depends(get_current_user)
65 | ):
66 |     note = db.query(InstructorNote).filter(InstructorNote.id == note_id).first()
67 |     if not note:
68 |         raise HTTPException(status_code=404, detail="Instructor note not found")
69 |
70 |     course = db.query(Course).filter(Course.id == note.course_id, Course.user_id ==
current_user.id).first()
71 |     if not course:
72 |         raise HTTPException(status_code=403, detail="Not authorized to edit this course content")
73 |
74 |     note.talking_points = payload.talking_points
75 |     note.teaching_tips = payload.teaching_tips
76 |     note.examples = payload.examples
77 |
78 |     db.commit()
79 |     return {"status": "Success", "message": f"Note {note_id} updated successfully", "note": {
80 |         "id": note.id, "talking_points": note.talking_points, "teaching_tips": note.teaching_tips,
"examples": note.examples
81 |     }}
82 |
83 | @router.put("/assessments/{assessment_id}")
84 | def update_assessment(
85 |     assessment_id: int,
86 |     payload: AssessmentUpdatePayload,
87 |     db: Session = Depends(get_db),
88 |     current_user: User = Depends(get_current_user)
89 | ):
90 |     assessment = db.query(Assessment).filter(Assessment.id == assessment_id).first()
91 |     if not assessment:
92 |         raise HTTPException(status_code=404, detail="Assessment not found")
93 |
94 |     course = db.query(Course).filter(Course.id == assessment.course_id, Course.user_id ==
current_user.id).first()
95 |     if not course:
96 |         raise HTTPException(status_code=403, detail="Not authorized to edit this course content")
97 |
98 |     assessment.question_text = payload.question_text
99 |     assessment.question_type = payload.question_type
100 |     assessment.options = payload.options
101 |     assessment.correct_answer = payload.correct_answer
102 |     assessment.bloom_level = payload.bloom_level
103 |
104 |     db.commit()
105 |     return {"status": "Success", "message": f"Assessment {assessment_id} updated successfully"}
106 |
107 | @router.post("/courses/{course_id}/personalize")
108 | def configure_personalization(
109 |     course_id: int,
110 |     payload: PersonalizationPayload,
111 |     db: Session = Depends(get_db),
112 |     current_user: User = Depends(get_current_user)
113 | ):
114 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
115 |     if not course:
116 |         raise HTTPException(status_code=404, detail="Course not found")
117 |
118 |     profile = db.query(PersonalizationProfile).filter(PersonalizationProfile.course_id ==
course_id).first()
119 |     if not profile:
120 |         profile = PersonalizationProfile(course_id=course_id, profile=payload.profile)
121 |         db.add(profile)
122 |     else:

```

```
123 |         profile.profile = payload.profile
124 |
125 |         db.commit()
126 |         return {"status": "Success", "message": "Personalization profile configured successfully", "profile":
profile.profile}
127 |
```

File: backend/app/routers/export.py

```
1 | import os
2 | import csv
3 | import io
4 | import json
5 | from fastapi import APIRouter, Depends, HTTPException, Response
6 | from fastapi.responses import FileResponse, PlainTextResponse
7 | from sqlalchemy.orm import Session
8 | from app.database import get_db
9 | from app.routers.auth import get_current_user
10 | from app.models import User, Course, GeneratedSlide, InstructorNote, Assessment, PptExport
11 | from app.services.exporter import generate_pptx_deck, generate_pdf_package
12 |
13 | router = APIRouter(prefix="/export", tags=["Exports"])
14 |
15 | def get_interactive_quiz_template(course_title: str, course_desc: str, questions_json: str) -> str:
16 |     return f"""<!DOCTYPE html>
17 |     <html lang="en">
18 |     <head>
19 |         <meta charset="UTF-8">
20 |         <meta name="viewport" content="width=device-width, initial-scale=1.0">
21 |         <title>Interactive Quiz: {course_title}</title>
22 |         <script src="https://cdn.tailwindcss.com"></script>
23 |         <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800;900&display=swap"
rel="stylesheet">
24 |         <style>
25 |             body {{
26 |                 font-family: 'Outfit', sans-serif;
27 |                 background: radial-gradient(circle at 50% 50%, #0f172a 0%, #020617 100%);
28 |                 min-height: 100vh;
29 |                 color: #f8fafc;
30 |             }}
31 |             .glass-panel {{
32 |                 background: rgba(15, 23, 42, 0.65);
33 |                 backdrop-filter: blur(16px);
34 |                 -webkit-backdrop-filter: blur(16px);
35 |                 border: 1px solid rgba(255, 255, 255, 0.08);
36 |                 box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
37 |             }}
38 |             .pulse-light {{
39 |                 box-shadow: 0 0 15px rgba(56, 189, 248, 0.2);
40 |                 animation: pulse 3s infinite;
41 |             }}
42 |             @keyframes pulse {{
43 |                 0%, 100% {{ box-shadow: 0 0 15px rgba(56, 189, 248, 0.2); }}
44 |                 50% {{ box-shadow: 0 0 25px rgba(56, 189, 248, 0.4); }}
45 |             }}
46 |             ::-webkit-scrollbar {{
47 |                 width: 8px;
48 |             }}
49 |             ::-webkit-scrollbar-track {{
50 |                 background: rgba(15, 23, 42, 0.2);
51 |             }}
52 |             ::-webkit-scrollbar-thumb {{
53 |                 background: rgba(255, 255, 255, 0.1);
54 |                 border-radius: 4px;
55 |             }}
56 |             ::-webkit-scrollbar-thumb:hover {{
57 |                 background: rgba(255, 255, 255, 0.2);
58 |             }}
59 |         </style>
60 |     </head>
```



```

61 | <body class="flex flex-col items-center justify-center p-4 md:p-8 min-h-screen">
62 |
63 |     <!-- MAIN APP CONTAINER -->
64 |     <div class="w-full max-w-3xl flex flex-col gap-6">
65 |
66 |         <!-- HEADER LOGO / TITLE -->
67 |         <div class="flex items-center justify-between px-2">
68 |             <div class="flex items-center gap-2">
69 |                 <span class="p-1.5 rounded-lg bg-gradient-to-tr from-sky-500 to-indigo-500 text-white
font-black text-sm tracking-widest shadow-md">K</span>
70 |                 <span class="text-sm font-black text-white tracking-widest">KALYX <span
class="text-sky-400 font-medium">ASSESSMENT</span></span>
71 |             </div>
72 |             <div class="text-[10px] text-slate-500 tracking-wider uppercase font-mono bg-slate-900 px-2.5
py-1 rounded-full border border-slate-800">
73 |                 Self-Hosted Test
74 |             </div>
75 |         </div>
76 |
77 |         <!-- 1. WELCOME SCREEN -->
78 |         <div id="welcome-screen" class="glass-panel p-8 md:p-12 rounded-3xl flex flex-col items-center
text-center gap-6 animate-fade-in">
79 |             <div class="w-16 h-16 rounded-2xl bg-indigo-500/10 border border-indigo-500/20
text-indigo-400 flex items-center justify-center shadow-lg">
80 |                 <svg class="w-8 h-8" fill="none" stroke="currentColor" stroke-width="1.5" viewBox="0 0 24
24">
81 |                     <path stroke-linecap="round" stroke-linejoin="round" d="M4.26 10.147a60.436 60.436 0
00-.491 6.347A48.62 48.62 0 0112 20.9c2.79 0 5.422-.94
7.524-2.535.214-1.613-3.24.292-4.878m-15.556-3.34h15.556m-15.556 0a48.188 48.188 0 011.87-7.758A48.62 48.62 0
0112 3c1.765 0 3.443.345 4.978.975a48.188 48.188 0 011.87 7.758m-15.556-.001a42.947 42.947 0
015.008-.387m10.548.387a42.947 42.947 0 00-5.008-.387m0 0a89.29 89.29 0 00-5.008.387m5.008-.387h.01"></path>
82 |                 </svg>
83 |             </div>
84 |
85 |             <div>
86 |                 <h1 class="text-2xl md:text-3xl font-black text-white tracking-tight
leading-tight">{course_title}</h1>
87 |                 <p class="text-slate-400 text-xs md:text-sm mt-3 max-w-lg
leading-relaxed">{course_desc}</p>
88 |             </div>
89 |
90 |             <div class="grid grid-cols-2 gap-4 w-full max-w-md my-4">
91 |                 <div class="bg-slate-950/55 border border-slate-900 p-4 rounded-2xl flex flex-col
items-center">
92 |                     <span class="text-[10px] text-slate-500 font-bold uppercase tracking-wider">Total
Questions</span>
93 |                     <span id="welcome-q-count" class="text-xl font-black text-white mt-1">0</span>
94 |                 </div>
95 |                 <div class="bg-slate-950/55 border border-slate-900 p-4 rounded-2xl flex flex-col
items-center">
96 |                     <span class="text-[10px] text-slate-500 font-bold uppercase tracking-wider">Estimated
Time</span>
97 |                     <span id="welcome-time" class="text-xl font-black text-white mt-1">10 min</span>
98 |                 </div>
99 |             </div>
100 |
101 |             <button onclick="startQuiz()" class="w-full max-w-sm py-4 bg-gradient-to-r from-sky-500
to-indigo-500 hover:from-sky-400 hover:to-indigo-400 text-white rounded-2xl text-sm font-bold shadow-lg
shadow-indigo-500/10 hover:shadow-indigo-500/20 active:scale-[0.98] transition-all pulse-light">
102 |                 Begin Challenge
103 |             </button>
104 |         </div>
105 |
106 |         <!-- 2. QUIZ SCREEN (HIDDEN BY DEFAULT) -->
107 |         <div id="quiz-screen" class="hidden glass-panel p-6 md:p-8 rounded-3xl flex flex-col gap-6">
108 |

```

```

109 |         <!-- Progress & Stats Bar -->
110 |         <div class="flex justify-between items-center text-xs text-slate-400">
111 |             <span class="font-mono" id="quiz-progress-text">Question 1 of 10</span>
112 |             <span class="px-2.5 py-0.5 rounded bg-indigo-500/10 border border-indigo-500/20
text-indigo-400 font-black tracking-widest text-[9px]" id="bloom-badge">
113 |                 REMEMBERING
114 |             </span>
115 |         </div>
116 |
117 |         <!-- Progress Bar -->
118 |         <div class="w-full bg-slate-950 h-1.5 rounded-full overflow-hidden">
119 |             <div id="quiz-progress-bar" class="bg-gradient-to-r from-sky-400 to-indigo-500 h-full
rounded-full transition-all duration-300" style="width: 10%;"></div>
120 |         </div>
121 |
122 |         <!-- Question Text -->
123 |         <h2 id="question-text" class="text-base md:text-lg font-black text-white leading-relaxed
tracking-tight min-h-[50px]">
124 |             Loading question...
125 |         </h2>
126 |
127 |         <!-- Options Grid -->
128 |         <div id="options-container" class="grid grid-cols-1 gap-3 my-2">
129 |             <!-- Javascript will inject options here -->
130 |         </div>
131 |
132 |         <!-- Navigation Buttons -->
133 |         <div class="flex justify-between items-center pt-4 border-t border-white/5">
134 |             <button onclick="prevQuestion()" id="prev-btn" class="px-5 py-2.5 bg-slate-950 border
border-slate-900 hover:border-slate-800 text-xs font-bold text-slate-400 hover:text-white rounded-xl
active:scale-95 transition-all">
135 |                 Back
136 |             </button>
137 |
138 |             <button onclick="nextOrSubmit()" id="next-btn" class="px-6 py-2.5 bg-gradient-to-r
from-sky-500 to-indigo-500 hover:from-sky-400 hover:to-indigo-400 text-xs font-bold text-white rounded-xl
active:scale-95 transition-all shadow-md">
139 |                 Next Question
140 |             </button>
141 |         </div>
142 |     </div>
143 |
144 |     <!-- 3. RESULTS SCREEN (HIDDEN BY DEFAULT) -->
145 |     <div id="result-screen" class="hidden flex flex-col gap-6">
146 |
147 |         <!-- Result Stats Card -->
148 |         <div class="glass-panel p-8 rounded-3xl flex flex-col md:flex-row items-center justify-around
gap-8 text-center md:text-left">
149 |
150 |             <!-- Circular Chart -->
151 |             <div class="relative flex items-center justify-center">
152 |                 <svg class="w-36 h-36 transform -rotate-90">
153 |                     <circle cx="72" cy="72" r="60" stroke="rgba(255,255,255,0.03)" stroke-width="12"
fill="transparent" />
154 |                     <circle id="progress-circle" cx="72" cy="72" r="60"
stroke="url(#result-gradient)" stroke-dasharray="376.8" stroke-dashoffset="376.8" stroke-width="12"
stroke-linecap="round" fill="transparent" class="transition-all duration-1000 ease-out" />
155 |                 <defs>
156 |                     <linearGradient id="result-gradient" x1="0%" y1="0%" x2="100%" y2="100%">
157 |                         <stop offset="0%" stop-color="#38bdf8" />
158 |                         <stop offset="100%" stop-color="#6366f1" />
159 |                     </linearGradient>
160 |                 </defs>
161 |             </div>
162 |             <span id="percentage-text" class="absolute text-3xl font-black text-white">0%</span>
163 |         </div>

```

```

164 |
165 |         <div class="flex flex-col gap-2 max-w-sm">
166 |             <span class="text-[10px] text-slate-500 font-bold block uppercase
tracking-wider">Assessment Outcome</span>
167 |             <h2 id="rating-title" class="text-2xl font-black text-white">Elite Mastery!</h2>
168 |             <p id="rating-desc" class="text-xs text-slate-400 leading-relaxed mt-1">Excellent
job! You have fully understood the primary building blocks of this curriculum.</p>
169 |             <div class="mt-4 flex items-center gap-6">
170 |                 <div>
171 |                     <span class="text-[10px] text-slate-500 font-bold block uppercase
tracking-wider">SCORE</span>
172 |                     <span id="score-fraction" class="text-lg font-black text-white mt-0.5">0 /
0</span>
173 |                 </div>
174 |                 <button onclick="location.reload()" class="px-4 py-2 bg-indigo-500/10 border
border-indigo-500/20 hover:bg-indigo-500/20 text-xs font-bold text-indigo-400 rounded-xl transition-all">
175 |                     Retake Challenge
176 |                 </button>
177 |             </div>
178 |         </div>
179 |     </div>
180 |
181 |     <!-- Review Section Title -->
182 |     <div class="px-1 flex justify-between items-center">
183 |         <h3 class="text-sm font-black text-white uppercase tracking-wider">Answer Review
Board</h3>
184 |         <span class="text-xs text-slate-500 font-medium">Auto-Graded locally</span>
185 |     </div>
186 |
187 |     <!-- Review Cards List -->
188 |     <div id="review-list" class="flex flex-col gap-4">
189 |         <!-- Javascript will inject review cards -->
190 |     </div>
191 |
192 | </div>
193 |
194 | </div>
195 |
196 | <!-- CORE INTERACTIVE JS -->
197 | <script>
198 |     const questions = {questions_json};
199 |     let currentIdx = 0;
200 |     const userAnswers = {}; // key: index, value: option index
201 |
202 |     // Setup Welcome details
203 |     document.getElementById('welcome-q-count').innerText = questions.length;
204 |     document.getElementById('welcome-time').innerText = Math.max(5, Math.round(questions.length *
0.75)) + " min";
205 |
206 |     function startQuiz() {{
207 |         document.getElementById('welcome-screen').classList.add('hidden');
208 |         document.getElementById('quiz-screen').classList.remove('hidden');
209 |         renderQuestion();
210 |     }}
211 |
212 |     function renderQuestion() {{
213 |         if (!questions.length) return;
214 |         const q = questions[currentIdx];
215 |
216 |         // Text values
217 |         document.getElementById('quiz-progress-text').innerText = `Question ${currentIdx + 1} of
${questions.length}`;
218 |         document.getElementById('question-text').innerText = q.question_text;
219 |
220 |         // Bloom badge
221 |         const badge = document.getElementById('bloom-badge');

```

```

222 |         badge.innerText = q.bloom_level ? q.bloom_level.toUpperCase() : "GENERAL";
223 |
224 |         // Progress bar width
225 |         const percent = ((currentIdx + 1) / questions.length) * 100;
226 |         document.getElementById('quiz-progress-bar').style.width = percent + "%";
227 |
228 |         // Load Options
229 |         const container = document.getElementById('options-container');
230 |         container.innerHTML = '';
231 |
232 |         if (q.options && q.options.length) {{
233 |             q.options.forEach((opt, idx) => {{
234 |                 const isSelected = userAnswers[currentIdx] === idx;
235 |                 const card = document.createElement('div');
236 |
237 |                 const charCode = String.fromCharCode(65 + idx); // A, B, C, D
238 |
239 |                 card.onclick = () => selectOption(idx);
240 |
241 |                 // Style depends on select status
242 |                 let style = "bg-slate-950/65 border-slate-900 text-slate-300 hover:border-slate-800
hover:text-white";
243 |                 if (isSelected) {{
244 |                     style = "bg-sky-500/10 border-sky-500 text-sky-200 shadow-md shadow-sky-500/5";
245 |                 }}
246 |
247 |                 card.className = `flex items-center gap-4 px-5 py-4 border rounded-2xl cursor-pointer
select-none transition-all duration-200 ${style}`;
248 |                 card.innerHTML = `
249 |                     <span class="flex-shrink-0 w-8 h-8 rounded-xl flex items-center justify-center
text-xs font-black ${isSelected ? "bg-sky-500 text-slate-950" : "bg-slate-900 text-slate-500"}">
250 |                         ${charCode}
251 |                     </span>
252 |                     <span class="text-xs leading-relaxed font-semibold">${opt}</span>
253 |                 `;
254 |                 container.appendChild(card);
255 |             }});
256 |         }}
257 |
258 |         // Nav states
259 |         document.getElementById('prev-btn').disabled = currentIdx === 0;
260 |         document.getElementById('prev-btn').style.opacity = currentIdx === 0 ? '0.4' : '1';
261 |
262 |         const nextBtn = document.getElementById('next-btn');
263 |         if (currentIdx === questions.length - 1) {{
264 |             nextBtn.innerText = "Submit Assessment";
265 |             nextBtn.className = "px-6 py-2.5 bg-gradient-to-r from-emerald-500 to-teal-500
hover:from-emerald-400 hover:to-teal-400 text-xs font-bold text-white rounded-xl active:scale-95 transition-all
shadow-md shadow-emerald-500/10 cursor-pointer";
266 |         }} else {{
267 |             nextBtn.innerText = "Next Question";
268 |             nextBtn.className = "px-6 py-2.5 bg-gradient-to-r from-sky-500 to-indigo-500
hover:from-sky-400 hover:to-indigo-400 text-xs font-bold text-white rounded-xl active:scale-95 transition-all
shadow-md shadow-indigo-500/10 cursor-pointer";
269 |         }}
270 |     }}
271 |
272 |     function selectOption(optIdx) {{
273 |         userAnswers[currentIdx] = optIdx;
274 |         renderQuestion();
275 |     }}
276 |
277 |     function prevQuestion() {{
278 |         if (currentIdx > 0) {{
279 |             currentIdx--;
280 |             renderQuestion();

```

```

281 |     }}
282 | }}
283 |
284 | function nextOrSubmit() {{
285 |     if (currentIdx === questions.length - 1) {{
286 |         // Submit!
287 |         submitQuiz();
288 |     }} else {{
289 |         currentIdx++;
290 |         renderQuestion();
291 |     }}
292 | }}
293 |
294 | // Allow keyboard triggers
295 | window.addEventListener('keydown', (e) => {{
296 |     if (document.getElementById('quiz-screen').classList.contains('hidden')) return;
297 |
298 |     const key = e.key.toLowerCase();
299 |     if (key === 'a' || key === '1') selectOption(0);
300 |     if (key === 'b' || key === '2') selectOption(1);
301 |     if (key === 'c' || key === '3') selectOption(2);
302 |     if (key === 'd' || key === '4') selectOption(3);
303 |
304 |     if (key === 'enter') nextOrSubmit();
305 |     if (key === 'arrowleft' && currentIdx > 0) prevQuestion();
306 | }});
307 |
308 | function submitQuiz() {{
309 |     document.getElementById('quiz-screen').classList.add('hidden');
310 |     document.getElementById('result-screen').classList.remove('hidden');
311 |     calculateResults();
312 | }}
313 |
314 | function calculateResults() {{
315 |     let score = 0;
316 |     questions.forEach((q, idx) => {{
317 |         const correctVal = q.correct_answer ? q.correct_answer.trim().toLowerCase() : "";
318 |         const selectedOptIdx = userAnswers[idx];
319 |
320 |         if (selectedOptIdx !== undefined && q.options[selectedOptIdx]) {{
321 |             const selectedVal = q.options[selectedOptIdx].trim().toLowerCase();
322 |             if (selectedVal === correctVal || correctVal.includes(selectedVal) ||
selectedVal.includes(correctVal)) {{
323 |                 score++;
324 |             }}
325 |         }}
326 |     }});
327 |
328 |     const percent = Math.round((score / questions.length) * 100);
329 |     document.getElementById('percentage-text').innerText = percent + "%";
330 |     document.getElementById('score-fraction').innerText = score + " / " + questions.length;
331 |
332 |     // Draw SVG circle progress
333 |     const circle = document.getElementById('progress-circle');
334 |     const radius = 60;
335 |     const circumference = 2 * Math.PI * radius;
336 |     circle.style.strokeDasharray = circumference;
337 |     const offset = circumference - (percent / 100) * circumference;
338 |     circle.style.strokeDashoffset = offset;
339 |
340 |     // Rating titles
341 |     let rating = "Keep Studying!";
342 |     let ratingDesc = "Review the course materials and try again to improve your grasp on the
topics.";
343 |     let ratingColor = "text-rose-400";
344 |     if (percent >= 90) {{

```

```

345 |         rating = "Elite Mastery!";
346 |         ratingDesc = "Outstanding performance! You have fully mastered the core concepts of this
course.";
347 |         ratingColor = "text-emerald-400";
348 |     }} else if (percent >= 70) {{
349 |         rating = "Competent Scholar!";
350 |         ratingDesc = "Great job! You have a solid understanding of most topics covered.";
351 |         ratingColor = "text-sky-400";
352 |     }} else if (percent >= 50) {{
353 |         rating = "Developing Thinker!";
354 |         ratingDesc = "Good effort, but there is still room for improvement in some areas.";
355 |         ratingColor = "text-amber-400";
356 |     }}
357 |
358 |     const ratingEl = document.getElementById('rating-title');
359 |     ratingEl.innerText = rating;
360 |     ratingEl.className = "text-2xl font-black mt-4 " + ratingColor;
361 |     document.getElementById('rating-desc').innerText = ratingDesc;
362 |
363 |     // Build Review Board
364 |     const reviewList = document.getElementById('review-list');
365 |     reviewList.innerHTML = '';
366 |     questions.forEach((q, idx) => {{
367 |         const selectedOptIdx = userAnswers[idx];
368 |         const correctVal = q.correct_answer ? q.correct_answer.trim().toLowerCase() : "";
369 |         let isCorrect = false;
370 |         let selectedText = "No answer selected";
371 |
372 |         if (selectedOptIdx !== undefined && q.options[selectedOptIdx]) {{
373 |             selectedText = q.options[selectedOptIdx];
374 |             const selectedVal = selectedText.trim().toLowerCase();
375 |             if (selectedVal === correctVal || correctVal.includes(selectedVal) ||
selectedVal.includes(correctVal)) {{
376 |                 isCorrect = true;
377 |             }}
378 |         }}
379 |
380 |         const card = document.createElement('div');
381 |         card.className = "glass-panel p-6 rounded-2xl border " + (isCorrect ?
"border-emerald-500/20 bg-emerald-950/5" : "border-rose-500/20 bg-rose-950/5") + " flex flex-col gap-3";
382 |
383 |         let optionsHtml = '';
384 |         q.options.forEach((opt, oIdx) => {{
385 |             let optStyle = "bg-slate-950/60 border border-slate-900/60 text-slate-400";
386 |             if (oIdx === selectedOptIdx) {{
387 |                 optStyle = isCorrect ? "bg-emerald-500/10 border-emerald-500 text-emerald-300
font-bold" : "bg-rose-500/10 border-rose-500 text-rose-300 font-bold";
388 |             }} else {{
389 |                 // Check if this option represents the correct answer
390 |                 const optVal = opt.trim().toLowerCase();
391 |                 if (correctVal && (optVal === correctVal || correctVal.includes(optVal) ||
optVal.includes(correctVal))) {{
392 |                     optStyle = "bg-emerald-500/5 border-emerald-500/30 text-emerald-400/90";
393 |                 }}
394 |             }}
395 |             optionsHtml += `<div class="px-4 py-2.5 rounded-xl text-xs
${{optStyle}}">${{opt}}</div>`;
396 |         }});
397 |
398 |         card.innerHTML = `
399 |             <div class="flex justify-between items-center text-[10px]">
400 |                 <span class="px-2.5 py-0.5 rounded bg-slate-900 border border-slate-800
text-slate-400 font-mono font-bold">
401 |                     QUESTION ${{idx + 1}}
402 |                 </span>
403 |                 <span class="px-2 py-0.5 rounded bg-indigo-500/10 border border-indigo-500/20

```

```

text-indigo-400 font-extrabold uppercase">
404 |                 ${{q.bloom_level || "GENERAL"}}
405 |             </span>
406 |         </div>
407 |         <h4 class="text-sm font-bold text-white leading-relaxed
mt-1">${{q.question_text}}</h4>
408 |         <div class="grid grid-cols-1 md:grid-cols-2 gap-2 my-2">
409 |             ${{optionsHtml}}
410 |         </div>
411 |         <div class="flex items-center gap-2 mt-1">
412 |             <span class="text-xs font-bold ${{isCorrect ? "text-emerald-400" :
"text-rose-400"}} flex items-center gap-1">
413 |                 ${{isCorrect ?
414 |                     `<svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2.5"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" d="M9 12.75L11.25 15 15 9.75M21 12a9 9
0 11-18 0 9 9 0 0118 0z"></path></svg> Correct` :
415 |                     `<svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2.5"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" d="M9.75 9.75l4.5 4.5m0-4.5l-4.5 4.5M21
12a9 9 0 11-18 0 9 9 0 0118 0z"></path></svg> Incorrect`
416 |                 }}
417 |             </span>
418 |             <span class="text-xs text-slate-500">|</span>
419 |             <span class="text-xs text-slate-350 font-medium">Selected: ${{selectedText}}</span>
420 |             <span class="text-xs text-slate-500">|</span>
421 |             <span class="text-xs text-slate-350 font-medium">Solution: <strong
class="text-emerald-400 font-bold">${{q.correct_answer}}</strong></span>
422 |         </div>
423 |     `;
424 |     reviewList.appendChild(card);
425 | }});
426 | }}
427 | </script>
428 | </body>
429 | </html>
430 | ""
431 |
432 | @router.get("/courses/{course_id}/ppts")
433 | def export_course_ppts(
434 |     course_id: int,
435 |     db: Session = Depends(get_db),
436 |     current_user: User = Depends(get_current_user)
437 | ):
438 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
439 |     if not course:
440 |         raise HTTPException(status_code=404, detail="Course not found")
441 |
442 |     slides = db.query(GeneratedSlide).filter(GeneratedSlide.course_id ==
course_id).order_by(GeneratedSlide.slide_index).all()
443 |     notes = db.query(InstructorNote).filter(InstructorNote.course_id ==
course_id).order_by(InstructorNote.slide_index).all()
444 |
445 |     if not slides:
446 |         raise HTTPException(status_code=400, detail="No slides generated yet for this course. Please
analyze a syllabus first.")
447 |
448 |     slides_data = [
449 |         {
450 |             "slide_index": s.slide_index,
451 |             "title": s.title,
452 |             "content": s.content,
453 |             "suggested_visuals": s.suggested_visuals
454 |         } for s in slides
455 |     ]
456 |
457 |     notes_data = [
458 |         {

```

```

459 |         "slide_index": n.slide_index,
460 |         "talking_points": n.talking_points,
461 |         "teaching_tips": n.teaching_tips,
462 |         "examples": n.examples
463 |     } for n in notes
464 | ]
465 |
466 | export_dir = os.path.join(os.getcwd(), "exports")
467 | os.makedirs(export_dir, exist_ok=True)
468 | file_name = f"Kalyx_Course_{course_id}.pptx"
469 | file_path = os.path.join(export_dir, file_name)
470 |
471 | try:
472 |     generate_pptx_deck(slides_data, notes_data, file_path)
473 | except Exception as e:
474 |     raise HTTPException(status_code=500, detail=f"Failed to generate PPTX deck: {e}")
475 |
476 | ppt_log = PptExport(
477 |     course_id=course_id,
478 |     file_name=file_name,
479 |     file_path=file_path
480 | )
481 | db.add(ppt_log)
482 | db.commit()
483 |
484 | return FileResponse(
485 |     path=file_path,
486 |     filename=file_name,
487 |     media_type="application/vnd.openxmlformats-officedocument.presentationml.presentation"
488 | )
489 |
490 | @router.get("/courses/{course_id}/pdf")
491 | def export_course_pdf(
492 |     course_id: int,
493 |     db: Session = Depends(get_db),
494 |     current_user: User = Depends(get_current_user)
495 | ):
496 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
497 |     if not course:
498 |         raise HTTPException(status_code=404, detail="Course not found")
499 |
500 |     slides = db.query(GeneratedSlide).filter(GeneratedSlide.course_id ==
course_id).order_by(GeneratedSlide.slide_index).all()
501 |     notes = db.query(InstructorNote).filter(InstructorNote.course_id ==
course_id).order_by(InstructorNote.slide_index).all()
502 |     assessments = db.query(Assessment).filter(Assessment.course_id == course_id).all()
503 |
504 |     if not slides:
505 |         raise HTTPException(status_code=400, detail="No slides generated yet for this course. Please
analyze a syllabus first.")
506 |
507 |     slides_data = [
508 |         {
509 |             "slide_index": s.slide_index,
510 |             "title": s.title,
511 |             "content": s.content,
512 |             "suggested_visuals": s.suggested_visuals
513 |         } for s in slides
514 |     ]
515 |
516 |     notes_data = [
517 |         {
518 |             "slide_index": n.slide_index,
519 |             "talking_points": n.talking_points,
520 |             "teaching_tips": n.teaching_tips,
521 |             "examples": n.examples

```



```

522 |         } for n in notes
523 |     ]
524 |
525 |     assessments_data = [
526 |         {
527 |             "question_text": a.question_text,
528 |             "question_type": a.question_type,
529 |             "options": a.options,
530 |             "correct_answer": a.correct_answer,
531 |             "bloom_level": a.bloom_level
532 |         } for a in assessments
533 |     ]
534 |
535 |     export_dir = os.path.join(os.getcwd(), "exports")
536 |     os.makedirs(export_dir, exist_ok=True)
537 |     file_name = f"Kalyx_Course_{course_id}.pdf"
538 |     file_path = os.path.join(export_dir, file_name)
539 |
540 |     try:
541 |         generate_pdf_package(course.title, course.description, slides_data, notes_data, assessments_data,
file_path)
542 |     except Exception as e:
543 |         raise HTTPException(status_code=500, detail=f"Failed to generate PDF package: {e}")
544 |
545 |     return FileResponse(
546 |         path=file_path,
547 |         filename=file_name,
548 |         media_type="application/pdf"
549 |     )
550 |
551 | @router.get("/courses/{course_id}/kahoot")
552 | def export_course_kahoot(
553 |     course_id: int,
554 |     db: Session = Depends(get_db),
555 |     current_user: User = Depends(get_current_user)
556 | ):
557 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
558 |     if not course:
559 |         raise HTTPException(status_code=404, detail="Course not found")
560 |
561 |     assessments = db.query(Assessment).filter(Assessment.course_id == course_id).all()
562 |     # Filter MCQ questions (those marked as MCQ or having non-empty options)
563 |     mcqs = [a for a in assessments if a.question_type.upper() == "MCQ" or (a.options and len(a.options) >
0)]
564 |
565 |     if not mcqs:
566 |         raise HTTPException(status_code=400, detail="No MCQ assessments found for this course.")
567 |
568 |     output = io.StringIO()
569 |     writer = csv.writer(output)
570 |
571 |     # Kahoot bulk import spreadsheet format headers:
572 |     # Question,Answer Option 1,Answer Option 2,Answer Option 3,Answer Option 4,Time Limit
(seconds),Correct Answer(s)
573 |     writer.writerow([
574 |         "Question",
575 |         "Answer Option 1",
576 |         "Answer Option 2",
577 |         "Answer Option 3",
578 |         "Answer Option 4",
579 |         "Time Limit (seconds)",
580 |         "Correct Answer(s)"
581 |     ])
582 |
583 |     for a in mcqs:
584 |         question_text = a.question_text

```

```

585 |         options = a.options or []
586 |         correct_answer = a.correct_answer or ""
587 |
588 |         # Max lengths: Question 120 chars, Option 75 chars (warning limit, but we preserve as much text
as possible)
589 |         # Ensure we have up to 4 options
590 |         opt1 = options[0] if len(options) > 0 else ""
591 |         opt2 = options[1] if len(options) > 1 else ""
592 |         opt3 = options[2] if len(options) > 2 else ""
593 |         opt4 = options[3] if len(options) > 3 else ""
594 |
595 |         # Find 1-based index/indices of correct answer
596 |         correct_indices = []
597 |         if correct_answer:
598 |             cleaned_options = [str(opt).strip().lower() for opt in options]
599 |             clean_correct = str(correct_answer).strip().lower()
600 |
601 |             # Exact Match
602 |             matched = False
603 |             for idx, cleaned_opt in enumerate(cleaned_options):
604 |                 if cleaned_opt == clean_correct:
605 |                     correct_indices.append(str(idx + 1))
606 |                     matched = True
607 |                     break
608 |
609 |             # Substring Match
610 |             if not matched:
611 |                 for idx, cleaned_opt in enumerate(cleaned_options):
612 |                     if cleaned_opt in clean_correct or clean_correct in cleaned_opt:
613 |                         correct_indices.append(str(idx + 1))
614 |                         matched = True
615 |                         break
616 |
617 |             # Digit Match
618 |             if not matched:
619 |                 digits = [c for c in clean_correct if c.isdigit()]
620 |                 for d in digits:
621 |                     val = int(d)
622 |                     if 1 <= val <= len(options):
623 |                         correct_indices.append(str(val))
624 |                         matched = True
625 |
626 |             # Fallback to index 1 if not matched
627 |             if not correct_indices:
628 |                 correct_indices = ["1"]
629 |
630 |             correct_str = ",".join(correct_indices)
631 |             time_limit = 30
632 |
633 |             writer.writerow([
634 |                 question_text,
635 |                 opt1,
636 |                 opt2,
637 |                 opt3,
638 |                 opt4,
639 |                 time_limit,
640 |                 correct_str
641 |             ])
642 |
643 |         csv_data = output.getvalue()
644 |         output.close()
645 |
646 |         headers = {
647 |             "Content-Disposition": f"attachment; filename=Kahoot_Import_Course_{course_id}.csv"
648 |         }
649 |         return Response(content=csv_data, media_type="text/csv", headers=headers)

```

```
650 |
651 | @router.get("/courses/{course_id}/interactive-quiz")
652 | def export_course_interactive_quiz(
653 |     course_id: int,
654 |     db: Session = Depends(get_db),
655 |     current_user: User = Depends(get_current_user)
656 | ):
657 |     course = db.query(Course).filter(Course.id == course_id, Course.user_id == current_user.id).first()
658 |     if not course:
659 |         raise HTTPException(status_code=404, detail="Course not found")
660 |
661 |     assessments = db.query(Assessment).filter(Assessment.course_id == course_id).all()
662 |     mcqs = [a for a in assessments if a.question_type.upper() == "MCQ" or (a.options and len(a.options) >
0)]
663 |
664 |     if not mcqs:
665 |         raise HTTPException(status_code=400, detail="No MCQ assessments found for this course.")
666 |
667 |     questions_data = []
668 |     for a in mcqs:
669 |         questions_data.append({
670 |             "id": a.id,
671 |             "question_text": a.question_text,
672 |             "bloom_level": a.bloom_level,
673 |             "options": a.options or [],
674 |             "correct_answer": a.correct_answer or ""
675 |         })
676 |
677 |     questions_json = json.dumps(questions_data)
678 |     course_title_esc = course.title.replace("'", '\\\'')
679 |     course_desc_esc = course.description.replace("'", '\\\'').replace('\n', ' ') if course.description
else ""
680 |
681 |     html_content = get_interactive_quiz_template(course_title_esc, course_desc_esc, questions_json)
682 |
683 |     headers = {
684 |         "Content-Disposition": f"attachment; filename=Interactive_Quiz_Course_{course_id}.html"
685 |     }
686 |     return Response(content=html_content, media_type="text/html", headers=headers)
687 |
688 |
```

File: backend/app/main.py

```
1 | import os
2 | import uvicorn
3 | from fastapi import FastAPI, Request
4 | from fastapi.responses import JSONResponse
5 | from fastapi.exceptions import HTTPException
6 | from fastapi.middleware.cors import CORSMiddleware
7 | import logging
8 | from app.database import engine, Base
9 | from app.routers import auth, courses, studio, export
10 |
11 | # Create database tables automatically if they don't exist
12 | try:
13 |     print("[Startup] Initializing relational database schemas...")
14 |     Base.metadata.create_all(bind=engine)
15 |
16 |     # Safety DB migration: add username column if it doesn't exist on users table
17 |     from app.database import SessionLocal
18 |     from sqlalchemy import text
19 |     db = SessionLocal()
20 |     try:
21 |         db.execute(text("ALTER TABLE users ADD COLUMN username VARCHAR(255)"))
22 |         db.commit()
23 |         print("[Startup] SQLite migration: Added username column to users table.")
24 |     except Exception as err:
25 |         # Ignore if column already exists
26 |         print(f"[Startup] SQLite migration check (users.username): {err}")
27 |         pass
28 |
29 |     try:
30 |         db.execute(text("ALTER TABLE curriculum_analysis ADD COLUMN pipeline_telemetry JSON"))
31 |         db.commit()
32 |         print("[Startup] SQLite migration: Added pipeline_telemetry column to curriculum_analysis
table.")
33 |     except Exception as err:
34 |         # Ignore if column already exists
35 |         print(f"[Startup] SQLite migration check (curriculum_analysis.pipeline_telemetry): {err}")
36 |         pass
37 |
38 |     # Ensure seeded users have their correct usernames
39 |     try:
40 |         from app.models import User
41 |         prof = db.query(User).filter(User.email == "professor.jones@stanford.edu").first()
42 |         if prof and not prof.username:
43 |             prof.username = "professor.jones"
44 |             db.commit()
45 |             print("[Startup] Fixed NULL username for professor.jones@stanford.edu")
46 |         dev = db.query(User).filter(User.email == "devashish@stanford.edu").first()
47 |         if dev and not dev.username:
48 |             dev.username = "devashish"
49 |             db.commit()
50 |             print("[Startup] Fixed NULL username for devashish@stanford.edu")
51 |     except Exception as err:
52 |         print(f"[Startup] Error updating seeded usernames: {err}")
53 |     finally:
54 |         db.close()
55 |
56 |     print("[Startup] Database schemas synchronized successfully.")
57 |
58 |     # Seed default developer course for instant hackathon evaluations
59 |     from app.models import Course, User
60 |     from app.routers.auth import hash_password
61 |
```

```

62 |     db = SessionLocal()
63 |     if db.query(User).count() == 0:
64 |         print("[Startup] Seeding default demo educator profile...")
65 |         default_user = User(
66 |             email="professor.jones@stanford.edu",
67 |             username="professor.jones",
68 |             hashed_password=hash_password("hackathon_demo_pass"),
69 |             full_name="Dr. Devashish Jones"
70 |         )
71 |         db.add(default_user)
72 |         db.commit()
73 |         db.refresh(default_user)
74 |
75 |         print("[Startup] Seeding default Advanced Machine Learning course node...")
76 |         default_course = Course(
77 |             user_id=default_user.id,
78 |             title="Advanced Machine Learning",
79 |             description="An advanced machine learning class focusing on model architectures, regression
models, classification boundaries, and support vector machines."
80 |         )
81 |         db.add(default_course)
82 |         db.commit()
83 |         db.close()
84 | except Exception as e:
85 |     print(f"[Startup] Database sync warning: {e}. Ensure server credentials are valid.")
86 |
87 | app = FastAPI(
88 |     title="KALYX - Agentic Curriculum Intelligence Platform API",
89 |     description="Multi-agent educational sequencing, slide generation, Bloom's taxonomy auditing, and
PPTX exporting.",
90 |     version="1.0.0"
91 | )
92 |
93 | # Configure CORS Middleware
94 | # Next.js defaults to localhost:3000. We allow full local cross-origin capabilities.
95 | app.add_middleware(
96 |     CORSMiddleware,
97 |     allow_origins=["*"], # Open for development convenience
98 |     allow_credentials=True,
99 |     allow_methods=["*"],
100 |    allow_headers=["*"],
101 | )
102 |
103 | # Standard Security Headers Middleware
104 | @app.middleware("http")
105 | async def add_security_headers(request: Request, call_next):
106 |     response = await call_next(request)
107 |     response.headers["X-Content-Type-Options"] = "nosniff"
108 |     response.headers["X-Frame-Options"] = "DENY"
109 |     response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
110 |     return response
111 |
112 | # Error Logging Logger
113 | logger = logging.getLogger("kalyx_security")
114 |
115 | # Unhandled Exceptions Handler (Prevent raw leaks)
116 | @app.exception_handler(Exception)
117 | async def unhandled_exception_handler(request: Request, exc: Exception):
118 |     logger.exception(f"Unhandled exception occurred on path {request.url.path}: {exc}")
119 |     return JSONResponse(
120 |         status_code=500,
121 |         content={"error": "Internal server error", "detail": "Internal server error"}
122 |     )
123 |
124 | # HTTP Exceptions Handler (Clean JSON errors compatible with frontend details)
125 | @app.exception_handler(HTTPException)

```

```
126 | async def http_exception_handler(request: Request, exc: HTTPException):
127 |     detail = exc.detail
128 |     if isinstance(detail, dict):
129 |         if "error" not in detail and "detail" in detail:
130 |             detail["error"] = detail["detail"]
131 |         elif "detail" not in detail and "error" in detail:
132 |             detail["detail"] = detail["error"]
133 |         return JSONResponse(
134 |             status_code=exc.status_code,
135 |             headers=exc.headers,
136 |             content=detail
137 |         )
138 |     return JSONResponse(
139 |         status_code=exc.status_code,
140 |         headers=exc.headers,
141 |         content={"error": detail, "detail": detail}
142 |     )
143 |
144 | # Register routers
145 | app.include_router(auth.router, prefix="/api")
146 | app.include_router(courses.router, prefix="/api")
147 | app.include_router(studio.router, prefix="/api")
148 | app.include_router(export.router, prefix="/api")
149 |
150 | # System Health and Security Status Audit Endpoint
151 | @app.get("/api/system/health")
152 | def get_system_health():
153 |     return {
154 |         "status": "healthy",
155 |         "auth": True,
156 |         "rate_limiting": True,
157 |         "validation": True
158 |     }
159 |
160 | @app.get("/")
161 | def read_root():
162 |     return {
163 |         "platform": "KALYX",
164 |         "description": "Agentic Curriculum Intelligence Platform Backend",
165 |         "status": "Online"
166 |     }
167 |
168 | if __name__ == "__main__":
169 |     port = int(os.getenv("PORT", "8000"))
170 |     host = os.getenv("HOST", "0.0.0.0")
171 |     uvicorn.run("app.main:app", host=host, port=port, reload=True)
172 |
```

File: backend/test_auth_audit.py

```
1 | import requests
2 | import sys
3 | import uuid
4 | import os
5 |
6 | # Adjust path to import from app
7 | sys.path.append(os.path.abspath(os.path.dirname(__file__)))
8 |
9 | from app.routers.auth import hash_password, verify_password
10 |
11 | BASE_URL = "http://localhost:8000/api"
12 |
13 | def run_audit():
14 |     print("=== STARTING AUTHENTICATION AUDIT ===")
15 |
16 |     # 0. Direct Password Hashing/Verification Type Audit (Failure Reproduction)
17 |     print("\n0. Direct Password Hashing & Verification Type Safety Test:")
18 |     plain_pw = "SuperSecurePass123!"
19 |
20 |     # Hash password to string
21 |     hashed_str = hash_password(plain_pw)
22 |     print(f"    Generated hash (str): {hashed_str} (Type: {type(hashed_str)})")
23 |
24 |     # Verify with string hash (should succeed)
25 |     verify_str_result = verify_password(plain_pw, hashed_str)
26 |     print(f"    Verification with str hash: {verify_str_result}")
27 |
28 |     # Simulate DB returning bytes (common with PostgreSQL on Render)
29 |     hashed_bytes = hashed_str.encode('utf-8')
30 |     print(f"    Simulating DB returned bytes hash: {hashed_bytes} (Type: {type(hashed_bytes)})")
31 |
32 |     # Verify with bytes hash (will fail/return False before the fix due to .encode('utf-8') on bytes)
33 |     verify_bytes_result = verify_password(plain_pw, hashed_bytes)
34 |     print(f"    Verification with bytes hash: {verify_bytes_result}")
35 |
36 |     if not verify_bytes_result:
37 |         print("    [REPRODUCED FAILURE] verify_password failed when hashed password is bytes!")
38 |     else:
39 |         print("    [SUCCESS] verify_password succeeded when hashed password is bytes!")
40 |
41 |     # 1. Signup Flow
42 |     unique_suffix = str(uuid.uuid4())[:8]
43 |     email = f"audit_{unique_suffix}@stanford.edu"
44 |     username = f"audituser_{unique_suffix}"
45 |     password = "MySecurePassword123!"
46 |     full_name = "Audit User"
47 |
48 |     signup_payload = {
49 |         "email": email,
50 |         "username": username,
51 |         "password": password,
52 |         "full_name": full_name
53 |     }
54 |     print(f"\n1. Attempting Signup with email={email}, username={username}...")
55 |     try:
56 |         r = requests.post(f"{BASE_URL}/auth/signup", json=signup_payload)
57 |         print(f"    Signup response status: {r.status_code}")
58 |         if r.status_code != 200:
59 |             print(f"    Signup failed: {r.text}")
60 |             sys.exit(1)
61 |         data = r.json()
62 |         print("    Signup successful! Received Token.")
```

```
63 |     assert "access_token" in data
64 |     assert data["user"]["email"] == email
65 |     assert data["user"]["username"] == username
66 | except Exception as e:
67 |     print(f"    Signup exception: {e}")
68 |     sys.exit(1)
69 |
70 | # 2. Login Flow - by Email
71 | login_payload_email = {
72 |     "username_or_email": email,
73 |     "password": password
74 | }
75 | print(f"\n2. Attempting Login by EMAIL={email}...")
76 | try:
77 |     r = requests.post(f"{BASE_URL}/auth/login", json=login_payload_email)
78 |     print(f"    Login (Email) response status: {r.status_code}")
79 |     if r.status_code != 200:
80 |         print(f"    Login failed: {r.text}")
81 |     else:
82 |         data = r.json()
83 |         print("    Login by Email successful!")
84 |         assert "access_token" in data
85 | except Exception as e:
86 |     print(f"    Login by Email exception: {e}")
87 |
88 | # 3. Login Flow - by Username
89 | login_payload_username = {
90 |     "username_or_email": username,
91 |     "password": password
92 | }
93 | print(f"\n3. Attempting Login by USERNAME={username}...")
94 | try:
95 |     r = requests.post(f"{BASE_URL}/auth/login", json=login_payload_username)
96 |     print(f"    Login (Username) response status: {r.status_code}")
97 |     if r.status_code != 200:
98 |         print(f"    Login failed: {r.text}")
99 |     else:
100 |         data = r.json()
101 |         print("    Login by Username successful!")
102 |         assert "access_token" in data
103 | except Exception as e:
104 |     print(f"    Login by Username exception: {e}")
105 |
106 | # 4. Login Flow - Case Insensitive / Whitespace
107 | login_payload_case = {
108 |     "username_or_email": f" {username.upper()} ",
109 |     "password": password
110 | }
111 | print(f"\n4. Attempting Login by UPPERCASE & WHITESPACE USERNAME=' {username.upper()} '...")
112 | try:
113 |     r = requests.post(f"{BASE_URL}/auth/login", json=login_payload_case)
114 |     print(f"    Login (Case/Whitespace) response status: {r.status_code}")
115 |     if r.status_code != 200:
116 |         print(f"    Login failed: {r.text}")
117 |     else:
118 |         data = r.json()
119 |         print("    Login by Case/Whitespace successful!")
120 |         assert "access_token" in data
121 | except Exception as e:
122 |     print(f"    Login by Case/Whitespace exception: {e}")
123 |
124 | if __name__ == "__main__":
125 |     run_audit()
126 |
```


File: frontend/src/app/page.tsx

```
1 | "use client";
2 |
3 | import React, { useState, useEffect } from "react";
4 | import { API_URL } from "@lib/config";
5 | import {
6 |   Sparkles, UploadCloud, BookOpen, Layers, Award, BarChart3,
7 |   Download, Play, Plus, Trash2, ArrowRight, CheckCircle2,
8 |   ChevronRight, Save, AlertCircle, Pencil,
9 |   Eye, FileText, ChevronLeft, Volume2, Lightbulb, GraduationCap,
10 |   FileSpreadsheet, Gamepad2, Cpu
11 | } from "lucide-react";
12 |
13 | // Types matching backend models
14 | interface Course {
15 |   id: number;
16 |   title: string;
17 |   description: string;
18 |   created_at: string;
19 | }
20 |
21 | interface Slide {
22 |   id: number;
23 |   slide_index: number;
24 |   title: string;
25 |   content: string[];
26 |   suggested_visuals: string;
27 | }
28 |
29 | interface Note {
30 |   id: number;
31 |   slide_index: number;
32 |   talking_points: string[];
33 |   teaching_tips: string;
34 |   examples: string[];
35 | }
36 |
37 | interface Assessment {
38 |   id: number;
39 |   question_text: string;
40 |   question_type: string;
41 |   options: string[] | null;
42 |   correct_answer: string | null;
43 |   bloom_level: string;
44 |   learning_outcome_id: number | null;
45 | }
46 |
47 | interface Outcome {
48 |   id: number;
49 |   text: string;
50 |   bloom_level: string;
51 | }
52 |
53 | interface Readiness {
54 |   score: number;
55 |   completeness: number;
56 |   outcome_coverage: number;
57 |   assessment_quality: number;
58 |   bloom_coverage: number;
59 |   industry_relevance: number;
60 |   breakdown: Record<string, string>;
61 | }
62 |
```

```

63 | interface GapReport {
64 |   status: string;
65 |   missing_topics: string[];
66 |   recommendations: string[];
67 | }
68 |
69 | interface Module {
70 |   id?: string | number;
71 |   title: string;
72 |   topics?: string[];
73 | }
74 |
75 | interface CurriculumMap {
76 |   modules?: Module[];
77 | }
78 |
79 | interface CourseDetails {
80 |   course: Course;
81 |   curriculum_analysis: {
82 |     curriculum_map: CurriculumMap | null;
83 |     gap_analysis: string | null;
84 |     industry_gap_report: GapReport;
85 |   } | null;
86 |   learning_outcomes: Outcome[];
87 |   slides: Slide[];
88 |   notes: Note[];
89 |   assessments: Assessment[];
90 |   readiness_score: Readiness | null;
91 | }
92 |
93 | // Helper function to parse database critique breakdowns into structured reason / recommendation
94 | function parseCritique(
95 |   breakdown: Record<string, string> | null | undefined,
96 |   metricKey: string,
97 |   val: number
98 | ) {
99 |   let comment = "";
100 |   if (breakdown) {
101 |     const keys = Object.keys(breakdown);
102 |     const matchedKey = keys.find(k => k.toLowerCase().includes(metricKey.toLowerCase())) ||
metricKey.toLowerCase().includes(k.toLowerCase());
103 |     if (matchedKey) {
104 |       comment = breakdown[matchedKey];
105 |     } else if (breakdown["Overall Score"] && metricKey === "Overall") {
106 |       comment = breakdown["Overall Score"];
107 |     }
108 |   }
109 |
110 |   // Fallbacks if comment is missing or empty
111 |   if (!comment) {
112 |     if (metricKey.toLowerCase().includes("completeness")) {
113 |       if (val >= 85) {
114 |         comment = "The syllabus contains a highly complete outline of topics and modules. Recommended:
Keep slides updated with new case studies.";
115 |       } else {
116 |         comment = "Some structural sections or detailed slide descriptions are missing or thin.
Recommended: Expand sub-topic bullet points to ensure complete coverage.";
117 |       }
118 |     } else if (metricKey.toLowerCase().includes("bloom")) {
119 |       if (val >= 85) {
120 |         comment = "Strong cognitive level coverage across lower and higher order objectives. Recommended:
Maintain project-based assessments.";
121 |       } else {
122 |         comment = "The course focuses heavily on Remembering and Understanding levels but lacks
activities targeting Evaluating and Creating. Recommended: Add project-based assessments and design
challenges.";

```

```

123 |     }
124 |   } else if (metricKey.toLowerCase().includes("assessment")) {
125 |     if (val >= 85) {
126 |       comment = "Assessments are highly aligned with the learning outcomes and cover appropriate
difficulty levels. Recommended: Introduce peer evaluations.";
127 |     } else {
128 |       comment = "Assessments are limited in count or not fully aligned to advanced learning outcomes.
Recommended: Add more diagnostic assessments and diverse question formats matching the outcomes.";
129 |     }
130 |   } else if (metricKey.toLowerCase().includes("relevance") ||
metricKey.toLowerCase().includes("industry")) {
131 |     if (val >= 85) {
132 |       comment = "Course curriculum is aligned with cutting-edge industry practices and technologies.
Recommended: Integrate live tool tutorials.";
133 |     } else {
134 |       comment = "The syllabus lacks recent real-world advancements or industry-relevant technical
tooling. Recommended: Integrate modern tools, industry APIs, and practical case studies.";
135 |     }
136 |   } else {
137 |     comment = `Readiness metric score is ${val.toFixed(0)}/100. Recommended: Review syllabus alignment
and completeness.`;
138 |   }
139 | }
140 |
141 | let reason = "";
142 | let recommendation = "";
143 |
144 | const indicators = ["Recommended:", "To improve:", "Recommendation:", "Improvement:", "Suggested:",
"Suggest:"];
145 | let foundIndicator = "";
146 | let splitIndex = -1;
147 |
148 | for (const ind of indicators) {
149 |   const idx = comment.indexOf(ind);
150 |   if (idx !== -1) {
151 |     if (splitIndex === -1 || idx < splitIndex) {
152 |       splitIndex = idx;
153 |       foundIndicator = ind;
154 |     }
155 |   }
156 | }
157 |
158 | if (splitIndex !== -1) {
159 |   reason = comment.substring(0, splitIndex).trim();
160 |   recommendation = comment.substring(splitIndex + foundIndicator.length).trim();
161 | } else {
162 |   const sentences = comment.split(/(?<=[!?])\s+/);
163 |   if (sentences.length > 1) {
164 |     recommendation = sentences.pop() || "";
165 |     reason = sentences.join(" ");
166 |   } else {
167 |     reason = comment;
168 |     if (val >= 85) {
169 |       recommendation = "Continue to maintain high quality standards and periodically refresh course
examples.";
170 |     } else {
171 |       recommendation = "Review syllabus detail and align exercises with higher-level Bloom taxonomy
verbs.";
172 |     }
173 |   }
174 | }
175 |
176 | reason = reason.trim();
177 | recommendation = recommendation.trim();
178 |
179 | if (reason.endsWith(",") || reason.endsWith(":") || reason.endsWith(".")) {

```

```

180 |     reason = reason.slice(0, -1).trim();
181 |   }
182 |
183 |   return { reason, recommendation };
184 | }
185 |
186 | interface AgentStatus {
187 |   name: string;
188 |   purpose: string;
189 |   status: "pending" | "running" | "completed" | "failed";
190 |   timestamp?: string;
191 |   duration?: number;
192 | }
193 |
194 | const initialPipeline: AgentStatus[] = [
195 |   { name: "Curriculum Analysis Agent", purpose: "Extracts modules, topics, and structures from raw
syllabus text.", status: "pending" },
196 |   { name: "Learning Outcome Agent", purpose: "Generates measurable learning outcomes mapped to Bloom's
Taxonomy.", status: "pending" },
197 |   { name: "Curriculum Planning Agent", purpose: "Coordinates weekly sequencing, lesson scheduling, and
academic pacing.", status: "pending" },
198 |   { name: "Slide Generation Agent", purpose: "Generates slide titles, layout outlines, and content bullet
points.", status: "pending" },
199 |   { name: "Instructor Notes Agent", purpose: "Develops comprehensive lecturer talking points and
real-world examples.", status: "pending" },
200 |   { name: "Assessment Agent", purpose: "Creates diagnostic assessment questions mapped to learning
outcomes.", status: "pending" },
201 |   { name: "Bloom Audit Agent", purpose: "Analyzes cognitive balance and produces a Bloom level
distribution report.", status: "pending" },
202 |   { name: "Industry Gap Agent", purpose: "Benchmarks curriculum mapping against modern technology
requirements.", status: "pending" },
203 |   { name: "Readiness Score Agent", purpose: "Computes overall 100-point accreditation readiness
scoring.", status: "pending" }
204 | ];
205 |
206 | export default function KalyxApp() {
207 |   // App views: 'landing' | 'workspace'
208 |   const [view, setView] = useState<"landing" | "workspace">("landing");
209 |   const [activeTab, setActiveTab] = useState<"overview" | "slides" | "notes" | "assessments" | "bloom" |
"readiness" | "gaps" | "export" | "traceability" | "pipeline">("overview");
210 |
211 |   // New state variables for Traceability and Readiness breakdowns
212 |   const [traceabilityData, setTraceabilityData] = useState<any[]>([]);
213 |   const [expandedOutcomes, setExpandedOutcomes] = useState<Record<number, boolean>>({});
214 |   const [isReadinessExpanded, setIsReadinessExpanded] = useState(false);
215 |   const [pipelineAgents, setPipelineAgents] = useState<AgentStatus[]>(initialPipeline);
216 |
217 |   const handleAuthError = () => {
218 |     localStorage.removeItem("kalyx_auth_token");
219 |     localStorage.removeItem("kalyx_auth_user");
220 |     setAuthToken(null);
221 |     setUser(null);
222 |     alert("Session expired or invalid. Please log in again.");
223 |   };
224 |
225 |   const allCompleted = pipelineAgents.every(a => a.status === "completed");
226 |   const totalDuration = allCompleted
227 |     ? pipelineAgents.reduce((sum, a) => sum + (a.duration || 0), 0)
228 |     : 0;
229 |
230 |   // Data State
231 |   const [courses, setCourses] = useState<Course[]>([]);
232 |   const [activeCourseId, setActiveCourseId] = useState<number | null>(null);
233 |   const [courseDetails, setCourseDetails] = useState<CourseDetails | null>(null);
234 |
235 |   // Create / Upload State

```

```

236 | const [showCreateModal, setShowCreateModal] = useState(false);
237 | const [newCourseTitle, setNewCourseTitle] = useState("");
238 | const [newCourseDesc, setNewCourseDesc] = useState("");
239 | const [uploadFile, setUploadFile] = useState<File | null>(null);
240 | const [isUploading, setIsUploading] = useState(false);
241 | const [uploadLogs, setUploadLogs] = useState<string[]>([]);
242 |
243 | const startPipelineSimulation = () => {
244 |   const initial: AgentStatus[] = [
245 |     { name: "Curriculum Analysis Agent", purpose: "Extracts modules, topics, and structures from raw
syllabus text.", status: "running" },
246 |     { name: "Learning Outcome Agent", purpose: "Generates measurable learning outcomes mapped to
Bloom's Taxonomy.", status: "pending" },
247 |     { name: "Curriculum Planning Agent", purpose: "Coordinates weekly sequencing, lesson scheduling,
and academic pacing.", status: "pending" },
248 |     { name: "Slide Generation Agent", purpose: "Generates slide titles, layout outlines, and content
bullet points.", status: "pending" },
249 |     { name: "Instructor Notes Agent", purpose: "Develops comprehensive lecturer talking points and
real-world examples.", status: "pending" },
250 |     { name: "Assessment Agent", purpose: "Creates diagnostic assessment questions mapped to learning
outcomes.", status: "pending" },
251 |     { name: "Bloom Audit Agent", purpose: "Analyzes cognitive balance and produces a Bloom level
distribution report.", status: "pending" },
252 |     { name: "Industry Gap Agent", purpose: "Benchmarks curriculum mapping against modern technology
requirements.", status: "pending" },
253 |     { name: "Readiness Score Agent", purpose: "Computes overall 100-point accreditation readiness
scoring.", status: "pending" }
254 |   ];
255 |   setPipelineAgents(initial);
256 |
257 |   let activeIdx = 0;
258 |   const interval = setInterval(() => {
259 |     setPipelineAgents(prev => {
260 |       const next = [...prev];
261 |       if (activeIdx < next.length) {
262 |         next[activeIdx] = {
263 |           ...next[activeIdx],
264 |           status: "completed",
265 |           timestamp: `Step ${activeIdx + 1} of 9`
266 |         };
267 |       }
268 |       activeIdx++;
269 |       if (activeIdx < next.length) {
270 |         next[activeIdx] = {
271 |           ...next[activeIdx],
272 |           status: "running"
273 |         };
274 |       } else {
275 |         clearInterval(interval);
276 |       }
277 |       return next;
278 |     });
279 |   }, 1500);
280 |
281 |   return interval;
282 | };
283 |
284 | const completePipelineSimulation = (telemetry?: any[]) => {
285 |   setPipelineAgents(prev => {
286 |     return prev.map((agent, idx) => {
287 |       const tItem = telemetry?.find(t => t.agent === agent.name);
288 |       if (tItem) {
289 |         const durationSec = typeof tItem.duration_ms === 'number' ? tItem.duration_ms / 1000 :
tItem.duration_seconds;
290 |         const formattedDuration = typeof durationSec === 'number' ? `${durationSec.toFixed(1)}s` :
'0.0s';

```

```

291 |         return {
292 |             ...agent,
293 |             status: "completed",
294 |             timestamp: `Completed - ${formattedDuration}`,
295 |             duration: durationSec
296 |         };
297 |     }
298 |     return {
299 |         ...agent,
300 |         status: "completed",
301 |         timestamp: `Step ${idx + 1} of 9`
302 |     };
303 | });
304 | });
305 | };
306 |
307 | const failPipelineSimulation = () => {
308 |     setPipelineAgents(prev => {
309 |         return prev.map(agent => {
310 |             if (agent.status === "running") {
311 |                 return { ...agent, status: "failed" };
312 |             }
313 |             return agent;
314 |         });
315 |     });
316 | };
317 |
318 | // AI Personalization settings
319 | const [personalization, setPersonalization] = useState({
320 |     tone: "Professional & Academic",
321 |     style: "Sleek Dark Mode",
322 |     examplesCount: "3 per slide",
323 |     customInstructions: ""
324 | });
325 |
326 | // Edit states for AI Studio
327 | const [activeSlideIndex, setActiveSlideIndex] = useState(0);
328 | const [editSlideTitle, setEditSlideTitle] = useState("");
329 | const [editSlideBullets, setEditSlideBullets] = useState<string[]>([]);
330 | const [editSlideVisuals, setEditSlideVisuals] = useState("");
331 | const [editNoteTalkingPoints, setEditNoteTalkingPoints] = useState<string[]>([]);
332 | const [editNoteTips, setEditNoteTips] = useState("");
333 | const [editNoteExamples, setEditNoteExamples] = useState<string[]>([]);
334 |
335 | // User Authentication States
336 | const [authToken, setAuthToken] = useState<string | null>(null);
337 | const [user, setUser] = useState<{ id: number; email: string; username: string; full_name: string |
null } | null>(null);
338 | const [authMode, setAuthMode] = useState<"login" | "signup">("login");
339 | const [authUsernameOrEmail, setAuthUsernameOrEmail] = useState("");
340 | const [authEmail, setAuthEmail] = useState("");
341 | const [authUsername, setAuthUsername] = useState("");
342 | const [authPassword, setAuthPassword] = useState("");
343 | const [authFullName, setAuthFullName] = useState("");
344 | const [authError, setAuthError] = useState("");
345 |
346 | const backendUrl = API_URL;
347 |
348 | const handleAuthSubmit = async (e: React.FormEvent) => {
349 |     e.preventDefault();
350 |     setAuthError("");
351 |
352 |     if (authMode === "login") {
353 |         if (!authUsernameOrEmail.trim() || !authPassword.trim()) {
354 |             setAuthError("Please enter your username/email and password.");
355 |             return;

```

```
356 |     }
357 |     try {
358 |       const res = await fetch(`${backendUrl}/api/auth/login`, {
359 |         method: "POST",
360 |         headers: { "Content-Type": "application/json" },
361 |         body: JSON.stringify({
362 |           username_or_email: authUsernameOrEmail,
363 |           password: authPassword
364 |         })
365 |       });
366 |       const data = await res.json();
367 |       if (!res.ok) {
368 |         setAuthError(data.detail || "Invalid credentials.");
369 |         return;
370 |       }
371 |       setAuthToken(data.access_token);
372 |       setUser(data.user);
373 |       localStorage.setItem("kalyx_token", data.access_token);
374 |       localStorage.setItem("kalyx_user", JSON.stringify(data.user));
375 |       fetchCourses(data.access_token);
376 |     } catch (err) {
377 |       setAuthError("Database connection refused or backend offline.");
378 |     }
379 |   } else {
380 |     if (!authEmail.trim() || !authUsername.trim() || !authPassword.trim() || !authFullName.trim()) {
381 |       setAuthError("Please fill out all signup fields.");
382 |       return;
383 |     }
384 |     try {
385 |       const res = await fetch(`${backendUrl}/api/auth/signup`, {
386 |         method: "POST",
387 |         headers: { "Content-Type": "application/json" },
388 |         body: JSON.stringify({
389 |           email: authEmail,
390 |           username: authUsername,
391 |           password: authPassword,
392 |           full_name: authFullName
393 |         })
394 |       });
395 |       const data = await res.json();
396 |       if (!res.ok) {
397 |         setAuthError(data.detail || "Registration failed.");
398 |         return;
399 |       }
400 |       setAuthToken(data.access_token);
401 |       setUser(data.user);
402 |       localStorage.setItem("kalyx_token", data.access_token);
403 |       localStorage.setItem("kalyx_user", JSON.stringify(data.user));
404 |       fetchCourses(data.access_token);
405 |     } catch (err) {
406 |       setAuthError("Database connection refused or backend offline.");
407 |     }
408 |   }
409 | };
410 |
411 | const handleLogout = () => {
412 |   setAuthToken(null);
413 |   setUser(null);
414 |   localStorage.removeItem("kalyx_token");
415 |   localStorage.removeItem("kalyx_user");
416 |   setCourses([]);
417 |   setCourseDetails(null);
418 |   setActiveCourseId(null);
419 |   setView("landing");
420 | };
421 |
```

```

422 | function loadSlideEditState(details: CourseDetails, index: number) {
423 |     setActiveSlideIndex(index);
424 |     const slide = details.slides[index];
425 |     if (slide) {
426 |         setEditSlideTitle(slide.title);
427 |         setEditSlideBullets([...slide.content]);
428 |         setEditSlideVisuals(slide.suggested_visuals || "");
429 |     }
430 |
431 |     const note = details.notes.find(n => n.slide_index === slide?.slide_index);
432 |     if (note) {
433 |         setEditNoteTalkingPoints([...note.talking_points]);
434 |         setEditNoteTips(note.teaching_tips || "");
435 |         setEditNoteExamples([...(note.examples || [])]);
436 |     } else {
437 |         setEditNoteTalkingPoints([]);
438 |         setEditNoteTips("");
439 |         setEditNoteExamples([]);
440 |     }
441 | }
442 |
443 | async function fetchTraceability(courseId: number, tokenOverride?: string) {
444 |     const token = tokenOverride || authToken;
445 |     if (!token) return;
446 |     try {
447 |         const res = await fetch(`${backendUrl}/api/courses/${courseId}/traceability`, {
448 |             headers: { "Authorization": `Bearer ${token}` }
449 |         });
450 |         if (res.ok) {
451 |             const data = await res.json();
452 |             setTraceabilityData(data);
453 |         }
454 |     } catch (err) {
455 |         console.error("Fetch traceability err:", err);
456 |     }
457 | }
458 |
459 | async function selectCourse(courseId: number, tokenOverride?: string, preserveSlideIndex?: boolean) {
460 |     const token = tokenOverride || authToken;
461 |     if (!token) return;
462 |
463 |     // Refresh outcomes mapping
464 |     fetchTraceability(courseId, token);
465 |
466 |     try {
467 |         const res = await fetch(`${backendUrl}/api/courses/${courseId}`, {
468 |             headers: { "Authorization": `Bearer ${token}` }
469 |         });
470 |         if (res.ok) {
471 |             const data = await res.json();
472 |             setCourseDetails(data);
473 |             setActiveCourseId(courseId);
474 |
475 |             // Hydrate pipeline agents with actual telemetry or fallback to Option B step indicators
476 |             const telemetry = data.curriculum_analysis?.pipeline_telemetry;
477 |             const staticPipeline: AgentStatus[] = [
478 |                 { name: "Curriculum Analysis Agent", purpose: "Extracts modules, topics, and structures from
raw syllabus text.", status: "completed" },
479 |                 { name: "Learning Outcome Agent", purpose: "Generates measurable learning outcomes mapped to
Bloom's Taxonomy.", status: "completed" },
480 |                 { name: "Curriculum Planning Agent", purpose: "Coordinates weekly sequencing, lesson
scheduling, and academic pacing.", status: "completed" },
481 |                 { name: "Slide Generation Agent", purpose: "Generates slide titles, layout outlines, and
content bullet points.", status: "completed" },
482 |                 { name: "Instructor Notes Agent", purpose: "Develops comprehensive lecturer talking points and
real-world examples.", status: "completed" },

```



```

483 |         { name: "Assessment Agent", purpose: "Creates diagnostic assessment questions mapped to
learning outcomes.", status: "completed" },
484 |         { name: "Bloom Audit Agent", purpose: "Analyzes cognitive balance and produces a Bloom level
distribution report.", status: "completed" },
485 |         { name: "Industry Gap Agent", purpose: "Benchmarks curriculum mapping against modern technology
requirements.", status: "completed" },
486 |         { name: "Readiness Score Agent", purpose: "Computes overall 100-point accreditation readiness
scoring.", status: "completed" }
487 |     ].map((agent, idx): AgentStatus => {
488 |         const tItem = telemetry?.find((t: any) => t.agent === agent.name);
489 |         if (tItem) {
490 |             const durationSec = typeof tItem.duration_ms === 'number' ? tItem.duration_ms / 1000 :
tItem.duration_seconds;
491 |             const formattedDuration = typeof durationSec === 'number' ? `${durationSec.toFixed(1)}s` :
'0.0s';
492 |             return {
493 |                 name: agent.name,
494 |                 purpose: agent.purpose,
495 |                 status: "completed",
496 |                 timestamp: `Completed - ${formattedDuration}`,
497 |                 duration: durationSec
498 |             };
499 |         }
500 |         return {
501 |             name: agent.name,
502 |             purpose: agent.purpose,
503 |             status: "completed",
504 |             timestamp: `Step ${idx + 1} of 9`
505 |         };
506 |     });
507 |     setPipelineAgents(staticPipeline);
508 |
509 |     // Set slide editing values
510 |     if (data.slides && data.slides.length > 0) {
511 |         const indexToLoad = (preserveSlideIndex && activeSlideIndex < data.slides.length) ?
activeSlideIndex : 0;
512 |         loadSlideEditState(data, indexToLoad);
513 |     }
514 |     } else if (res.status === 401 || res.status === 403) {
515 |         handleAuthError();
516 |     }
517 |     } catch (err) {
518 |         console.error("Get course details err:", err);
519 |     }
520 | }
521 |
522 | async function fetchCourses(token: string) {
523 |     try {
524 |         const res = await fetch(`${backendUrl}/api/courses/`, {
525 |             headers: { "Authorization": `Bearer ${token}` }
526 |         });
527 |         if (res.ok) {
528 |             const data = await res.json();
529 |             setCourses(data);
530 |             if (data.length > 0) {
531 |                 // Auto select first course
532 |                 selectCourse(data[0].id, token);
533 |             }
534 |         } else if (res.status === 401 || res.status === 403) {
535 |             handleAuthError();
536 |         }
537 |     } catch (err) {
538 |         console.error("Fetch courses err:", err);
539 |     }
540 | }
541 |

```

```

542 | // Load authentication state from localStorage on mount
543 | useEffect(() => {
544 |   const savedToken = localStorage.getItem("kalyx_token");
545 |   const savedUser = localStorage.getItem("kalyx_user");
546 |   if (savedToken && savedUser) {
547 |     setAuthToken(savedToken);
548 |     try {
549 |       const parsedUser = JSON.parse(savedUser);
550 |       setUser(parsedUser);
551 |       fetchCourses(savedToken);
552 |     } catch (err) {
553 |       console.error("Failed to parse saved user credentials:", err);
554 |     }
555 |   }
556 |   // eslint-disable-next-line react-hooks/exhaustive-deps
557 | }, []);
558 |
559 | const handleCreateCourse = async () => {
560 |   if (!newCourseTitle.trim() || !authToken) return;
561 |   try {
562 |     const res = await fetch(`${backendUrl}/api/courses/`, {
563 |       method: "POST",
564 |       headers: {
565 |         "Content-Type": "application/json",
566 |         "Authorization": `Bearer ${authToken}`
567 |       },
568 |       body: JSON.stringify({ title: newCourseTitle, description: newCourseDesc })
569 |     });
570 |     if (res.ok) {
571 |       const data = await res.json();
572 |       setCourses(prev => [data, ...prev]);
573 |       setActiveCourseId(data.id);
574 |       setCourseDetails(null); // Clear out previous details
575 |       setShowCreateModal(false);
576 |       setNewCourseTitle("");
577 |       setNewCourseDesc("");
578 |       setActiveTab("overview");
579 |     } else if (res.status === 401 || res.status === 403) {
580 |       handleAuthError();
581 |     } else {
582 |       const errData = await res.json().catch(() => ({}));
583 |       alert(`Failed to create package: ${errData.detail || errData.error || res.statusText || 'Unknown
error'}`);
584 |     }
585 |   } catch (err) {
586 |     console.error("Create course err:", err);
587 |     alert("Error connecting to backend server.");
588 |   }
589 | };
590 |
591 | const handleDeleteCourse = async (courseId: number, e: React.MouseEvent) => {
592 |   e.stopPropagation();
593 |   if (!authToken) return;
594 |   if (!confirm("Are you sure you want to delete this course package?")) return;
595 |   try {
596 |     const res = await fetch(`${backendUrl}/api/courses/${courseId}`, {
597 |       method: "DELETE",
598 |       headers: { "Authorization": `Bearer ${authToken}` }
599 |     });
600 |     if (res.ok) {
601 |       const updated = courses.filter(c => c.id !== courseId);
602 |       setCourses(updated);
603 |       if (updated.length > 0) {
604 |         selectCourse(updated[0].id);
605 |       } else {
606 |         setActiveCourseId(null);

```

```

607 |         setCourseDetails(null);
608 |     }
609 | }
610 | } catch (err) {
611 |     console.error("Delete course err:", err);
612 | }
613 | };
614 |
615 | // Syllabus Parsing & Agent Pipeline Trigger
616 | const handleAnalyzeSyllabus = async (e: React.FormEvent) => {
617 |     e.preventDefault();
618 |     if (!activeCourseId || !uploadFile || !authToken) return;
619 |
620 |     setIsUploading(true);
621 |     setUploadLogs(["Initiating connection to KALYX backend...", "Allocating dedicated RAG chunk
vectors..."]);
622 |     const pipelineInterval = startPipelineSimulation();
623 |
624 |     const formData = new FormData();
625 |     formData.append("file", uploadFile);
626 |
627 |     // Dynamic terminal updates to simulate real multi-agent work
628 |     const logInterval = setInterval(() => {
629 |         const agents = [
630 |             "Curriculum Intelligence Agent: Scrutinizing syllabus text for educational modules...",
631 |             "Content Generation Agent: Drafting customized slides and instructor talking points...",
632 |             "Assessment Intelligence Agent: Structuring diagnostic test items and cognitive metrics...",
633 |             "Curriculum Evaluation Agent: Auditing industry gaps and compliance scores..."
634 |         ];
635 |         const randomMsg = agents[Math.floor(Math.random() * agents.length)];
636 |         setUploadLogs(prev => [...prev, randomMsg]);
637 |     }, 1800);
638 |
639 |     try {
640 |         const res = await fetch(`${backendUrl}/api/courses/${activeCourseId}/analyze`, {
641 |             method: "POST",
642 |             headers: {
643 |                 "Authorization": `Bearer ${authToken}`
644 |             },
645 |             body: formData
646 |         });
647 |
648 |         clearInterval(logInterval);
649 |         clearInterval(pipelineInterval);
650 |
651 |         if (res.ok) {
652 |             const data = await res.json();
653 |             if (data.status === "partial_generation") {
654 |                 setUploadLogs(prev => [
655 |                     ...prev,
656 |                     `WARNING: Pipeline interrupted due to API ${data.error_type} at ${data.failed_agent}.`,
657 |                     `Successfully completed stages: ${data.successful_agents.join(", ") || "None"}.`,
658 |                     "Partial classroom package saved. Please retry later to complete the remaining sections."
659 |                 ]);
660 |                 failPipelineSimulation();
661 |                 setIsUploading(false);
662 |                 setUploadFile(null);
663 |                 selectCourse(activeCourseId);
664 |                 alert(`Partial Generation Warning:\n\nThe pipeline was interrupted due to Gemini API Rate Limit
/ Resource Exhaustion (${data.error_type}) at ${data.failed_agent}.\n\nSuccessful sections:
${data.successful_agents.join(", ") || "None"}.\n\nSuccessfully generated items have been saved. You can try
regenerating later to complete the rest!`);
665 |                 return;
666 |             }
667 |             setUploadLogs(prev => [...prev, ...data.logs, "SUCCESS: Complete classroom package compiled!"]);
668 |             completePipelineSimulation(data.pipeline_telemetry);

```

```

669 |         // Hydrate workspace
670 |         setTimeout(() => {
671 |             setIsUploading(false);
672 |             setUploadFile(null);
673 |             selectCourse(activeCourseId);
674 |         }, 1500);
675 |     } else {
676 |         setIsUploading(false);
677 |         failPipelineSimulation();
678 |         alert("Failed to analyze syllabus. Please try again.");
679 |     }
680 | } catch (err) {
681 |     clearInterval(logInterval);
682 |     clearInterval(pipelineInterval);
683 |     setIsUploading(false);
684 |     failPipelineSimulation();
685 |     console.error("Analyze syllabus err:", err);
686 |     alert("Error occurred. Check that the backend server is running.");
687 | }
688 | };
689 |
690 | // Save edits back to database in AI Studio
691 | const handleSaveSlideChanges = async () => {
692 |     if (!courseDetails || !authToken) return;
693 |     const slide = courseDetails.slides[activeSlideIndex];
694 |     if (!slide) return;
695 |
696 |     try {
697 |         // 1. Save Slide bullet structure
698 |         const slideRes = await fetch(`${backendUrl}/api/studio/slides/${slide.id}`, {
699 |             method: "PUT",
700 |             headers: {
701 |                 "Content-Type": "application/json",
702 |                 "Authorization": `Bearer ${authToken}`
703 |             },
704 |             body: JSON.stringify({
705 |                 title: editSlideTitle,
706 |                 content: editSlideBullets,
707 |                 suggested_visuals: editSlideVisuals
708 |             })
709 |         });
710 |
711 |         // 2. Save note talking points structure
712 |         const note = courseDetails.notes.find(n => n.slide_index === slide.slide_index);
713 |         if (note) {
714 |             await fetch(`${backendUrl}/api/studio/notes/${note.id}`, {
715 |                 method: "PUT",
716 |                 headers: {
717 |                     "Content-Type": "application/json",
718 |                     "Authorization": `Bearer ${authToken}`
719 |                 },
720 |                 body: JSON.stringify({
721 |                     talking_points: editNoteTalkingPoints,
722 |                     teaching_tips: editNoteTips,
723 |                     examples: editNoteExamples
724 |                 })
725 |             });
726 |         }
727 |
728 |         if (slideRes.ok) {
729 |             // Reload details to sync database state and preserve active slide index
730 |             await selectCourse(activeCourseId!, undefined, true);
731 |             if (activeTab === "notes") {
732 |                 alert("Instructor speaker notes saved successfully!");
733 |             } else {
734 |                 alert("Educational slide adjustments saved successfully!");

```

```

735 |     }
736 |   }
737 |   } catch (err) {
738 |     console.error("Save slide edits err:", err);
739 |   }
740 | };
741 |
742 | // Export triggers
743 | const handleDownloadPptx = () => {
744 |   if (!activeCourseId) return;
745 |   window.open(`${backendUrl}/api/export/courses/${activeCourseId}/pptx?token=${authToken}`);
746 | };
747 |
748 | const handleDownloadPdf = () => {
749 |   if (!activeCourseId) return;
750 |   window.open(`${backendUrl}/api/export/courses/${activeCourseId}/pdf?token=${authToken}`);
751 | };
752 |
753 |
754 | const handleDownloadInteractiveQuiz = () => {
755 |   if (!activeCourseId) return;
756 |
window.open(`${backendUrl}/api/export/courses/${activeCourseId}/interactive-quiz?token=${authToken}`);
757 | };
758 |
759 | const handleRegenerateDeck = async () => {
760 |   if (!activeCourseId || !authToken) return;
761 |   setIsUploading(true);
762 |   setUploadLogs(["Initiating connection to KALYX backend...", "Saving personalization profile
details..."]);
763 |   const pipelineInterval = startPipelineSimulation();
764 |
765 |   try {
766 |     const saveProfileRes = await
fetch(`${backendUrl}/api/studio/courses/${activeCourseId}/personalize`, {
767 |       method: "POST",
768 |       headers: {
769 |         "Content-Type": "application/json",
770 |         "Authorization": `Bearer ${authToken}`
771 |       },
772 |       body: JSON.stringify({
773 |         profile: {
774 |           tone: personalization.tone,
775 |           style: personalization.style,
776 |           examplesCount: personalization.examplesCount,
777 |           customInstructions: personalization.customInstructions
778 |         }
779 |       })
780 |     });
781 |
782 |     if (!saveProfileRes.ok) {
783 |       throw new Error("Failed to save personalization details.");
784 |     }
785 |
786 |     setUploadLogs(prev => [...prev, "Personalization profile saved successfully.", "Triggering
multi-agent regeneration graph..."]);
787 |
788 |     const regenerateInterval = setInterval(() => {
789 |       const agentLogs = [
790 |         "Curriculum Intelligence Agent: Scrutinizing syllabus text for educational modules...",
791 |         "Content Generation Agent: Drafting customized slides and instructor talking points...",
792 |         "Assessment Intelligence Agent: Structuring diagnostic test items and cognitive metrics...",
793 |         "Curriculum Evaluation Agent: Auditing industry gaps and compliance scores..."
794 |       ];
795 |       const randomMsg = agentLogs[Math.floor(Math.random() * agentLogs.length)];
796 |       setUploadLogs(prev => [...prev, randomMsg]);

```

```

797 |     }, 1800);
798 |
799 |     const res = await fetch(`${backendUrl}/api/courses/${activeCourseId}/regenerate`, {
800 |         method: "POST",
801 |         headers: {
802 |             "Authorization": `Bearer ${authToken}`
803 |         }
804 |     });
805 |
806 |     clearInterval(regenerateInterval);
807 |     clearInterval(pipelineInterval);
808 |
809 |     if (res.ok) {
810 |         const data = await res.json();
811 |         if (data.status === "partial_generation") {
812 |             setUploadLogs(prev => [
813 |                 ...prev,
814 |                 `WARNING: Pipeline interrupted due to API ${data.error_type} at ${data.failed_agent}.`,
815 |                 `Successfully completed stages: ${data.successful_agents.join(", ") || "None"}.`,
816 |                 "Partial classroom package saved. Please retry later to complete the remaining sections."
817 |             ]);
818 |             failPipelineSimulation();
819 |             setIsUploading(false);
820 |             selectCourse(activeCourseId);
821 |             alert(`Partial Generation Warning:\n\nThe pipeline was interrupted due to Gemini API Rate Limit
/ Resource Exhaustion (${data.error_type}) at ${data.failed_agent}.\n\nSuccessful sections:
${data.successful_agents.join(", ") || "None"}.\n\nSuccessfully generated items have been saved. You can try
regenerating later to complete the rest!`);
822 |             return;
823 |         }
824 |         setUploadLogs(prev => [...prev, ...data.logs, "SUCCESS: Complete classroom package
regenerated!"]);
825 |         completePipelineSimulation(data.pipeline_telemetry);
826 |         setTimeout(() => {
827 |             setIsUploading(false);
828 |             selectCourse(activeCourseId);
829 |         }, 1500);
830 |     } else {
831 |         setIsUploading(false);
832 |         failPipelineSimulation();
833 |         alert("Failed to regenerate course slides. Please try again.");
834 |     }
835 |
836 | } catch (err) {
837 |     clearInterval(pipelineInterval);
838 |     setIsUploading(false);
839 |     failPipelineSimulation();
840 |     console.error("Regenerate course deck err:", err);
841 |     alert("Error occurred. Check that the backend server is running.");
842 | }
843 | };
844 |
845 | if (!authToken) {
846 |     return (
847 |         <div className="flex-1 flex items-center justify-center min-h-screen bg-[#030712] p-4 relative
overflow-hidden text-slate-100">
848 |             {/* background decorative nodes */}
849 |             <div className="absolute top-1/4 left-1/4 w-96 h-96 bg-sky-500/10 rounded-full filter
blur-[80px]" />
850 |             <div className="absolute bottom-1/4 right-1/4 w-96 h-96 bg-indigo-500/10 rounded-full filter
blur-[80px]" />
851 |
852 |             <div className="glass-panel p-8 rounded-3xl w-full max-w-md flex flex-col gap-6 text-left
relative z-10 border-white/5 shadow-2xl">
853 |                 {/* Logo / Brand Header */}
854 |                 <div className="flex items-center gap-3 justify-center mb-2">

```

```

855 |         <div className="p-2 bg-gradient-to-br from-sky-400 to-indigo-500 rounded-xl shadow-lg
shadow-sky-500/15">
856 |             <Sparkles className="h-6 w-6 text-white" />
857 |         </div>
858 |         <span className="font-extrabold text-2xl tracking-tight bg-gradient-to-r from-white
via-slate-100 to-sky-300 bg-clip-text text-transparent">KALYX</span>
859 |     </div>
860 |
861 |     <div className="text-center">
862 |         <h2 className="text-lg font-bold text-white tracking-tight">
863 |             {authMode === "login" ? "Welcome back educator" : "Create educator profile"}
864 |         </h2>
865 |         <p className="text-xs text-slate-400 mt-1">
866 |             {authMode === "login" ? "Log in to manage agentic course generation models" : "Sign up to
initialize curriculum sequences"}
867 |         </p>
868 |     </div>
869 |
870 |     {authError && (
871 |         <div className="p-3.5 bg-rose-500/10 border border-rose-500/20 rounded-xl text-xs
text-rose-400 flex items-center gap-2">
872 |             <AlertCircle className="h-4 w-4 shrink-0" />
873 |             <span>{authError}</span>
874 |         </div>
875 |     )}
876 |
877 |     <form onSubmit={handleAuthSubmit} className="flex flex-col gap-4">
878 |         {authMode === "signup" && (
879 |             <>
880 |                 <div className="flex flex-col gap-1.5">
881 |                     <label className="text-[10px] font-extrabold text-slate-400 tracking-wider">FULL
NAME</label>
882 |                     <input
883 |                         type="text"
884 |                         placeholder="e.g. Dr. John Doe"
885 |                         value={authFullName}
886 |                         onChange={(e) => setAuthFullName(e.target.value)}
887 |                         className="px-4 py-3 bg-slate-950/70 border border-slate-900 focus:border-sky-500/50
rounded-xl text-xs text-white placeholder-slate-600 outline-none transition-all"
888 |                     />
889 |                 </div>
890 |                 <div className="flex flex-col gap-1.5">
891 |                     <label className="text-[10px] font-extrabold text-slate-400 tracking-wider">EMAIL
ADDRESS</label>
892 |                     <input
893 |                         type="email"
894 |                         placeholder="e.g. john.doe@university.edu"
895 |                         value={authEmail}
896 |                         onChange={(e) => setAuthEmail(e.target.value)}
897 |                         className="px-4 py-3 bg-slate-950/70 border border-slate-900 focus:border-sky-500/50
rounded-xl text-xs text-white placeholder-slate-600 outline-none transition-all"
898 |                     />
899 |                 </div>
900 |                 <div className="flex flex-col gap-1.5">
901 |                     <label className="text-[10px] font-extrabold text-slate-400
tracking-wider">USERNAME</label>
902 |                     <input
903 |                         type="text"
904 |                         placeholder="e.g. professor.jones"
905 |                         value={authUsername}
906 |                         onChange={(e) => setAuthUsername(e.target.value)}
907 |                         className="px-4 py-3 bg-slate-950/70 border border-slate-900 focus:border-sky-500/50
rounded-xl text-xs text-white placeholder-slate-600 outline-none transition-all"
908 |                     />
909 |                 </div>
910 |             </>

```

```

911 |         })
912 |
913 |         {authMode === "login" && (
914 |             <div className="flex flex-col gap-1.5">
915 |                 <label className="text-[10px] font-extrabold text-slate-400 tracking-wider">USERNAME OR
EMAIL</label>
916 |                 <input
917 |                     type="text"
918 |                     placeholder="e.g. professor.jones or jones@stanford.edu"
919 |                     value={authUsernameOrEmail}
920 |                     onChange={(e) => setAuthUsernameOrEmail(e.target.value)}
921 |                     className="px-4 py-3 bg-slate-950/70 border border-slate-900 focus:border-sky-500/50
rounded-xl text-xs text-white placeholder-slate-600 outline-none transition-all"
922 |                 />
923 |             </div>
924 |         })
925 |
926 |         <div className="flex flex-col gap-1.5">
927 |             <label className="text-[10px] font-extrabold text-slate-400
tracking-wider">PASSWORD</label>
928 |             <input
929 |                 type="password"
930 |                 placeholder="-----"
931 |                 value={authPassword}
932 |                 onChange={(e) => setAuthPassword(e.target.value)}
933 |                 className="px-4 py-3 bg-slate-950/70 border border-slate-900 focus:border-sky-500/50
rounded-xl text-xs text-white placeholder-slate-600 outline-none transition-all"
934 |             />
935 |         </div>
936 |
937 |         <button
938 |             type="submit"
939 |             className="mt-2 w-full py-3.5 bg-gradient-to-r from-sky-500 to-indigo-500
hover:from-sky-400 hover:to-indigo-400 rounded-xl text-xs font-bold text-white shadow-xl shadow-indigo-500/10
active:scale-[0.98] transition-all cursor-pointer"
940 |         >
941 |             {authMode === "login" ? "Sign In" : "Sign Up"}
942 |         </button>
943 |     </form>
944 |
945 |     <div className="text-center text-xs text-slate-400 mt-2">
946 |         {authMode === "login" ? (
947 |             <span>
948 |                 New to KALYX?{" "}
949 |                 <button onClick={() => { setAuthMode("signup"); setAuthError(""); }}
className="text-sky-400 hover:text-sky-300 font-bold hover:underline transition-all">
950 |                     Create an account
951 |                 </button>
952 |             </span>
953 |         ) : (
954 |             <span>
955 |                 Already have an account?{" "}
956 |                 <button onClick={() => { setAuthMode("login"); setAuthError(""); }}
className="text-sky-400 hover:text-sky-300 font-bold hover:underline transition-all">
957 |                     Sign in
958 |                 </button>
959 |             </span>
960 |         )}
961 |     </div>
962 |
963 |     <div className="border-t border-slate-900/60 pt-4 text-center">
964 |         <span className="text-[10px] font-mono text-slate-500">
965 |             Demo Credentials: <span className="text-indigo-400 font-bold">professor.jones</span> /
<span className="text-indigo-400 font-bold">hackathon_demo_pass</span>
966 |         </span>
967 |     </div>

```



```

968 |         </div>
969 |     </div>
970 | );
971 | }
972 |
973 | return (
974 |     <div className="flex-1 flex flex-col text-slate-100 min-h-screen">
975 |
976 |         {/* ----- 1. LANDING PAGE VIEW ----- */}
977 |         {view === "landing" && (
978 |             <div className="flex-1 flex flex-col justify-start p-6 md:p-12 relative overflow-y-auto h-screen
scroll-smooth gap-12">
979 |                 {/* Animated Background decorative items */}
980 |                 <div className="absolute top-1/4 left-1/4 w-96 h-96 bg-sky-500/10 rounded-full filter
blur-[80px] animate-pulse-glow" />
981 |                 <div className="absolute bottom-1/4 right-1/4 w-96 h-96 bg-indigo-500/10 rounded-full filter
blur-[80px] animate-pulse-glow" style={{ animationDelay: "2s" }} />
982 |
983 |                 {/* Navigation Bar */}
984 |                 <header className="flex justify-between items-center max-w-7xl w-full mx-auto z-10">
985 |                     <div className="flex items-center gap-3">
986 |                         <div className="p-2 bg-gradient-to-br from-sky-400 to-indigo-500 rounded-xl shadow-lg
shadow-sky-500/10">
987 |                             <Sparkles className="h-6 w-6 text-white" />
988 |                         </div>
989 |                         <span className="font-extrabold text-xl tracking-tight bg-gradient-to-r from-white
via-slate-100 to-sky-300 bg-clip-text text-transparent">KALYX</span>
990 |                     </div>
991 |                     <div className="flex items-center gap-4">
992 |                         <span className="text-xs text-slate-400 hidden sm:inline">Logged in as: <strong
className="text-white font-bold">{user ? user.full_name : ""}</strong></span>
993 |                         <button
994 |                             onClick={handleLogout}
995 |                             className="px-3.5 py-1.5 bg-slate-900 border border-slate-800 hover:border-slate-700
hover:text-white rounded-xl text-xs font-semibold active:scale-95 transition-all cursor-pointer"
996 |                         >
997 |                             Logout
998 |                         </button>
999 |                         <button
1000 |                             onClick={() => setView("workspace")}
1001 |                             className="px-5 py-2.5 bg-gradient-to-r from-sky-500 to-indigo-500 hover:from-sky-400
hover:to-indigo-400 rounded-xl text-sm font-semibold shadow-xl shadow-sky-500/20 active:scale-95 transition-all
cursor-pointer"
1002 |                         >
1003 |                             Launch Workspace
1004 |                         </button>
1005 |                     </div>
1006 |                 </header>
1007 |
1008 |                 {/* Hero Section */}
1009 |                 <main className="max-w-7xl w-full mx-auto flex flex-col lg:flex-row items-center gap-16 py-12
z-10 flex-1 justify-center min-h-[calc(100vh-160px)] shrink-0">
1010 |
1011 |                     {/* Left Copy block */}
1012 |                     <div className="flex-1 flex flex-col gap-6 text-left max-w-2xl">
1013 |                         <div className="inline-flex items-center gap-2 px-3.5 py-1.5 rounded-full bg-slate-900/80
border border-slate-800 text-xs font-medium text-sky-400 w-fit">
1014 |                             <Sparkles className="h-3.5 w-3.5" />
1015 |                             <span>Next-Gen Agentic Hackathon Product</span>
1016 |                         </div>
1017 |
1018 |                         <h1 className="text-4xl sm:text-6xl font-black tracking-tight leading-[1.08] text-white">
1019 |                             Agentic <span className="bg-gradient-to-r from-sky-400 via-sky-300 to-indigo-400
bg-clip-text text-transparent">Curriculum</span> Intelligence.
1020 |                         </h1>
1021 |

```

```

1022 |         <p className="text-lg text-slate-300 leading-relaxed">
1023 |             KALYX is an intelligent multi-agent pipeline that transforms an academic syllabus into
classroom-ready slides, pedagogical notes, targeted assessments, and modernization recommendations.
1024 |         </p>
1025 |
1026 |         <div className="flex flex-col sm:flex-row gap-4 mt-4">
1027 |             <button
1028 |                 onClick={() => setView("workspace")}
1029 |                 className="px-8 py-4 bg-gradient-to-r from-sky-500 to-indigo-500 hover:from-sky-400
hover:to-indigo-400 rounded-xl text-base font-bold shadow-2xl shadow-sky-500/20 active:scale-95 transition-all
flex items-center justify-center gap-2 text-white"
1030 |             >
1031 |                 Enter Platform
1032 |                 <ArrowRight className="h-5 w-5" />
1033 |             </button>
1034 |             <a
1035 |                 href="#features"
1036 |                 className="px-8 py-4 glass-panel glass-panel-hover rounded-xl text-base font-semibold
flex items-center justify-center text-slate-200"
1037 |             >
1038 |                 Explore Capabilities
1039 |             </a>
1040 |         </div>
1041 |     </div>
1042 |
1043 |     { /* Right futuristic Visual Preview */ }
1044 |     <div className="flex-1 w-full max-w-lg lg:max-w-none relative">
1045 |         <div className="glass-panel p-2 rounded-2xl border-white/10 shadow-2xl relative
overflow-hidden bg-slate-950/40">
1046 |
1047 |             { /* Visual Window Header */ }
1048 |             <div className="flex justify-between items-center px-4 py-3 bg-slate-900/60 rounded-t-xl
border-b border-slate-800">
1049 |                 <div className="flex items-center gap-2">
1050 |                     <span className="w-3 h-3 rounded-full bg-rose-500/80" />
1051 |                     <span className="w-3 h-3 rounded-full bg-amber-500/80" />
1052 |                     <span className="w-3 h-3 rounded-full bg-emerald-500/80" />
1053 |                 </div>
1054 |                 <span className="text-xs font-mono text-slate-400">Kalyx_Platform_Preview.tsx</span>
1055 |                 <div className="w-12" />
1056 |             </div>
1057 |
1058 |             { /* Simulated UI Screen */ }
1059 |             <div className="p-6 bg-slate-950/80 flex flex-col gap-6 text-left rounded-b-xl relative
min-h-[300px]">
1060 |                 <div className="flex justify-between items-center">
1061 |                     <div className="flex items-center gap-3">
1062 |                         <div className="h-8 w-8 rounded-lg bg-sky-500/20 flex items-center justify-center">
1063 |                             <Sparkles className="h-4.5 w-4.5 text-sky-400" />
1064 |                         </div>
1065 |                         <div>
1066 |                             <h4 className="text-sm font-bold text-white leading-none">Advanced Machine
Learning</h4>
1067 |                             <span className="text-[10px] text-slate-500">Dr. John Doe</span>
1068 |                         </div>
1069 |                     </div>
1070 |                     <div className="px-2.5 py-1 rounded bg-sky-500/10 border border-sky-500/20
text-sky-400 text-xs font-extrabold">
1071 |                         89 / 100 Readiness
1072 |                     </div>
1073 |                 </div>
1074 |
1075 |             { /* Slides mini preview card */ }
1076 |             <div className="glass-panel p-4 rounded-xl border-white/5 bg-slate-900/40 flex flex-col
gap-3 relative">
1077 |                 <div className="text-xs font-extrabold text-sky-400 tracking-wider">GENERATED SLIDE

```

```

3</div>
1078 |           <div className="text-base font-bold text-white leading-tight">Linear Regression &
Gradient Descent</div>
1079 |           <div className="flex flex-col gap-1.5 text-xs text-slate-300">
1080 |             <div>- Cost Function: Mean Squared Error (MSE) measures prediction errors</div>
1081 |             <div>- Gradient Descent: Iterative weight updates using alpha learning rate</div>
1082 |           </div>
1083 |           {/* Visual box overlay */}
1084 |           <div className="absolute right-3 bottom-3 text-[9px] text-slate-500 font-mono border
border-slate-800 px-2 py-0.5 rounded">
1085 |             [ AI Visual: Cost Bowl ]
1086 |           </div>
1087 |         </div>
1088 |
1089 |         {/* Agent workflow indicators */}
1090 |         <div className="flex justify-between items-center text-xs border-t border-slate-900
pt-4">
1091 |           <div className="flex items-center gap-2 text-slate-400">
1092 |             <CheckCircle2 className="h-4 w-4 text-emerald-400" />
1093 |             <span>9-Agent Validation Loop</span>
1094 |           </div>
1095 |           <div className="flex items-center gap-1.5 text-sky-400 font-semibold">
1096 |             <span>Interactive Studio ready</span>
1097 |             <ChevronRight className="h-3 w-3" />
1098 |           </div>
1099 |         </div>
1100 |       </div>
1101 |     </div>
1102 |   </div>
1103 | </main>
1104 |
1105 |   {/* Capabilities Section */}
1106 |   <section id="features" className="max-w-7xl w-full mx-auto py-20 border-t border-slate-900/60
z-10 flex flex-col gap-12 scroll-mt-6 shrink-0">
1107 |     <div className="text-center max-w-3xl mx-auto flex flex-col gap-4">
1108 |       <div className="inline-flex items-center gap-2 px-3.5 py-1.5 rounded-full bg-sky-500/10
border border-sky-500/20 text-xs font-semibold text-sky-400 w-fit mx-auto">
1109 |         <Sparkles className="h-3.5 w-3.5 animate-pulse" />
1110 |         <span>Explore KALYX Capabilities</span>
1111 |       </div>
1112 |       <h2 className="text-3xl sm:text-5xl font-black tracking-tight text-white leading-tight">
An Intelligent Multi-Agent Engine
1113 |     </h2>
1114 |     <p className="text-slate-400 text-sm sm:text-base leading-relaxed">
Kalyx coordinates specialized AI agents working in sequence to extract pedagogical value,
validate cognitive depth, and formulate pristine classroom instruction materials.
1117 |     </p>
1118 |   </div>
1119 |
1120 |   <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
1121 |     {/* Feature 1 */}
1122 |     <div className="glass-panel p-6 rounded-2xl border-white/5 bg-slate-950/40
hover:border-sky-500/30 hover:scale-[1.02] transition-all flex flex-col gap-4 group">
1124 |       <div className="p-3 bg-sky-500/10 border border-sky-500/20 rounded-xl w-fit
group-hover:bg-sky-500/20 transition-all text-sky-400">
1125 |         <Layers className="h-6 w-6" />
1126 |       </div>
1127 |       <div>
1128 |         <h3 className="text-base font-bold text-white mb-2">Multi-Agent Planning</h3>
1129 |         <p className="text-xs text-slate-400 leading-relaxed">
Transforms raw academic syllabus inputs into a structured curriculum timeline and
modular lecture sequence using hierarchical planning graphs.
1131 |         </p>
1132 |       </div>
1133 |     </div>

```

```

1134 |
1135 |         {/* Feature 2 */}
1136 |         <div className="glass-panel p-6 rounded-2xl border-white/5 bg-slate-950/40
hover:border-indigo-500/30 hover:scale-[1.02] transition-all flex flex-col gap-4 group">
1137 |             <div className="p-3 bg-indigo-500/10 border border-indigo-500/20 rounded-xl w-fit
group-hover:bg-indigo-500/20 transition-all text-indigo-400">
1138 |                 <Award className="h-6 w-6" />
1139 |             </div>
1140 |         </div>
1141 |         <h3 className="text-base font-bold text-white mb-2">Taxonomic Auditing</h3>
1142 |         <p className="text-xs text-slate-400 leading-relaxed">
1143 |             Audits lesson coverage across all six levels of Bloom's Taxonomy, checking if
cognitive depth matches educational standards.
1144 |         </p>
1145 |     </div>
1146 | </div>
1147 |
1148 |     {/* Feature 3 */}
1149 |     <div className="glass-panel p-6 rounded-2xl border-white/5 bg-slate-950/40
hover:border-emerald-500/30 hover:scale-[1.02] transition-all flex flex-col gap-4 group">
1150 |         <div className="p-3 bg-emerald-500/10 border border-emerald-500/20 rounded-xl w-fit
group-hover:bg-emerald-500/20 transition-all text-emerald-400">
1151 |             <GraduationCap className="h-6 w-6" />
1152 |         </div>
1153 |     </div>
1154 |     <h3 className="text-base font-bold text-white mb-2">Targeted Quizzes</h3>
1155 |     <p className="text-xs text-slate-400 leading-relaxed">
1156 |         Generates diagnostic question banks, multiple-choice quizzes, and interactive
classroom tests mapped directly to learning outcomes.
1157 |     </p>
1158 | </div>
1159 | </div>
1160 |
1161 |     {/* Feature 4 */}
1162 |     <div className="glass-panel p-6 rounded-2xl border-white/5 bg-slate-950/40
hover:border-pink-500/30 hover:scale-[1.02] transition-all flex flex-col gap-4 group">
1163 |         <div className="p-3 bg-pink-500/10 border border-pink-500/20 rounded-xl w-fit
group-hover:bg-pink-500/20 transition-all text-pink-400">
1164 |             <Download className="h-6 w-6" />
1165 |         </div>
1166 |     </div>
1167 |     <h3 className="text-base font-bold text-white mb-2">Enterprise Exports</h3>
1168 |     <p className="text-xs text-slate-400 leading-relaxed">
1169 |         Downloads pristine, high-fidelity slide decks in PPTX format, complete speaker notes,
comprehensive PDFs, or interactive tests.
1170 |     </p>
1171 | </div>
1172 | </div>
1173 |
1174 | </div>
1175 | </section>
1176 |
1177 |     {/* Footer */}
1178 |     <footer className="max-w-7xl w-full mx-auto border-t border-slate-900 py-6 text-center text-xs
text-slate-500 z-10 shrink-0">
1179 |         KALYX - Developed for Agentic AI Hackathon 2026. Made with FastAPI, Next.js, and LangGraph.
1180 |     </footer>
1181 | </div>
1182 | })
1183 |
1184 |     {/* ----- 2. WORKSPACE DASHBOARD VIEW ----- */}
1185 |     {view === "workspace" && (
1186 |         <div className="flex-1 flex min-h-screen overflow-hidden bg-[#030712]">
1187 |
1188 |             {/* A. WORKSPACE LEFT SIDEBAR */}
1189 |             <aside className="w-80 bg-slate-950 border-r border-slate-900 flex flex-col justify-between

```

```

shrink-0">
1190 |
1191 |         {/* Top Brand section */}
1192 |         <div className="p-5 flex flex-col gap-6">
1193 |
1194 |             {/* Logo / Brand Header */}
1195 |             <div className="flex items-center justify-between">
1196 |                 <div className="flex items-center gap-2.5">
1197 |                     <div className="p-1.5 bg-gradient-to-br from-sky-500 to-indigo-500 rounded-lg">
1198 |                         <Sparkles className="h-5 w-5 text-white" />
1199 |                     </div>
1200 |                     <span className="font-black text-lg tracking-tight bg-gradient-to-r from-white
to-sky-300 bg-clip-text text-transparent">KALYX</span>
1201 |                 </div>
1202 |                 <div className="flex gap-1">
1203 |                     <button
1204 |                         onClick={() => setView("landing")}
1205 |                         className="text-[10px] text-slate-400 hover:text-white px-2 py-0.5 border
border-slate-800 rounded hover:bg-slate-900 cursor-pointer"
1206 |                     >
1207 |                         Home
1208 |                     </button>
1209 |                     <button
1210 |                         onClick={handleLogout}
1211 |                         className="text-[10px] text-slate-400 hover:text-white px-2 py-0.5 border
border-rose-950/20 bg-rose-500/5 hover:bg-rose-500/10 rounded cursor-pointer"
1212 |                     >
1213 |                         Logout
1214 |                     </button>
1215 |                 </div>
1216 |             </div>
1217 |
1218 |             {/* Course Selector card */}
1219 |             <div className="flex flex-col gap-2">
1220 |                 <div className="flex justify-between items-center">
1221 |                     <span className="text-[10px] font-extrabold text-slate-500 tracking-wider">ACTIVE
COURSES</span>
1222 |                     <button
1223 |                         onClick={() => setShowCreateModal(true)}
1224 |                         className="p-1 text-slate-400 hover:text-white bg-slate-900 border border-slate-800
rounded hover:border-slate-700"
1225 |                         title="Create new course"
1226 |                     >
1227 |                         <Plus className="h-3.5 w-3.5" />
1228 |                     </button>
1229 |                 </div>
1230 |
1231 |                 <div className="flex flex-col gap-1.5 max-h-[220px] overflow-y-auto pr-1">
1232 |                     {courses.map((c) => (
1233 |                         <div
1234 |                             key={c.id}
1235 |                             onClick={() => selectCourse(c.id)}
1236 |                             className={`group p-3 rounded-xl border flex items-center justify-between
cursor-pointer transition-all ${
1237 |                                 activeCourseId === c.id
1238 |                                 ? "bg-sky-500/10 border-sky-500/40 text-white"
1239 |                                 : "bg-slate-900/40 border-slate-900 hover:bg-slate-900 text-slate-300"
1240 |                             }`}
1241 |                         >
1242 |                             <div className="flex items-center gap-3 overflow-hidden">
1243 |                                 <BookOpen className={`h-4.5 w-4.5 shrink-0 ${activeCourseId === c.id ?
"text-sky-400" : "text-slate-500"}`} />
1244 |                                 <span className="text-xs font-bold truncate leading-none">{c.title}</span>
1245 |                             </div>
1246 |                             <button
1247 |                                 onClick={(e) => handleDeleteCourse(c.id, e)}

```

```

1248 |         className="opacity-0 group-hover:opacity-100 p-1 text-slate-500
hover:text-rose-400 hover:bg-slate-800/80 rounded transition-all"
1249 |         >
1250 |         <Trash2 className="h-3.5 w-3.5" />
1251 |     </button>
1252 | </div>
1253 |     )}}
1254 |     {courses.length === 0 && (
1255 |         <div className="text-center py-6 border border-dashed border-slate-900 rounded-xl
text-slate-500 text-xs">
1256 |             No active courses. Create one to begin.
1257 |         </div>
1258 |     )}
1259 | </div>
1260 | </div>
1261 |
1262 |     { /* Sidebar Action Tabs */ }
1263 |     <nav className="flex flex-col gap-1.5 border-t border-slate-900 pt-6">
1264 |         <span className="text-[10px] font-extrabold text-slate-500 tracking-wider
mb-1.5">PLATFORM SECTIONS</span>
1265 |         {[
1266 |             { id: "overview", label: "Course Dashboard", icon: Layers },
1267 |             { id: "slides", label: "AI Studio (Slides)", icon: Eye },
1268 |             { id: "notes", label: "Instructor Notes", icon: Volume2 },
1269 |             { id: "assessments", label: "Assessment Bank", icon: GraduationCap },
1270 |             { id: "bloom", label: "Bloom Taxonomy Audit", icon: Award },
1271 |             { id: "readiness", label: "Readiness Score", icon: BarChart3 },
1272 |             { id: "traceability", label: "Traceability", icon: BookOpen },
1273 |             { id: "pipeline", label: "Execution Pipeline", icon: Cpu },
1274 |             { id: "gaps", label: "Industry Gap Analyzer", icon: AlertCircle },
1275 |             { id: "export", label: "Export & Exporters", icon: Download }
1276 |         ] as const}.map((tab) => {
1277 |             const Icon = tab.icon;
1278 |             const isDisabled = !courseDetails && tab.id !== "overview";
1279 |             return (
1280 |                 <button
1281 |                     key={tab.id}
1282 |                     disabled={isDisabled}
1283 |                     onClick={() => setActiveTab(tab.id)}
1284 |                     className={`w-full flex items-center gap-3 px-4 py-3 rounded-xl text-xs
font-semibold border transition-all ${
1285 |                         isDisabled
1286 |                             ? "opacity-40 cursor-not-allowed border-transparent text-slate-600"
1287 |                             : activeTab === tab.id
1288 |                                 ? "bg-gradient-to-r from-sky-500/10 to-indigo-500/10 border-sky-500/30
text-sky-400"
1289 |                                 : "border-transparent text-slate-400 hover:bg-slate-900 hover:text-white"
1290 |                             }` }
1291 |                 >
1292 |                     <Icon className="h-4.5 w-4.5 shrink-0" />
1293 |                     <span>{tab.label}</span>
1294 |                 </button>
1295 |             );
1296 |         })}
1297 |     </nav>
1298 |
1299 | </div>
1300 |
1301 |     { /* User Profile display bottom */ }
1302 |     <div className="p-5 border-t border-slate-900 bg-slate-950/60 flex items-center gap-3">
1303 |         <div className="h-9.5 w-9.5 rounded-full bg-gradient-to-br from-sky-400 to-indigo-500 flex
items-center justify-center font-bold text-xs shadow-lg text-white">
1304 |             {user ? (user.full_name ? user.full_name.split(" ").map((n: string) =>
n[0]).join("").slice(0, 2).toUpperCase() : user.email.slice(0, 2).toUpperCase()) : "ED"}
1305 |         </div>
1306 |         <div className="overflow-hidden">

```

```

1307 |         <h5 className="text-xs font-bold text-white truncate">{user ? user.full_name : "Educator
Profile"}</h5>
1308 |         <span className="text-[10px] text-slate-500 truncate block">{user ? (user.username ? "@"
+ user.username : user.email) : ""}</span>
1309 |     </div>
1310 | </div>
1311 |
1312 | </aside>
1313 |
1314 | {/* B. MAIN INTERACTIVE CONTENT STREAM */}
1315 | <main className="flex-1 flex flex-col overflow-hidden bg-slate-950/40 relative">
1316 |
1317 |     {/* Header Toolbar */}
1318 |     <header className="h-16 border-b border-slate-900 bg-slate-950/80 px-6 flex items-center
justify-between shrink-0 z-10">
1319 |         <div className="flex items-center gap-3">
1320 |             <h2 className="text-sm font-bold text-white">
1321 |                 {courseDetails ? courseDetails.course.title : "Get Started"}
1322 |             </h2>
1323 |             {courseDetails?.readiness_score && (
1324 |                 <span className="px-2.5 py-0.5 rounded-full bg-sky-500/10 border border-sky-500/20
text-sky-400 text-[10px] font-black">
1325 |                     {courseDetails.readiness_score.score.toFixed(0)}/100 Readiness
1326 |                 </span>
1327 |             )}
1328 |         </div>
1329 |
1330 |         <div className="flex items-center gap-4">
1331 |             {/* Status indicators */}
1332 |             <div className="flex items-center gap-2 text-xs text-slate-400">
1333 |                 <span className="w-2 h-2 rounded-full bg-emerald-500" />
1334 |                 <span>Agent Graph Online</span>
1335 |             </div>
1336 |         </div>
1337 |     </header>
1338 |
1339 |     {/* Core Views Panels */}
1340 |     <div className="flex-1 overflow-y-auto p-6 md:p-8">
1341 |
1342 |         {/* OVERVIEW PANEL */}
1343 |         {activeTab === "overview" && (
1344 |             <div className="max-w-5xl mx-auto flex flex-col gap-8">
1345 |
1346 |                 {/* Active Course Selector Welcome banner */}
1347 |                 {!courseDetails ? (
1348 |                     <div className="glass-panel p-8 rounded-2xl text-center border-white/5 flex flex-col
items-center justify-center py-20 max-w-xl mx-auto my-auto gap-5">
1349 |                         <div className="p-4 bg-sky-500/10 rounded-full border border-sky-500/20">
1350 |                             <UploadCloud className="h-10 w-10 text-sky-400" />
1351 |                         </div>
1352 |                         <div>
1353 |                             <h3 className="text-lg font-bold text-white">Upload Your Syllabus</h3>
1354 |                             <p className="text-xs text-slate-400 mt-1.5 leading-relaxed">
1355 |                                 To get started, select an existing course or create a new one, then upload your
course syllabus PDF/TXT to activate the 9-agent analysis.
1356 |                             </p>
1357 |                         </div>
1358 |
1359 |                         <button
1360 |                             onClick={() => setShowCreateModal(true)}
1361 |                             className="px-5 py-2.5 bg-gradient-to-r from-sky-500 to-indigo-500 rounded-xl
text-xs font-semibold shadow-xl shadow-sky-500/10"
1362 |                         >
1363 |                             Create New Course Package
1364 |                         </button>
1365 |                     </div>

```

```

1366 |         ) : (
1367 |             <>
1368 |                 { /* Grid cards overview */ }
1369 |                 <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
1370 |
1371 |                     <div className="glass-panel p-5 rounded-2xl border-white/5 flex flex-col
justify-between h-36">
1372 |                         <span className="text-[10px] font-extrabold text-slate-500
tracking-wider">TOTAL SLIDES</span>
1373 |                         <h3 className="text-3xl font-black
text-white">{courseDetails.slides.length}</h3>
1374 |                         <span className="text-[10px] text-slate-400">Classroom-ready slides
drafted</span>
1375 |                     </div>
1376 |
1377 |                     <div className="glass-panel p-5 rounded-2xl border-white/5 flex flex-col
justify-between h-36">
1378 |                         <span className="text-[10px] font-extrabold text-slate-500
tracking-wider">QUIZZES & ASSESSMENTS</span>
1379 |                         <h3 className="text-3xl font-black
text-white">{courseDetails.assessments.length}</h3>
1380 |                         <span className="text-[10px] text-slate-400">Mapped questions generated</span>
1381 |                     </div>
1382 |
1383 |                     <div className="glass-panel p-5 rounded-2xl border-white/5 flex flex-col
justify-between h-36">
1384 |                         <span className="text-[10px] font-extrabold text-slate-500
tracking-wider">BLOOM LEVELS</span>
1385 |                         <h3 className="text-3xl font-black text-white">
1386 |                             {new Set(courseDetails.learning_outcomes.map(o => o.bloom_level)).size}
1387 |                         </h3>
1388 |                         <span className="text-[10px] text-slate-400">Cognitive dimensions mapped</span>
1389 |                     </div>
1390 |
1391 |                     <div className="glass-panel p-5 rounded-2xl border-sky-500/20 bg-sky-500/5 flex
flex-col justify-between h-36">
1392 |                         <span className="text-[10px] font-extrabold text-sky-400
tracking-wider">READINESS SCORE</span>
1393 |                         <h3 className="text-3xl font-black text-sky-400">
1394 |                             {courseDetails.readiness_score?.score.toFixed(0)}/100
1395 |                         </h3>
1396 |                         <span className="text-[10px] text-slate-300">Audited educational quality</span>
1397 |                     </div>
1398 |
1399 |                 </div>
1400 |
1401 |                 { /* Course details & upload replacement form */ }
1402 |                 <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
1403 |
1404 |                     <div className="lg:col-span-2 flex flex-col gap-6">
1405 |                         { /* Course summary panel */ }
1406 |                         <div className="glass-panel p-6 rounded-2xl border-white/5">
1407 |                             <h3 className="text-sm font-extrabold text-slate-400 tracking-wide
mb-3">COURSE DESCRIPTION</h3>
1408 |                             <p className="text-sm text-slate-200 leading-relaxed">
1409 |                                 {courseDetails.course.description || "An advanced machine learning class
focusing on model architectures, regression models, classification boundaries, and support vector machines."}
1410 |                             </p>
1411 |                         </div>
1412 |
1413 |                         { /* Syllabus syllabus topics sequence roadmap */ }
1414 |                         <div className="glass-panel p-6 rounded-2xl border-white/5">
1415 |                             <h3 className="text-sm font-extrabold text-slate-400 tracking-wide
mb-4">LEARNING MODULES ROADMAP</h3>
1416 |                             <div className="flex flex-col gap-3">
1417 |                                 {courseDetails.curriculum_analysis?.curriculum_map?.modules?.map((m:

```



```

Module, idx: number) => (
1418 |                                     <div key={m.id || idx} className="flex gap-4 border-l-2 border-sky-500/20
pl-4 py-1.5 hover:border-sky-500 transition-colors">
1419 |                                     <div className="flex-1">
1420 |                                         <h4 className="text-xs font-bold text-white">Module {m.id || idx +
1}: {m.title}</h4>
1421 |                                     <div className="flex flex-wrap gap-1.5 mt-2">
1422 |                                         {m.topics?.map((topic: string, tIdx: number) => (
1423 |                                             <span key={tIdx} className="px-2 py-0.5 bg-slate-900 border
border-slate-800 rounded text-[10px] text-slate-400 font-mono">
1424 |                                                 {topic}
1425 |                                             </span>
1426 |                                         )}}
1427 |                                     </div>
1428 |                                 </div>
1429 |                             </div>
1430 |                         )}}
1431 |                     </div>
1432 |                 </div>
1433 |             </div>
1434 |
1435 |             /* Right sidebar details - replace syllabus parser */
1436 |             <div className="flex flex-col gap-6">
1437 |                 <div className="glass-panel p-6 rounded-2xl border-white/5 flex flex-col
gap-4">
1438 |                     <h3 className="text-sm font-bold text-white flex items-center gap-2">
1439 |                         <UploadCloud className="h-4.5 w-4.5 text-sky-400" />
1440 |                         <span>Syllabus Upload Center</span>
1441 |                     </h3>
1442 |                     <p className="text-xs text-slate-400 leading-relaxed">
1443 |                         Need to update details or trigger the multi-agent graph? Select a syllabus
PDF or TXT to analyze the course package.
1444 |                     </p>
1445 |
1446 |                     <form onSubmit={handleAnalyzeSyllabus} className="flex flex-col gap-3">
1447 |                         <div className="border-2 border-dashed border-slate-800 rounded-xl p-4
text-center cursor-pointer hover:border-sky-500/40 hover:bg-sky-500/5 transition-all relative">
1448 |                             <input
1449 |                                 type="file"
1450 |                                 onChange={(e) => setUploadFile(e.target.files?.[0] || null)}
1451 |                                 className="absolute inset-0 w-full h-full opacity-0 cursor-pointer"
1452 |                                 accept=".pdf, .txt, .docx"
1453 |                             />
1454 |                             <UploadCloud className="h-6 w-6 text-slate-500 mx-auto mb-2" />
1455 |                             <span className="text-xs text-slate-300 font-bold block truncate">
1456 |                                 {uploadFile ? uploadFile.name : "Select syllabus file"}
1457 |                             </span>
1458 |                             <span className="text-[10px] text-slate-500 mt-1 block">PDF, TXT, or DOCX
(max 10MB)</span>
1459 |                         </div>
1460 |
1461 |                         <button
1462 |                             type="submit"
1463 |                             disabled={!uploadFile}
1464 |                             className={`w-full py-2.5 rounded-xl text-xs font-semibold flex
items-center justify-center gap-2 transition-all ${
1465 |                                 uploadFile
1466 |                                 ? "bg-gradient-to-r from-sky-500 to-indigo-500 hover:from-sky-400
hover:to-indigo-400 text-white cursor-pointer"
1467 |                                 : "bg-slate-900 border border-slate-800 text-slate-500
cursor-not-allowed"
1468 |                             }`}
1469 |                         >
1470 |                             <Play className="h-3.5 w-3.5" />
1471 |                             <span>Trigger 9-Agent Analysis</span>
1472 |                         </button>

```

```

1473 |         </form>
1474 |     </div>
1475 | </div>
1476 |
1477 |     </div>
1478 | </>
1479 | })
1480 |
1481 | </div>
1482 | })
1483 |
1484 | {/* LEARNING OUTCOME TRACEABILITY TAB */}
1485 | {activeTab === "traceability" && courseDetails && (
1486 |     <div className="max-w-5xl mx-auto flex flex-col gap-6 text-left">
1487 |
1488 |         <div className="flex flex-col gap-1.5">
1489 |             <h3 className="text-lg font-bold text-white">Learning Outcome Traceability
Matrix</h3>
1490 |             <p className="text-xs text-slate-400">
1491 |                 Trace learning outcomes directly to slide decks, instructor speaker scripts, and
diagnostic assessment questions.
1492 |             </p>
1493 |         </div>
1494 |
1495 |         <div className="flex flex-col gap-4">
1496 |             {traceabilityData.map((outcome, idx) => {
1497 |                 const isExpanded = !!expandedOutcomes[outcome.id];
1498 |
1499 |                 // Status colors
1500 |                 let badgeClass = "text-sky-400 border-sky-500/20 bg-sky-500/10";
1501 |                 let barColor = "bg-sky-500";
1502 |                 if (outcome.coverage_percentage >= 80) {
1503 |                     badgeClass = "text-emerald-400 border-emerald-500/20 bg-emerald-500/10";
1504 |                     barColor = "bg-emerald-500";
1505 |                 } else if (outcome.coverage_percentage >= 50) {
1506 |                     badgeClass = "text-amber-400 border-amber-500/20 bg-amber-500/10";
1507 |                     barColor = "bg-amber-500";
1508 |                 } else {
1509 |                     badgeClass = "text-rose-400 border-rose-500/20 bg-rose-500/10";
1510 |                     barColor = "bg-rose-500";
1511 |                 }
1512 |
1513 |                 return (
1514 |                     <div
1515 |                         key={outcome.id}
1516 |                         className={`glass-panel rounded-2xl border-white/5 overflow-hidden
transition-all duration-300 ${
1517 |                             isExpanded ? "border-sky-500/20 shadow-lg shadow-sky-500/5" : ""
1518 |                         }`}
1519 |                     >
1520 |                         {/* Card Header */}
1521 |                         <div
1522 |                             onClick={() => setExpandedOutcomes(prev => ({ ...prev, [outcome.id]:
!prev[outcome.id] })))}
1523 |                             className="p-5 flex items-center justify-between gap-4 cursor-pointer
hover:bg-slate-900/30 transition-all select-none"
1524 |                         >
1525 |                             <div className="flex-1 flex flex-col md:flex-row md:items-center gap-3">
1526 |                                 <span className="px-2.5 py-0.5 rounded-full bg-slate-900 border
border-slate-800 text-[10px] font-black text-slate-400 shrink-0 w-fit">
1527 |                                     LO-{idx + 1}
1528 |                                 </span>
1529 |                                 <h4 className="text-xs font-bold text-white leading-relaxed line-clamp-1
md:line-clamp-none">
1530 |                                     {outcome.outcome_text}
1531 |                                 </h4>

```

```

1532 |         <span className="px-2 py-0.5 rounded-full bg-indigo-500/10 border
border-indigo-500/20 text-indigo-400 text-[9px] font-bold shrink-0 w-fit">
1533 |             Bloom: {outcome.bloom_level}
1534 |         </span>
1535 |     </div>
1536 |
1537 |     <div className="flex items-center gap-5 shrink-0">
1538 |         <div className="flex items-center gap-3">
1539 |             <span className={`px-2 py-0.5 rounded-full text-[9px] font-black border
${badgeClass}`}>
1540 |                 {outcome.coverage_percentage}% Covered
1541 |             </span>
1542 |             <div className="w-16 bg-slate-900 border border-slate-800/80 h-1.5
rounded-full overflow-hidden hidden sm:block">
1543 |                 <div className={` ${barColor} h-full rounded-full`} style={{ width:
`${outcome.coverage_percentage}%` }} />
1544 |             </div>
1545 |         </div>
1546 |         {isExpanded ? (
1547 |             <ChevronLeft className="h-4.5 w-4.5 text-slate-500 rotate-90
transition-transform" />
1548 |         ) : (
1549 |             <ChevronRight className="h-4.5 w-4.5 text-slate-500 transition-transform"
/>
1550 |         )}
1551 |     </div>
1552 | </div>
1553 |
1554 |     { /* Card Body - Mapped Deliverables */ }
1555 |     {isExpanded && (
1556 |         <div className="px-6 pb-6 border-t border-slate-900/60 bg-slate-950/20 flex
flex-col gap-5 pt-5 animate-fadeIn">
1557 |
1558 |             <div className="grid grid-cols-1 md:grid-cols-2 gap-5">
1559 |
1560 |                 { /* Related Slides Column */ }
1561 |                 <div className="flex flex-col gap-3">
1562 |                     <span className="text-[10px] font-extrabold text-slate-500
tracking-wider flex items-center gap-2">
1563 |                         <Eye className="h-4 w-4 text-sky-400" />
1564 |                         <span>COVERED BY SLIDES & NOTES</span>
1565 |                     </span>
1566 |
1567 |                     <div className="flex flex-col gap-2">
1568 |                         {outcome.related_slides.length > 0 ? (
1569 |                             outcome.related_slides.map((slide: any, sIdx: number) => {
1570 |                                 const matchedNote = outcome.related_notes.find((n: any) =>
n.slide_index === slide.slide_index);
1571 |                                 return (
1572 |                                     <div
1573 |                                         key={sIdx}
1574 |                                         onClick={() => {
1575 |                                             setActiveTab("slides");
1576 |                                             const actualIdx = courseDetails.slides.findIndex(s =>
s.slide_index === slide.slide_index);
1577 |                                             if (actualIdx !== -1) {
1578 |                                                 loadSlideEditState(courseDetails, actualIdx);
1579 |                                             }
1580 |                                         }}
1581 |                                         className="glass-panel p-3.5 rounded-xl border-white/5
hover:border-sky-500/20 cursor-pointer transition-all flex items-center justify-between text-left"
1582 |                                     >
1583 |                                         <div className="overflow-hidden">
1584 |                                             <span className="text-[9.5px] font-extrabold text-slate-500
block mb-0.5">SLIDE {slide.slide_index}</span>
1585 |                                             <p className="text-xs font-bold text-white

```

```

truncate">{slide.title}</p>
1586 |                                     </div>
1587 |                                     {matchedNote && (
1588 |                                         <span className="px-2 py-0.5 rounded-md bg-sky-500/5 border
border-sky-500/10 text-[9px] font-bold text-sky-400 shrink-0">
1589 |                                             {matchedNote.talking_points_count} Speaker Scripts
1590 |                                         </span>
1591 |                                     )}
1592 |                                     </div>
1593 |                                 );
1594 |                             })
1595 |                         ) : (
1596 |                             <div className="text-center py-5 border border-dashed
border-slate-900 rounded-xl text-slate-500 text-xs">
1597 |                                 No matching slides found for this outcome keyword signature.
1598 |                             </div>
1599 |                         )}
1600 |                     </div>
1601 |                 </div>
1602 |
1603 |                 {/* Related Assessments Column */}
1604 |                 <div className="flex flex-col gap-3">
1605 |                     <span className="text-[10px] font-extrabold text-slate-500
tracking-wider flex items-center gap-2">
1606 |                         <GraduationCap className="h-4 w-4 text-indigo-400" />
1607 |                         <span>COVERED BY QUIZ QUESTIONS</span>
1608 |                     </span>
1609 |
1610 |                     <div className="flex flex-col gap-2">
1611 |                         {outcome.related_assessments.length > 0 ? (
1612 |                             outcome.related_assessments.map((quiz: any, qIdx: number) => (
1613 |                                 <div
1614 |                                     key={qIdx}
1615 |                                     onClick={() => {
1616 |                                         setActiveTab("assessments");
1617 |                                     }}
1618 |                                     className="glass-panel p-3.5 rounded-xl border-white/5
hover:border-indigo-500/20 cursor-pointer transition-all text-left flex flex-col gap-1.5"
1619 |                                 >
1620 |                                     <div className="flex justify-between items-center gap-2">
1621 |                                         <span className="text-[9.5px] font-extrabold text-slate-500
block">QUESTION {qIdx + 1}</span>
1622 |                                         <span className="px-2 py-0.5 rounded-md bg-indigo-500/5
border border-indigo-500/10 text-[9px] font-bold text-indigo-400">
1623 |                                             {quiz.bloom_level}
1624 |                                         </span>
1625 |                                     </div>
1626 |                                     <p className="text-xs font-semibold text-slate-200 line-clamp-1
leading-relaxed">
1627 |                                         {quiz.question_text}
1628 |                                     </p>
1629 |                                 </div>
1630 |                             ))
1631 |                         ) : (
1632 |                             <div className="text-center py-5 border border-dashed
border-slate-900 rounded-xl text-slate-500 text-xs">
1633 |                                 No assessment bank questions linked to this learning outcome ID.
1634 |                             </div>
1635 |                         )}
1636 |                     </div>
1637 |                 </div>
1638 |
1639 |             </div>
1640 |
1641 |         </div>
1642 |     )}

```

```

1643 |         </div>
1644 |     );
1645 |     }}}
1646 | </div>
1647 |
1648 | </div>
1649 | }}
1650 |
1651 | {activeTab === "pipeline" && courseDetails && (
1652 |     <div className="max-w-4xl mx-auto flex flex-col gap-6 text-left">
1653 |
1654 |         <div className="flex flex-col gap-1.5">
1655 |             <h3 className="text-lg font-bold text-white">Multi-Agent Execution Pipeline</h3>
1656 |             <p className="text-xs text-slate-400">
1657 |                 Visual log trace showing chronological execution flow, operational purposes, and
1658 |                 completed timestamps for the KALYX multi-agent graph.
1659 |             </p>
1660 |         </div>
1661 |
1662 |         {/* Pipeline Summary Box */}
1663 |         {totalDuration > 0 && (
1664 |             <div className="bg-emerald-500/10 border border-emerald-500/20 rounded-2xl p-5
1665 |                 text-left flex flex-col gap-1.5 shadow-lg shadow-emerald-500/2">
1666 |                 <div className="text-[10px] font-bold text-emerald-400 uppercase
1667 |                     tracking-widest">9-Agent Workflow Completed</div>
1668 |                 <div className="text-lg font-black text-white">Total Execution Time:
1669 |                 {totalDuration.toFixed(2)} seconds</div>
1670 |             </div>
1671 |         )}
1672 |
1673 |         <div className="glass-panel p-6 md:p-8 rounded-2xl border-white/5 relative flex
1674 |             flex-col gap-8">
1675 |             {/* Vertical line connecting nodes */}
1676 |             <div className="absolute left-[35px] md:left-[43px] top-12 bottom-12 w-[2px]
1677 |                 bg-dashed border-l border-slate-900 border-dashed z-0" />
1678 |
1679 |             {pipelineAgents.map((agent, idx) => {
1680 |                 // Color schemes per status
1681 |                 let dotColor = "bg-slate-800 border-slate-700";
1682 |                 let statusText = "Pending";
1683 |                 let badgeStyle = "text-slate-400 border-slate-800/80 bg-slate-900/60";
1684 |
1685 |                 if (agent.status === "completed") {
1686 |                     dotColor = "bg-emerald-500 border-emerald-400/20
1687 |                         shadow-[0_0_12px_rgba(16,185,129,0.4)]";
1688 |                     statusText = "Completed";
1689 |                     badgeStyle = "text-emerald-400 border-emerald-500/20 bg-emerald-500/10";
1690 |                 } else if (agent.status === "running") {
1691 |                     dotColor = "bg-sky-500 border-sky-400/20 shadow-[0_0_12px_rgba(14,165,233,0.4)]
1692 |                         animate-pulse";
1693 |                     statusText = "Running";
1694 |                     badgeStyle = "text-sky-400 border-sky-500/20 bg-sky-500/10 animate-pulse";
1695 |                 } else if (agent.status === "failed") {
1696 |                     dotColor = "bg-rose-500 border-rose-400/20 shadow-[0_0_12px_rgba(244,63,94,0.4)]
1697 |                         animate-ping";
1698 |                     statusText = "Failed";
1699 |                     badgeStyle = "text-rose-400 border-rose-500/20 bg-rose-500/10";
1700 |                 }
1701 |
1702 |                 return (
1703 |                     <div key={idx} className="flex gap-4 md:gap-6 items-start relative z-10">
1704 |                         {/* Dot / Status symbol */}
1705 |                         <div className={`h-8 w-8 md:h-10 md:w-10 rounded-full border flex items-center
1706 |                             justify-center shrink-0 text-white font-bold text-xs ${dotColor} transition-all duration-300`}
1707 |                             {agent.status === "completed" ? (
1708 |                                 <CheckCircle2 className="h-4.5 w-4.5 md:h-5 md:w-5 text-white" />

```

```

1699 |         ) : agent.status === "running" ? (
1700 |             <div className="h-2 w-2 rounded-full bg-white animate-ping" />
1701 |         ) : agent.status === "failed" ? (
1702 |             <AlertCircle className="h-4.5 w-4.5 md:h-5 md:w-5 text-white" />
1703 |         ) : (
1704 |             <span className="text-[10px] text-slate-500">{idx + 1}</span>
1705 |         )}
1706 |     </div>
1707 |
1708 |     {/* Agent Card Details */}
1709 |     <div className="flex-1 glass-panel p-4 rounded-xl border-white/5
hover:border-white/10 transition-all flex flex-col md:flex-row md:items-center justify-between gap-4">
1710 |         <div className="flex-1 text-left">
1711 |             <h4 className="text-xs font-bold text-white flex items-center gap-2">
1712 |                 <span>{agent.name}</span>
1713 |             </h4>
1714 |             <p className="text-[11px] text-slate-400 leading-relaxed
mt-1">{agent.purpose}</p>
1715 |         </div>
1716 |
1717 |         <div className="flex items-center justify-between md:justify-end gap-4
shrink-0 border-t border-slate-900 md:border-t-0 pt-2.5 md:pt-0">
1718 |             {agent.timestamp && (
1719 |                 <span className="text-[10px] font-mono text-slate-500 font-bold">
1720 |                     {agent.timestamp}
1721 |                 </span>
1722 |             )}
1723 |             <span className={`px-2.5 py-0.5 rounded-full border text-[9px] font-black
uppercase tracking-wider ${badgeStyle}`}>
1724 |                 {statusText}
1725 |             </span>
1726 |         </div>
1727 |     </div>
1728 | </div>
1729 | );
1730 | }}}
1731 | </div>
1732 |
1733 | </div>
1734 | })
1735 |
1736 | {/* SLIDES PREVIEW TAB (AI STUDIO) */}
1737 | {activeTab === "slides" && courseDetails && (
1738 |     <div className="w-full max-w-[1440px] mx-auto flex flex-col gap-4 h-[calc(100vh-12rem)]">
1739 |
1740 |         <div className="flex-1 flex flex-col lg:flex-row gap-5 overflow-hidden">
1741 |
1742 |             {/* Left Thumbnails selection panel */}
1743 |             <div className="w-full lg:w-56 shrink-0 flex flex-row lg:flex-col gap-3
overflow-x-auto lg:overflow-y-auto pb-2 lg:pb-0 pr-1 border-b lg:border-b-0 lg:border-r border-slate-900/60
lg:pr-3">
1744 |                 {courseDetails.slides.map((s, idx) => (
1745 |                     <div
1746 |                         key={s.id}
1747 |                         onClick={() => loadSlideEditState(courseDetails, idx)}
1748 |                         className={`p-3 rounded-xl border cursor-pointer flex flex-col gap-1.5
text-left transition-all shrink-0 w-44 lg:w-full ${
1749 |                             activeSlideIndex === idx
1750 |                             ? "bg-sky-500/10 border-sky-500/40 text-white shadow-lg"
1751 |                             : "bg-slate-900/40 border-slate-900 hover:bg-slate-900 text-slate-300"
1752 |                         }`}
1753 |                     >
1754 |                         <span className="text-[9px] font-black text-sky-400">SLIDE
{s.slide_index}</span>
1755 |                         <span className="text-xs font-extrabold truncate">{s.title}</span>
1756 |                         <span className="text-[10px] text-slate-500 truncate leading-none">

```

```

1757 |         {s.content.length} bullet points
1758 |     </span>
1759 | </div>
1760 |     )}}
1761 | </div>
1762 |
1763 |     {/* Middle Workspace Slide Canvas Editor */}
1764 |     <div className="flex-1 flex flex-col gap-4 overflow-y-auto pr-1 min-w-0">
1765 |
1766 |         {/* Interactive slide canvas mock */}
1767 |         <div className="aspect-[16/9] max-h-[360px] min-h-[300px] w-full max-w-[640px]
mx-auto rounded-2xl bg-[#0a0f1e] border border-slate-800 p-6 flex flex-col justify-between relative shadow-2xl
shrink-0">
1768 |
1769 |             {/* Title bar decoration */}
1770 |             <div className="flex justify-between items-center text-[10px] font-mono
text-sky-400">
1771 |                 <span>[ KALYX PRESENTATION SYSTEM ]</span>
1772 |                 <span>SLIDE {activeSlideIndex + 1} OF {courseDetails.slides.length}</span>
1773 |             </div>
1774 |
1775 |             {/* Title editable preview with edit indicator */}
1776 |             <div className="flex items-center gap-2 border-b border-transparent
hover:border-slate-800 focus-within:border-sky-500/40 transition-all pb-1 group/title">
1777 |                 <input
1778 |                     type="text"
1779 |                     value={editSlideTitle}
1780 |                     onChange={(e) => setEditSlideTitle(e.target.value)}
1781 |                     className="bg-transparent text-white font-black text-lg md:text-xl
lg:text-2xl focus:outline-none flex-1 leading-tight tracking-tight"
1782 |                     title="Click to edit slide title"
1783 |                 />
1784 |                 <Pencil className="h-4 w-4 text-slate-500 group-hover/title:text-sky-400
transition-colors shrink-0" />
1785 |             </div>
1786 |
1787 |             {/* Bullet items container with hover edit indicators */}
1788 |             <div className="flex flex-col gap-1.5 py-2 overflow-y-auto max-h-[140px]">
1789 |                 {editSlideBullets.map((bullet, bIdx) => (
1790 |                     <div key={bIdx} className="flex gap-3 items-center group/bullet w-full
border-b border-transparent hover:border-slate-900 focus-within:border-sky-500/40 transition-all pb-0.5">
1791 |                         <span className="text-sky-400 font-bold shrink-0">-</span>
1792 |                         <input
1793 |                             type="text"
1794 |                             value={bullet}
1795 |                             onChange={(e) => {
1796 |                                 const updated = [...editSlideBullets];
1797 |                                 updated[bIdx] = e.target.value;
1798 |                                 setEditSlideBullets(updated);
1799 |                             }}
1800 |                             className="bg-transparent text-slate-200 text-xs focus:outline-none
flex-1"
1801 |                             title="Click to edit bullet point"
1802 |                         />
1803 |                         <Pencil className="h-3.5 w-3.5 text-slate-600 opacity-0
group-hover/bullet:opacity-100 focus-within:opacity-100 transition-opacity shrink-0" />
1804 |                     </div>
1805 |                 )}}
1806 |             </div>
1807 |
1808 |             {/* Visual overlay indicator */}
1809 |             <div className="flex justify-between items-center text-[10px] text-slate-500
font-mono border-t border-slate-900/60 pt-2.5">
1810 |                 <span className="text-sky-400 font-semibold">[ Bloom Level:
{courseDetails.assessments[activeSlideIndex]?.bloom_level || "Applying"} ]</span>
1811 |                 <span>AI Visual: {editSlideVisuals ? editSlideVisuals.substring(0, 30) + "..."
```

```

: "No visuals"]</span>
1812 |         </div>
1813 |
1814 |     </div>
1815 |
1816 |     {/ Visual Suggestions Box editable */}
1817 |     <div className="glass-panel p-4 rounded-xl border-white/5 text-left flex flex-col
gap-2.5 max-w-[640px] mx-auto w-full">
1818 |         <div className="flex justify-between items-center">
1819 |             <span className="text-[10px] font-extrabold text-sky-400 tracking-wider">AI
VISUAL SUGGESTIONS</span>
1820 |             <div className="flex items-center gap-1.5 text-slate-500 text-[10px]">
1821 |                 <span>Click to edit</span>
1822 |                 <Pencil className="h-3 w-3" />
1823 |             </div>
1824 |         </div>
1825 |         <textarea
1826 |             rows={2}
1827 |             value={editSlideVisuals}
1828 |             onChange={(e) => setEditSlideVisuals(e.target.value)}
1829 |             className="w-full bg-slate-950 border border-slate-900 rounded-xl p-2.5 text-xs
text-slate-300 focus:border-sky-500/40 focus:outline-none focus:bg-slate-950/80 transition-all resize-none"
1830 |             placeholder="Describe the conceptual background graphic layouts..."
1831 |         />
1832 |     </div>
1833 |
1834 |     {/ Compact actions row containing Save and Download buttons */}
1835 |     <div className="flex flex-col sm:flex-row gap-3 items-center justify-between
bg-slate-900/40 border border-slate-900/60 rounded-xl p-4 shrink-0 max-w-[640px] mx-auto w-full">
1836 |         <div className="flex items-center gap-2">
1837 |             <Save className="h-4 w-4 text-sky-400" />
1838 |             <span className="text-xs text-slate-300 font-bold">Slide Actions</span>
1839 |         </div>
1840 |
1841 |         <div className="flex flex-wrap gap-2 items-center">
1842 |             <button
1843 |                 onClick={handleSaveSlideChanges}
1844 |                 className="py-1.5 px-3 bg-gradient-to-r from-sky-500 to-indigo-500
hover:from-sky-400 hover:to-indigo-400 rounded-lg text-xs font-bold shadow-md shadow-sky-500/10 active:scale-95
transition-all flex items-center gap-1.5 text-white"
1845 |             >
1846 |                 <Save className="h-3.5 w-3.5" />
1847 |                 <span>Save Changes</span>
1848 |             </button>
1849 |
1850 |             <button
1851 |                 onClick={handleDownloadPptx}
1852 |                 className="py-1.5 px-3 bg-slate-950 border border-slate-800 text-xs font-bold
rounded-lg hover:bg-slate-900 hover:border-slate-700 active:scale-95 transition-all flex items-center gap-1.5
text-slate-200"
1853 |             >
1854 |                 <Download className="h-3.5 w-3.5 text-sky-400" />
1855 |                 <span>Download PPTX</span>
1856 |             </button>
1857 |
1858 |             <button
1859 |                 onClick={handleDownloadPdf}
1860 |                 className="py-1.5 px-3 bg-slate-950 border border-slate-800 text-xs font-bold
rounded-lg hover:bg-slate-900 hover:border-slate-700 active:scale-95 transition-all flex items-center gap-1.5
text-slate-200"
1861 |             >
1862 |                 <Download className="h-3.5 w-3.5 text-indigo-400" />
1863 |                 <span>Download PDF</span>
1864 |             </button>
1865 |         </div>
1866 |     </div>

```



```

1867 |
1868 |         </div>
1869 |
1870 |     </div>
1871 |
1872 | </div>
1873 | }}
1874 |
1875 | {/* INSTRUCTOR NOTES TAB */}
1876 | {activeTab === "notes" && courseDetails && (
1877 |     <div className="max-w-4xl mx-auto flex flex-col gap-6 text-left">
1878 |
1879 |         {/* Title card selector indicators */}
1880 |         <div className="flex justify-between items-center">
1881 |             <div>
1882 |                 <h3 className="text-lg font-bold text-white">Lecture Instructor Notes & Script</h3>
1883 |                 <p className="text-xs text-slate-400 mt-1">Talking points, clarifications, and
teaching tips compiled for slide: {courseDetails.slides[activeSlideIndex]?.title}</p>
1884 |             </div>
1885 |
1886 |             <div className="flex gap-1.5">
1887 |                 <button
1888 |                     disabled={activeSlideIndex === 0}
1889 |                     onClick={() => loadSlideEditState(courseDetails, activeSlideIndex - 1)}
1890 |                     className="p-1.5 bg-slate-900 border border-slate-800 rounded hover:bg-slate-800
disabled:opacity-40 disabled:cursor-not-allowed"
1891 |                 >
1892 |                     <ChevronLeft className="h-4 w-4" />
1893 |                 </button>
1894 |                 <span className="px-3 py-1 bg-slate-900 border border-slate-800 rounded text-xs
font-mono text-slate-300">
1895 |                     {activeSlideIndex + 1} / {courseDetails.slides.length}
1896 |                 </span>
1897 |                 <button
1898 |                     disabled={activeSlideIndex === courseDetails.slides.length - 1}
1899 |                     onClick={() => loadSlideEditState(courseDetails, activeSlideIndex + 1)}
1900 |                     className="p-1.5 bg-slate-900 border border-slate-800 rounded hover:bg-slate-800
disabled:opacity-40 disabled:cursor-not-allowed"
1901 |                 >
1902 |                     <ChevronRight className="h-4 w-4" />
1903 |                 </button>
1904 |             </div>
1905 |         </div>
1906 |
1907 |         {/* Main script details */}
1908 |         <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
1909 |
1910 |             {/* Notes talking points box list */}
1911 |             <div className="md:col-span-2 flex flex-col gap-6">
1912 |
1913 |                 <div className="glass-panel p-6 rounded-2xl border-white/5 flex flex-col gap-4">
1914 |                     <span className="text-[10px] font-extrabold text-sky-400 tracking-wider flex
items-center gap-1.5">
1915 |                         <Volume2 className="h-4 w-4" />
1916 |                         <span>LECTURING TALKING POINTS</span>
1917 |                     </span>
1918 |
1919 |                     <div className="flex flex-col gap-3">
1920 |                         {editNoteTalkingPoints.map((point, ptIdx) => (
1921 |                             <div key={ptIdx} className="flex gap-3 items-start border-b
border-slate-900/60 pb-3">
1922 |                                 <span className="w-5 h-5 rounded-full bg-sky-500/10 text-sky-400 font-mono
text-[10px] flex items-center justify-center shrink-0 mt-0.5">
1923 |                                     {ptIdx + 1}
1924 |                                 </span>
1925 |                                 <textarea

```

```

1926 |         value={point}
1927 |         onChange={(e) => {
1928 |             const updated = [...editNoteTalkingPoints];
1929 |             updated[ptIdx] = e.target.value;
1930 |             setEditNoteTalkingPoints(updated);
1931 |         }}
1932 |         rows={2}
1933 |         className="bg-transparent text-slate-200 text-xs md:text-sm border-b
border-transparent focus:border-sky-500/40 focus:outline-none flex-1 focus:bg-slate-950/40 p-1.5 rounded"
1934 |         />
1935 |     </div>
1936 |     )}}
1937 | </div>
1938 | </div>
1939 |
1940 |     {/* Editable Clarifying Examples */}
1941 |     <div className="glass-panel p-6 rounded-2xl border-white/5 flex flex-col gap-4">
1942 |         <span className="text-[10px] font-extrabold text-indigo-400 tracking-wider flex
items-center gap-1.5">
1943 |             <Lightbulb className="h-4 w-4" />
1944 |             <span>CLARIFYING EXAMPLES</span>
1945 |         </span>
1946 |
1947 |         <div className="flex flex-col gap-3">
1948 |             {editNoteExamples.map((ex, exIdx) => (
1949 |                 <div key={exIdx} className="flex gap-2.5 items-center">
1950 |                     <span className="text-[10px] text-slate-500 font-bold font-mono">EX:</span>
1951 |                     <input
1952 |                         type="text"
1953 |                         value={ex}
1954 |                         onChange={(e) => {
1955 |                             const updated = [...editNoteExamples];
1956 |                             updated[exIdx] = e.target.value;
1957 |                             setEditNoteExamples(updated);
1958 |                         }}
1959 |                         className="bg-transparent text-slate-300 text-xs border-b
border-transparent focus:border-sky-500/40 focus:outline-none flex-1 pb-1"
1960 |                     />
1961 |                 </div>
1962 |             )}}
1963 |         </div>
1964 |     </div>
1965 |
1966 | </div>
1967 |
1968 |     {/* Pedagogical Tips Sidebar info */}
1969 |     <div className="flex flex-col gap-6">
1970 |
1971 |         <div className="glass-panel p-6 rounded-2xl border-sky-500/20 bg-sky-500/5 flex
flex-col gap-4">
1972 |             <span className="text-[10px] font-extrabold text-sky-400 tracking-wider flex
items-center gap-1.5">
1973 |                 <GraduationCap className="h-4.5 w-4.5" />
1974 |                 <span>PEDAGOGICAL TEACHING TIP</span>
1975 |             </span>
1976 |
1977 |             <textarea
1978 |                 rows={6}
1979 |                 value={editNoteTips}
1980 |                 onChange={(e) => setEditNoteTips(e.target.value)}
1981 |                 className="bg-slate-950 border border-slate-900 rounded-xl p-3 text-xs
text-slate-200 focus:outline-none focus:border-sky-500/40"
1982 |             />
1983 |         </div>
1984 |
1985 |         <button

```

```

1986 |         onClick={handleSaveSlideChanges}
1987 |         className="w-full py-3 bg-gradient-to-r from-sky-500 to-indigo-500
hover:from-sky-400 hover:to-indigo-400 rounded-xl text-xs font-bold flex items-center justify-center gap-2"
1988 |         >
1989 |         <Save className="h-4 w-4" />
1990 |         <span>Save Speaker Notes</span>
1991 |     </button>
1992 |
1993 |     </div>
1994 |
1995 | </div>
1996 |
1997 | </div>
1998 | )}
1999 |
2000 | {/* ASSESSMENT BANK TAB */}
2001 | {activeTab === "assessments" && courseDetails && (
2002 |     <div className="max-w-4xl mx-auto flex flex-col gap-6 text-left">
2003 |
2004 |         <div className="flex flex-col md:flex-row md:items-center md:justify-between gap-4
border-b border-white/5 pb-4">
2005 |             <div>
2006 |                 <h3 className="text-lg font-bold text-white">Pedagogical Assessment & Quiz
Bank</h3>
2007 |                 <p className="text-xs text-slate-400 mt-1">Multiple-choice questions (MCQs)
generated by KALYX, mapped to distinct learning outcomes and Bloom levels.</p>
2008 |             </div>
2009 |             <div className="flex flex-row gap-2 flex-wrap">
2010 |                 <button
2011 |                     onClick={handleDownloadInteractiveQuiz}
2012 |                     className="px-4 py-2.5 bg-gradient-to-r from-sky-500 to-indigo-500
hover:from-sky-400 hover:to-indigo-400 text-white rounded-xl text-xs font-bold flex items-center gap-1.5
shadow-md shadow-indigo-500/10 hover:shadow-indigo-500/20 transition-all cursor-pointer animate-pulse-light"
2013 |                     >
2014 |                         <Gamepad2 className="h-4 w-4" />
2015 |                         <span>Interactive Test (HTML)</span>
2016 |                     </button>
2017 |                 </div>
2018 |             </div>
2019 |
2020 |             <div className="flex flex-col gap-4">
2021 |                 {courseDetails.assessments.filter(a => a.question_type.toUpperCase() === "MCQ" ||
(a.options && a.options.length > 0)).map((a, idx) => (
2022 |                     <div key={a.id || idx} className="glass-panel p-6 rounded-2xl border-white/5
relative hover:border-sky-500/20 transition-all flex flex-col gap-3">
2023 |
2024 |                         {/* Upper tagging bar */}
2025 |                         <div className="flex justify-between items-center text-[10px]">
2026 |                             <span className="px-2.5 py-0.5 rounded bg-slate-900 border border-slate-800
text-slate-400 font-mono">
2027 |                                 {a.question_type} QUESTION
2028 |                             </span>
2029 |
2030 |                             <div className="flex items-center gap-3">
2031 |                                 <span className="text-slate-500">Bloom level:</span>
2032 |                                 <span className="px-2 py-0.5 rounded bg-indigo-500/10 border
border-indigo-500/20 text-indigo-400 font-black">
2033 |                                     {a.bloom_level}
2034 |                                 </span>
2035 |                             </div>
2036 |                         </div>
2037 |
2038 |                         {/* Question Text */}
2039 |                         <h4 className="text-sm font-bold text-white
leading-relaxed">{a.question_text}</h4>
2040 |

```

```

2041 |             {/* Options if MCQ */}
2042 |             {a.options && (
2043 |                 <div className="grid grid-cols-1 md:grid-cols-2 gap-2 my-2">
2044 |                     {a.options.map((opt, oIdx) => (
2045 |                         <div key={oIdx} className="px-4 py-2 bg-slate-950 border
border-slate-900/60 rounded-xl text-xs text-slate-300">
2046 |                             {opt}
2047 |                         </div>
2048 |                     ))}
2049 |                 </div>
2050 |             )}
2051 |
2052 |             {/* Correct Answer */}
2053 |             <div className="p-3 bg-emerald-500/5 border border-emerald-500/10 rounded-xl
text-xs text-emerald-400 font-mono">
2054 |                 <span className="font-extrabold mr-1">Correct Solution:</span>
2055 |                 {a.correct_answer}
2056 |             </div>
2057 |
2058 |         </div>
2059 |     )}}
2060 | </div>
2061 |
2062 | </div>
2063 | })
2064 |
2065 | {/* BLOOM TAXONOMY AUDIT TAB */}
2066 | {activeTab === "bloom" && courseDetails && (
2067 |     <div className="max-w-4xl mx-auto flex flex-col gap-6 text-left">
2068 |
2069 |         <div>
2070 |             <h3 className="text-lg font-bold text-white">Bloom&apos;s Taxonomy Coverage
Analysis</h3>
2071 |             <p className="text-xs text-slate-400 mt-1">Audited distribution of course materials
across the six tiers of modern cognitive psychology.</p>
2072 |             </div>
2073 |
2074 |             {/* Aesthetic custom grid representing Bloom Tiers */}
2075 |             <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
2076 |                 {[
2077 |                     { tier: "Remembering", val: 100, desc: "Recalling basic facts, definitions,
algorithms, and core formulas.", color: "text-rose-400 border-rose-500/20 bg-rose-500/5" },
2078 |                     { tier: "Understanding", val: 100, desc: "Explaining principles, translating
models, classifying target nodes.", color: "text-amber-400 border-amber-500/20 bg-amber-500/5" },
2079 |                     { tier: "Applying", val: 85, desc: "Implementing regression, training ML algorithms
on tabular datasets.", color: "text-emerald-400 border-emerald-500/20 bg-emerald-500/5" },
2080 |                     { tier: "Analyzing", val: 90, desc: "Deconstructing outputs, diagnosing
overfitting, comparing error boundaries.", color: "text-sky-400 border-sky-500/20 bg-sky-500/5" },
2081 |                     { tier: "Evaluating", val: 92, desc: "Justifying neural network selection over
classical tree models.", color: "text-indigo-400 border-indigo-500/20 bg-indigo-500/5" },
2082 |                     { tier: "Creating", val: 80, desc: "Assembling ML feature pipelines, designing
modern data loaders.", color: "text-violet-400 border-violet-500/20 bg-violet-500/5" }
2083 |                 ].map((tier, idx) => (
2084 |                     <div key={idx} className={`glass-panel p-5 rounded-2xl border ${tier.color} flex
flex-col justify-between min-h-[160px] hover:scale-[1.02] transition-all`}>
2085 |                         <div className="flex justify-between items-center">
2086 |                             <span className="text-xs font-black tracking-wider
uppercase">{tier.tier}</span>
2087 |                             <span className="px-2.5 py-0.5 rounded-full bg-white/5 border border-white/10
text-xs font-extrabold">{tier.val}% Cover</span>
2088 |                         </div>
2089 |                         <p className="text-xs text-slate-300 leading-relaxed my-3">{tier.desc}</p>
2090 |                     {/* Dynamic Progress indicator */}
2091 |                     <div className="w-full bg-slate-950/60 rounded-full h-1.5 overflow-hidden">
2092 |                         <div
2093 |                             className="bg-current h-1.5 rounded-full"

```

```

2094 |         style={{ width: `${tier.val}%` }}
2095 |         />
2096 |     </div>
2097 | </div>
2098 |     )}}
2099 | </div>
2100 |
2101 |     {/* Recommendations */}
2102 |     <div className="glass-panel p-6 rounded-2xl border-white/5 flex gap-4 items-start">
2103 |         <Award className="h-10 w-10 text-sky-400 shrink-0 mt-0.5" />
2104 |         <div>
2105 |             <h4 className="text-sm font-bold text-white">KALYX Pedagogical Audit
Assessment</h4>
2106 |                 <p className="text-xs text-slate-300 leading-relaxed mt-1.5">
2107 |                     {courseDetails.curriculum_analysis?.gap_analysis || "The syllabus is highly
balanced. Consider adding one more design challenge to push 'Creating' coverage to 90%."}
2108 |                 </p>
2109 |             </div>
2110 |         </div>
2111 |
2112 |     </div>
2113 | )}
2114 |
2115 | {/* READINESS SCORE TAB */}
2116 | {activeTab === "readiness" && courseDetails && (
2117 |     <div className="max-w-4xl mx-auto flex flex-col gap-8 text-left">
2118 |
2119 |         <div>
2120 |             <h3 className="text-lg font-bold text-white">Course Readiness & Quality
Breakdown</h3>
2121 |                 <p className="text-xs text-slate-400 mt-1">Mathematical calculation representing
overall course package quality across multiple core vectors.</p>
2122 |             </div>
2123 |
2124 |             <div className="grid grid-cols-1 md:grid-cols-3 gap-6 items-center">
2125 |
2126 |                 {/* Big central score dial SVG */}
2127 |                 <button
2128 |                     onClick={() => setIsReadinessExpanded(!isReadinessExpanded)}
2129 |                     className={`glass-panel p-8 rounded-2xl border-white/5 flex flex-col items-center
justify-center min-h-[300px] text-center transition-all duration-300 hover:border-sky-500/30 hover:scale-[1.02]
active:scale-[0.98] cursor-pointer w-full group ${
2130 |                         isReadinessExpanded ? "ring-2 ring-sky-500/20 border-sky-500/30" : ""
2131 |                     }`}
2132 |                 >
2133 |                     <span className="text-[10px] font-extrabold text-slate-500 tracking-wider mb-4
group-hover:text-sky-400 transition-colors">
2134 |                         {isReadinessExpanded ? "CLOSE DETAILED ANALYSIS" : "COMPREHENSIVE RATING"}
2135 |                     </span>
2136 |
2137 |                     {/* Circular meter */}
2138 |                     <div className="relative w-44 h-44 flex items-center justify-center mb-4 transform
transition-transform group-hover:scale-105 duration-300">
2139 |                         <svg className="absolute inset-0 w-full h-full transform -rotate-90">
2140 |                             <circle
2141 |                                 cx="88" cy="88" r="74"
2142 |                                 className="stroke-slate-900 fill-none"
2143 |                                 strokeWidth="10"
2144 |                             />
2145 |                             <circle
2146 |                                 cx="88" cy="88" r="74"
2147 |                                 className="stroke-sky-400 fill-none filter
drop-shadow-[0_0_8px_rgba(14,165,233,0.3)] transition-all duration-500"
2148 |                                 strokeWidth="10"
2149 |                                 strokeDasharray="465"
2150 |                                 strokeDashoffset={465 - (465 * (courseDetails.readiness_score?.score || 89))}

```

```

/ 100}
2151 |             />
2152 |         </svg>
2153 |         <div className="flex flex-col items-center">
2154 |             <span className="text-4xl font-black text-white group-hover:text-sky-400
transition-colors">
2155 |                 {courseDetails.readiness_score?.score.toFixed(0)}
2156 |             </span>
2157 |             <span className="text-[10px] font-bold text-slate-500">POINTS OUT OF 100</span>
2158 |         </div>
2159 |     </div>
2160 |
2161 |     <span className="text-xs font-black text-sky-400 group-hover:text-white
transition-colors">
2162 |         {isReadinessExpanded ? "HIDE BREAKDOWN" : "CLICK FOR DETAILED CRITIQUE"}
2163 |     </span>
2164 | </button>
2165 |
2166 |     {/* Right side component breakdown logs */}
2167 |     <div className="md:col-span-2 flex flex-col gap-3">
2168 |         {[
2169 |             { label: "Learning Outcome Coverage (25% Weight)", val:
courseDetails.readiness_score?.outcome_coverage || 92, scoreStr: "Perfect mappings across 6 outcome blocks" },
2170 |             { label: "Bloom Taxonomy Coverage (20% Weight)", val:
courseDetails.readiness_score?.bloom_coverage || 91, scoreStr: "Robust distribution across remembering & design"
},
2171 |             { label: "Assessment Mapping Quality (20% Weight)", val:
courseDetails.readiness_score?.assessment_quality || 90, scoreStr: "Quizzes completely match objectives" },
2172 |             { label: "Content Completeness Index (20% Weight)", val:
courseDetails.readiness_score?.completeness || 88, scoreStr: "Core modules sequenced and slide decks formatted"
},
2173 |             { label: "Modern Industry Relevance (15% Weight)", val:
courseDetails.readiness_score?.industry_relevance || 82, scoreStr: "Lacks modern deep learning transformers" }
2174 |         ].map((item, idx) => (
2175 |             <div key={idx} className="glass-panel p-4 rounded-xl border-white/5 flex
justify-between items-center">
2176 |                 <div className="flex-1 text-left">
2177 |                     <h4 className="text-xs font-bold text-white">{item.label}</h4>
2178 |                     <span className="text-[10px] text-slate-400 block
mt-0.5">{item.scoreStr}</span>
2179 |                 </div>
2180 |                 <div className="flex items-center gap-4">
2181 |                     <span className="text-sm font-black
text-sky-400">{item.val.toFixed(0)}%</span>
2182 |                     {/* Bar mini indicator */}
2183 |                     <div className="w-12 bg-slate-900 border border-slate-800 h-2 rounded
overflow-hidden">
2184 |                         <div className="bg-sky-400 h-full rounded animate-pulse-glow" style={{
width: `${item.val}%` }} />
2185 |                     </div>
2186 |                 </div>
2187 |             </div>
2188 |         )]}
2189 |     </div>
2190 |
2191 | </div>
2192 |
2193 | {isReadinessExpanded && (
2194 |     <div className="mt-4 flex flex-col gap-6 animate-fadeIn">
2195 |         <div className="border-t border-slate-900 pt-6">
2196 |             <h4 className="text-sm font-bold text-white mb-1">Pedagogical Critique &
Recommendations</h4>
2197 |             <p className="text-xs text-slate-400">Detailed qualitative analysis compiled by
the KALYX Accreditation Lead Reviewer agent.</p>
2198 |         </div>
2199 |

```

```

2200 |         <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
2201 |             {[
2202 |                 {
2203 |                     key: "completeness",
2204 |                     label: "Curriculum Completeness",
2205 |                     val: courseDetails.readiness_score?.completeness || 85
2206 |                 },
2207 |                 {
2208 |                     key: "assessment",
2209 |                     label: "Assessment Quality",
2210 |                     val: courseDetails.readiness_score?.assessment_quality || 72
2211 |                 },
2212 |                 {
2213 |                     key: "bloom",
2214 |                     label: "Bloom Coverage",
2215 |                     val: courseDetails.readiness_score?.bloom_coverage || 54
2216 |                 },
2217 |                 {
2218 |                     key: "relevance",
2219 |                     label: "Industry Relevance",
2220 |                     val: courseDetails.readiness_score?.industry_relevance || 61
2221 |                 }
2222 |             ]}.map((metric) => {
2223 |                 const { reason, recommendation } = parseCritique(
2224 |                     courseDetails.readiness_score?.breakdown,
2225 |                     metric.key,
2226 |                     metric.val
2227 |                 );
2228 |
2229 |                 let barColor = "bg-rose-500";
2230 |                 let textClass = "text-rose-400 border-rose-500/20 bg-rose-500/10";
2231 |                 let statusText = "Needs Improvement";
2232 |                 if (metric.val >= 90) {
2233 |                     barColor = "bg-emerald-500";
2234 |                     textClass = "text-emerald-400 border-emerald-500/20 bg-emerald-500/10";
2235 |                     statusText = "Excellent";
2236 |                 } else if (metric.val >= 70) {
2237 |                     barColor = "bg-amber-500";
2238 |                     textClass = "text-amber-400 border-amber-500/20 bg-amber-500/10";
2239 |                     statusText = "Good";
2240 |                 }
2241 |
2242 |                 return (
2243 |                     <div key={metric.key} className="glass-panel p-5 rounded-2xl border-white/5
flex flex-col gap-4">
2244 |                         <div className="flex justify-between items-start">
2245 |                             <div>
2246 |                                 <h5 className="text-xs font-extrabold text-white uppercase
tracking-wider">{metric.label}</h5>
2247 |                                 <span className="text-[10px] text-slate-500 font-semibold block
mt-0.5">Qualitative Score Metric</span>
2248 |                             </div>
2249 |                             <div className="flex items-center gap-2">
2250 |                                 <span className="text-sm font-black
text-white">{metric.val.toFixed(0)}/100</span>
2251 |                                 <span className={`px-2 py-0.5 rounded-full border text-[9px] font-black
${textClass}`}>
2252 |                                     {statusText}
2253 |                                 </span>
2254 |                             </div>
2255 |                         </div>
2256 |
2257 |                         <div className="w-full bg-slate-900 border border-slate-800/80 h-2 rounded
overflow-hidden">
2258 |                             <div className={` ${barColor} h-full rounded transition-all duration-500`}
style={{ width: `${metric.val}%` }} />

```

```

2259 |         </div>
2260 |
2261 |         <div className="flex flex-col gap-2.5 text-xs bg-slate-950/40 p-3.5 border
border-slate-900/60 rounded-xl">
2262 |             <div>
2263 |                 <span className="text-[9.5px] font-extrabold text-slate-500 uppercase
tracking-wider block mb-1">Pedagogical Assessment</span>
2264 |                 <p className="text-slate-300 leading-relaxed">{reason}</p>
2265 |             </div>
2266 |             <div className="border-t border-slate-900/60 pt-2.5">
2267 |                 <span className="text-[9.5px] font-extrabold text-sky-400 uppercase
tracking-wider block mb-1">Improvement Action</span>
2268 |                 <p className="text-slate-300 leading-relaxed
font-semibold">{recommendation}</p>
2269 |             </div>
2270 |         </div>
2271 |     </div>
2272 |     );
2273 |   }}}}
2274 | </div>
2275 | </div>
2276 |   })
2277 |
2278 | </div>
2279 | })
2280 |
2281 | {/* INDUSTRY GAP ANALYZER TAB */}
2282 | {activeTab === "gaps" && courseDetails && (
2283 |   <div className="max-w-4xl mx-auto flex flex-col gap-6 text-left">
2284 |
2285 |     <div>
2286 |       <h3 className="text-lg font-bold text-white">Modern Industry Gap Analyzer</h3>
2287 |       <p className="text-xs text-slate-400 mt-1">Checks course topics against the cutting
edge 2026 technical landscape (Transformers, Vector databases, LLM agents).</p>
2288 |     </div>
2289 |
2290 |     <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
2291 |
2292 |       {/* Detected Gaps Box */}
2293 |       <div className="glass-panel p-6 rounded-2xl border-rose-500/20 bg-rose-500/5 flex
flex-col gap-4">
2294 |         <span className="text-[10px] font-extrabold text-rose-400 tracking-wider flex
items-center gap-2">
2295 |           <AlertCircle className="h-4.5 w-4.5" />
2296 |           <span>DETECTED MODERN OUTDATED CONCEPTS</span>
2297 |         </span>
2298 |
2299 |         <div className="flex flex-col gap-3 my-2">
2300 |           {courseDetails.curriculum_analysis?.industry_gap_report?.missing_topics?.map((t,
idx) => (
2301 |             <div key={idx} className="px-4 py-2.5 bg-slate-950 border border-rose-500/10
rounded-xl text-xs text-slate-300 font-mono">
2302 |               - Missing Topic: {t}
2303 |             </div>
2304 |           )}}
2305 |           {(!courseDetails.curriculum_analysis?.industry_gap_report?.missing_topics ||
courseDetails.curriculum_analysis?.industry_gap_report.missing_topics.length
=== 0) && (
2307 |             <div className="text-xs text-slate-500 py-6 text-center">
2308 |               No major technology gaps detected! Syllabus is state-of-the-art.
2309 |             </div>
2310 |           )}
2311 |         </div>
2312 |       </div>
2313 |
2314 |       {/* AI Recommendations box */}

```



```

2315 |         <div className="glass-panel p-6 rounded-2xl border-white/5 flex flex-col gap-4">
2316 |             <span className="text-[10px] font-extrabold text-sky-400 tracking-wider flex
items-center gap-2">
2317 |                 <Sparkles className="h-4.5 w-4.5" />
2318 |                 <span>AI MODERNIZATION RECOMMENDATIONS</span>
2319 |             </span>
2320 |
2321 |             <div className="flex flex-col gap-3.5 my-2">
2322 |                 {courseDetails.curriculum_analysis?.industry_gap_report?.recommendations?.map((r,
idx) => (
2323 |                     <div key={idx} className="flex gap-3 text-xs leading-relaxed text-slate-200">
2324 |                         <span className="text-sky-400 shrink-0 font-bold">{idx + 1}</span>
2325 |                         <span>{r}</span>
2326 |                     </div>
2327 |                 )]}
2328 |             </div>
2329 |         </div>
2330 |
2331 |     </div>
2332 |
2333 | </div>
2334 | )}
2335 |
2336 | {/* EXPORT CENTER TAB */}
2337 | {activeTab === "export" && courseDetails && (
2338 |     <div className="max-w-2xl mx-auto flex flex-col gap-8 text-center py-12">
2339 |
2340 |         <div className="flex flex-col gap-2">
2341 |             <h3 className="text-xl font-bold text-white">Export & Download Classroom Assets</h3>
2342 |             <p className="text-xs text-slate-400 leading-relaxed max-w-lg mx-auto">
2343 |                 Compile your dynamic slide adjustments, speaker talking scripts, and quizzes
directly into high-fidelity downloadable presentation and documentation files.
2344 |             </p>
2345 |         </div>
2346 |
2347 |         <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
2348 |
2349 |             {/* PPTX Export Button Card */}
2350 |             <div className="glass-panel p-8 rounded-2xl border-white/5 hover:border-sky-500/20
hover:scale-[1.02] transition-all flex flex-col items-center gap-5 justify-between">
2351 |                 <div className="p-4 bg-sky-500/10 rounded-full border border-sky-500/20">
2352 |                     <Sparkles className="h-8 w-8 text-sky-400" />
2353 |                 </div>
2354 |                 <div>
2355 |                     <h4 className="text-sm font-bold text-white leading-none">PowerPoint Slide
Deck</h4>
2356 |                     <span className="text-[10px] text-slate-500 block mt-2">True PPTX with speaker
notes embedded</span>
2357 |                 </div>
2358 |
2359 |                 <button
2360 |                     onClick={handleDownloadPptx}
2361 |                     className="w-full py-3 bg-gradient-to-r from-sky-500 to-indigo-500 rounded-xl
text-xs font-semibold shadow-xl shadow-sky-500/20 active:scale-95 transition-all flex items-center
justify-center gap-2"
2362 |                 >
2363 |                     <Download className="h-4 w-4" />
2364 |                     <span>Download PowerPoint Deck</span>
2365 |                 </button>
2366 |             </div>
2367 |
2368 |             {/* PDF Export Button Card */}
2369 |             <div className="glass-panel p-8 rounded-2xl border-white/5 hover:border-sky-500/20
hover:scale-[1.02] transition-all flex flex-col items-center gap-5 justify-between">
2370 |                 <div className="p-4 bg-rose-500/10 rounded-full border border-rose-500/20">
2371 |                     <FileText className="h-8 w-8 text-rose-400" />

```

```

2372 |         </div>
2373 |         <div>
2374 |             <h4 className="text-sm font-bold text-white leading-none">PDF Syllabus
Package</h4>
2375 |             <span className="text-[10px] text-slate-500 block mt-2">Perfect to print,
distribute or read offline</span>
2376 |         </div>
2377 |
2378 |         <button
2379 |             onClick={handleDownloadPdf}
2380 |             className="w-full py-3 bg-slate-900 border border-slate-800 text-slate-200
rounded-xl text-xs font-semibold hover:bg-slate-800 hover:border-slate-700 active:scale-95 transition-all flex
items-center justify-center gap-2"
2381 |             >
2382 |                 <Download className="h-4 w-4" />
2383 |                 <span>Download PDF Package</span>
2384 |             </button>
2385 |         </div>
2386 |
2387 |     </div>
2388 |
2389 | </div>
2390 | })
2391 |
2392 | </div>
2393 |
2394 | {/* C. POPUP FLOATING LIVE LOG MONITOR (AGENT GRAPH EXECUTING) */}
2395 | {isUploading && (
2396 |     <div className="absolute inset-0 bg-slate-950/80 backdrop-blur-md z-50 flex items-center
justify-center p-6">
2397 |         <div className="glass-panel p-8 rounded-2xl border-white/10 max-w-4xl w-full flex
flex-col gap-6 shadow-2xl bg-slate-950 text-left">
2398 |
2399 |             {/* Title bar */}
2400 |             <div className="flex justify-between items-center border-b border-slate-900 pb-4">
2401 |                 <div className="flex items-center gap-2.5">
2402 |                     <div className="h-2.5 w-2.5 rounded-full bg-sky-400 animate-ping shrink-0" />
2403 |                     <span className="font-extrabold text-base text-white tracking-tight">KALYX Agentic
Orchestration Pipeline</span>
2404 |                 </div>
2405 |                 <span className="text-[10px] font-mono text-slate-500">[ compiled:
compiled_graph.invoke() ]</span>
2406 |             </div>
2407 |
2408 |             {/* Split Screen layout */}
2409 |             <div className="grid grid-cols-1 lg:grid-cols-5 gap-6 overflow-hidden">
2410 |
2411 |                 {/* Left: Real-time Agent Timeline */}
2412 |                 <div className="lg:col-span-3 flex flex-col gap-4 max-h-[420px] overflow-y-auto pr-2
relative">
2413 |
2414 |                     {/* Vertical line connecting nodes */}
2415 |                     <div className="absolute left-[19px] top-6 bottom-6 w-[2px] bg-dashed border-l
border-slate-900 border-dashed z-0" />
2416 |
2417 |                     {pipelineAgents.map((agent, idx) => {
2418 |                         let dotColor = "bg-slate-900 border-slate-800";
2419 |                         let statusText = "Pending";
2420 |                         let badgeStyle = "text-slate-500 border-slate-900 bg-slate-950/40";
2421 |
2422 |                         if (agent.status === "completed") {
2423 |                             dotColor = "bg-emerald-500 border-emerald-400/20
shadow-[0_0_8px_rgba(16,185,129,0.3)]";
2424 |                             statusText = "Completed";
2425 |                             badgeStyle = "text-emerald-400 border-emerald-500/20 bg-emerald-500/10";
2426 |                         } else if (agent.status === "running") {

```

```

animate-pulse";
2427 |                 statusText = "Running";
2428 |                 badgeStyle = "text-sky-400 border-sky-500/20 bg-sky-500/10 animate-pulse";
2429 |             } else if (agent.status === "failed") {
2430 |                 dotColor = "bg-rose-500 border-rose-400/20 shadow-[0_0_8px_rgba(244,63,94,0.3)]
animate-ping";
2431 |                 statusText = "Failed";
2432 |                 badgeStyle = "text-rose-400 border-rose-500/20 bg-rose-500/10";
2433 |             }
2434 |
2435 |             return (
2436 |                 <div key={idx} className="flex gap-3.5 items-start relative z-10">
2437 |                     {/* Dot indicator */}
2438 |                     <div className={`h-6.5 w-6.5 rounded-full border flex items-center
justify-center shrink-0 transition-all duration-300 ${dotColor}`}>
2439 |                         {agent.status === "completed" ? (
2440 |                             <CheckCircle2 className="h-3.5 w-3.5 text-white" />
2441 |                         ) : agent.status === "running" ? (
2442 |                             <div className="h-1.5 w-1.5 rounded-full bg-white animate-ping" />
2443 |                         ) : agent.status === "failed" ? (
2444 |                             <AlertCircle className="h-3.5 w-3.5 text-white" />
2445 |                         ) : (
2446 |                             <span className="text-[9px] text-slate-600 font-bold">{idx + 1}</span>
2447 |                         )}
2448 |                     </div>
2449 |
2450 |                     {/* Card summary */}
2451 |                     <div className="flex-1 bg-slate-900/40 border border-slate-900/60 p-2.5
rounded-xl flex items-center justify-between gap-3">
2452 |                         <div className="overflow-hidden">
2453 |                             <h5 className="text-[11px] font-bold text-white
leading-none">{agent.name}</h5>
2454 |                             <span className="text-[9px] text-slate-500 block truncate
mt-1">{agent.purpose}</span>
2455 |                         </div>
2456 |                         <span className={`px-2 py-0.5 rounded-full border text-[8px] font-black
uppercase shrink-0 ${badgeStyle}`}>
2457 |                             {statusText}
2458 |                         </span>
2459 |                     </div>
2460 |                 </div>
2461 |             );
2462 |         }}}
2463 |     </div>
2464 |
2465 |     {/* Right: scrolling terminal logs */}
2466 |     <div className="lg:col-span-2 flex flex-col gap-3 overflow-hidden">
2467 |         <span className="text-[10px] font-extrabold text-slate-500 tracking-wider">CONSOLE
OUTPUT LOGS</span>
2468 |         <div className="bg-slate-950 border border-slate-900 rounded-xl p-4 font-mono
text-[9px] text-slate-300 h-[380px] overflow-y-auto flex flex-col gap-2">
2469 |             {uploadLogs.map((log, lIdx) => (
2470 |                 <div key={lIdx} className="leading-relaxed break-words">
2471 |                     <span className="text-sky-400 mr-1.5">&gt;</span>
2472 |                     {log}
2473 |                 </div>
2474 |             ))}
2475 |             <div className="h-2" />
2476 |         </div>
2477 |     </div>
2478 |
2479 | </div>
2480 |
2481 | <span className="text-[9.5px] text-slate-500 leading-relaxed text-center block border-t
border-slate-900 pt-4">
2482 |     Evaluating Bloom Taxonomy layers, sliding presentation bullets, and computing

```

```

assessment matrix in SQLite.
2483 |         </span>
2484 |     </div>
2485 | </div>
2486 |     })
2487 |
2488 | </main>
2489 |
2490 |     { /* D. POPUP MODAL FOR NEW COURSE INITIATING */ }
2491 |     { showCreateModal && (
2492 |         <div className="absolute inset-0 bg-slate-950/80 backdrop-blur-sm z-40 flex items-center
justify-center p-6">
2493 |             <div className="glass-panel p-6 rounded-2xl border-white/10 max-w-md w-full flex flex-col
gap-5 text-left bg-slate-950">
2494 |                 <h3 className="text-base font-bold text-white">Create New Course Package</h3>
2495 |
2496 |                 <div className="flex flex-col gap-4 text-xs">
2497 |                     <div>
2498 |                         <label className="text-[10px] text-slate-500 font-bold block mb-1">COURSE
TITLE</label>
2499 |                         <input
2500 |                             type="text"
2501 |                             placeholder="e.g. Advanced Machine Learning"
2502 |                             value={newCourseTitle}
2503 |                             onChange={(e) => setNewCourseTitle(e.target.value)}
2504 |                             className="w-full bg-slate-900 border border-slate-800 rounded px-3 py-2.5
text-slate-200 focus:outline-none focus:border-sky-500/40"
2505 |                         />
2506 |                     </div>
2507 |
2508 |                     <div>
2509 |                         <label className="text-[10px] text-slate-500 font-bold block mb-1">COURSE
DESCRIPTION</label>
2510 |                         <textarea
2511 |                             placeholder="Enter course scope, targets, prerequisites..."
2512 |                             value={newCourseDesc}
2513 |                             onChange={(e) => setNewCourseDesc(e.target.value)}
2514 |                             rows={3}
2515 |                             className="w-full bg-slate-900 border border-slate-800 rounded px-3 py-2.5
text-slate-200 focus:outline-none focus:border-sky-500/40"
2516 |                         />
2517 |                     </div>
2518 |                 </div>
2519 |
2520 |                 <div className="flex gap-3 justify-end text-xs font-semibold">
2521 |                     <button
2522 |                         onClick={() => setShowCreateModal(false)}
2523 |                         className="px-4 py-2 border border-slate-800 text-slate-400 hover:text-white
rounded-lg"
2524 |                     >
2525 |                         Cancel
2526 |                     </button>
2527 |                     <button
2528 |                         onClick={handleCreateCourse}
2529 |                         className="px-4 py-2 bg-gradient-to-r from-sky-500 to-indigo-500 rounded-lg
text-white"
2530 |                     >
2531 |                         Create Package
2532 |                     </button>
2533 |                 </div>
2534 |             </div>
2535 |         </div>
2536 |     })
2537 |
2538 | </div>
2539 | })

```

```
2540 |  
2541 |     </div>  
2542 | );  
2543 | }  
2544 |
```